

# Game Changer: A Modular Framework for OPRF Security

Karla Friedrichs<sup>✉</sup>, Anja Lehmann<sup>✉</sup>, and Cavit Özbay<sup>✉</sup>

Hasso Plattner Institute, University of Potsdam, Germany  
{karla.friedrichs,anja.lehmann,cavit.oezbay}@hpi.de

**Abstract.** Oblivious pseudorandom functions (OPRFs) allow the blind evaluation of a pseudorandom function, which makes them a versatile building block that enjoys usage in numerous applications. So far, security of OPRFs is predominantly captured in the Universal Composability (UC) framework, where an ideal functionality covers the expected security and privacy properties. While the OPRF functionality appears intuitive at first, the ideal-world paradigm also comes with a number of challenges: from imposing idealized building blocks when building OPRFs, to the lack of modularity, and requiring intricate UC knowledge to securely maneuver their usage. Game-based definitions are a simpler way to cover security properties. They model each property in a single game, which grants modularity in formalizing, proving, and using OPRFs. Interestingly, the few game-based works on OPRFs each re-invent the security model, with considerable variation. Thus, the advantages of the game-based approach remain out of reach: definitions are not easily accessible and comparability across works is low. In this work, we therefore systematize all existing notions into a clear, hierarchical framework. We unify or separate properties, making hidden relations explicit. This effort reveals the necessity of two novel properties: an intermediate privacy notion and a stronger unpredictability notion. Finally, we analyze the two most prominent constructions in our framework: HashDH and 2HashDH. The former does not achieve UC security, but has advantages in applications that require key rotation or updatability; yet it lacks a security analysis. We show that it achieves most security properties in our framework. We also observe that HashDH and 2HashDH do not satisfy our strongest privacy notion, indicating that the guarantees by the UC functionality are not as well understood as we might expect them to be. Overall, we hope that our framework facilitates the usage and design of OPRFs.

**Keywords:** OPRFs · provable security · game-based model · HashDH

## 1 Introduction

Oblivious pseudorandom functions (OPRFs) [FIPR05, CHL22] allow two parties, an evaluator holding a secret key  $k$  and requester holding a private input  $x$ , to jointly compute a pseudorandom value  $y = F(k, x)$ . The obliviousness thereby guarantees that the evaluator does not learn anything about the input  $x$  nor

output  $y$  for which the PRF is computed. At the same time, the requester can only learn as many PRF values as the evaluator agrees to. These properties make the OPRF a powerful building block, and they enjoy wide-spread usage in privacy-preserving protocols (e.g., [HL08, JL09, JL10, FHV13]) and protocols for the secure key management and user authentication (e.g., [JKK14, CLN15, JKKX17, BFH<sup>+</sup>20, DHL22, FHH<sup>+</sup>24]). The two most prominent applications are Privacy Pass [DGS<sup>+</sup>18], ensuring privacy-preserving access control through blindly issued one-time tokens, and OPAQUE [JKX18], the first strongly secure asymmetric PAKE. Both are standardized in the IETF [CDVW24, BKLW24], and also the underlying OPRF – 2HashDH – is part of an IRTF RFC [DFHSW23] that covers practical instantiations of oblivious PRFs from prime-order groups.

**OPRF Functionalities.** The most established way to define the security of OPRFs is through modelling their behavior as an ideal functionality  $\mathcal{F}_{\text{OPRF}}$  in the Universal Composability (UC) framework [Can01] (e.g., [JKK14, JKKX16, BDFH25, BBC<sup>+</sup>24]). The ideal OPRF functionality receives all private inputs and then internally enforces the desired behavior. Here, this desired functionality would be a consistent mapping from arbitrary inputs to pseudorandom outputs, where the evaluator only learns that a query was made, but nothing about the in- or output. Thus, the ideal world makes capturing the combination of (perfect) privacy and consistency trivial. When designing protocols relying on OPRFs, one uses  $\mathcal{F}_{\text{OPRF}}$  instead of the concrete instantiation, thereby abstracting from any particular OPRF design choice. In theory, UC promises a simple usage with strong guarantees under arbitrary composition. In practice, however, this approach comes with a number of challenges, outlined in the following.

*Multitude of Monolithic Functionalities.* There is no single, standardized definition for  $\mathcal{F}_{\text{OPRF}}$  – in fact, many applications introduce tailored ideal functionalities for their protocols (e.g., [JKK14, JKKX16, JKKX17, JKX18, BFH<sup>+</sup>20, BKLW20, CDCGS19, KMP<sup>+</sup>17, KCM24]). Consequently, security proofs must be redone from scratch, even when two OPRF instantiations are very similar. The tailored design and the monolithic nature of ideal functionalities also make a comparison of their implied guarantees challenging.

*(Too) Strong Requirements.* Further, realizing UC functionalities usually requires idealized primitives to provide the extractability and simulatability needed for the security proof. For OPRFs, this is easily achieved by using the 2HashDH construction  $F(k, x) = H_2(x, H_1(x)^k)$ , where the outer hash function provides both capabilities in the random oracle model. However, this outer hash takes away any “direct access” to the PRF key  $k$ , and protocols that require such access, e.g., for key rotation [ECS<sup>+</sup>15] or key homomorphic properties [Leh19], had to revert to HashDH and ad-hoc defined security guarantees.

*Functionalities Abstract All Crypto.* UC functionalities can be too abstract, in that they do not expose cryptographic values. For  $\mathcal{F}_{\text{OPRF}}$ , there are no cryptographic messages sent or received by the evaluator, which prevents protocols

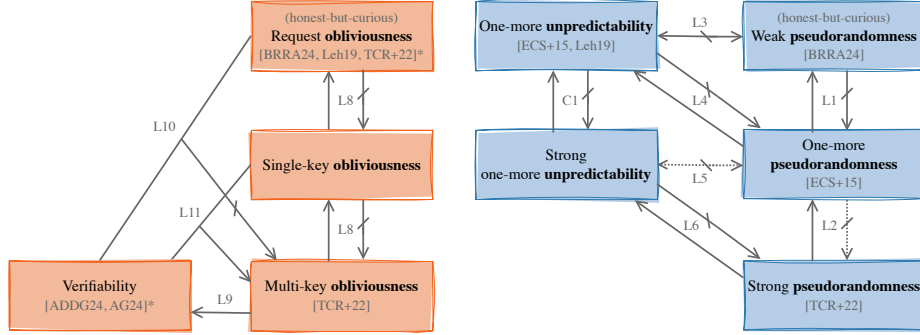
from incorporating these values into a larger design, ultimately hindering modularity. For instance, Faller et al. [FHH<sup>+</sup>24] require the output of an OPRF to be signed by a server – but since no evaluator output is modeled, the authors resort to using 2HashDH instead of a substitutable  $\mathcal{F}_{\text{OPRF}}$ . Consequently, their proof cannot rely on any of the ideal properties. Similarly, Backendal et al. [BDG<sup>+</sup>24] build 2HashDH as the only non-generic component into their cloud encryption protocol because their security proof requires the outer random oracle.

*The UC Divide.* Finally, UC proofs and functionalities often remain inaccessible to a large part of the cryptographic community, especially its more applied side. Quoting Bellare et al. [BRRA24]: “There is a divide, in the cryptographic community, between those who speak and use the language of UC, and those who don’t.” In fact, even the eponymous property of composability has its pitfalls: It only holds if the composed building blocks are fully independent. For instance, the OPAQUE protocol was proven secure for a single user only, relying on UC self-composition for scaling the protocol to multiple users. This requires using independent OPRF instances for all users. In the OPAQUE IRTF draft [BKLW24], however, re-using related OPRF keys was suggested, which not only invalidates the proven security guarantees but can also lead to actual attacks [DL24].

**Game-Based OPRF Definitions.** Game-based definitions offer a more established and accessible approach for modelling and applying security guarantees. Each security game addresses a particular property where an attacker with a set of abilities tries to perform a particular attack. A construction is deemed secure if the game cannot be beaten by any efficient attacker. Splitting the model into such elementary properties allows to compose an analysis for an arbitrary set of aspects. Especially compared to UC ideal functionalities, this facilitates reusable protocols by breaking down complex proofs into manageable steps and allowing for easy comparison of achieved properties across constructions. Furthermore, the game-based approach requires a concrete, algorithmic representation of the construction, exposing key material and cryptographic components – which enhances modularity in protocol design compared to abstract UC interfaces.

*In Search of a Standard Definition.* While there is some previous work on security games for OPRFs [ECS<sup>+</sup>15, Leh19, TCR<sup>+</sup>22, BRRA24, AG24, ADDG24], contributions are overall scarce. Strikingly, instead of an agreed-upon definition, each work comes up with their own syntax and properties of OPRFs, often tailored to a specific construction. To a wider audience, it is desirable to have definitions readily available, especially considering that OPRFs are often just one building block in larger applications. Unfortunately, at the time of writing, some definitions overlap or are equivalent, others use strikingly different approaches to express the same property. Being aware of the variants and selecting one becomes a tedious task. This issue even seems to extend to more expert readers: looking ahead, some property relations turn out intricate and non-trivial to prove.

Overall, in the current state of research, the advantages of game-based over UC definitions cannot take effect: Accessibility of models is low, the proven se-



**Fig. 1.** Overview of our definitional framework. Here we cite the definitions we adapted or modified (\*), more related works are discussed in the corresponding section. Arrows are implications, barred arrows are separations shown in our work, labelled with the corresponding lemma or corollary. Relations with a dotted line require additional assumptions. All lemmas are new except for L4, which has been stated before by [ECS<sup>+</sup>15], and L8 which was informally treated by [TCR<sup>+</sup>22].

curity from different works is hard to compare and protocols cannot be designed in a modular way as long as the modules lack clear relations.

### 1.1 Our Contributions

In this work we propose a hierarchical framework for game-based security of OPRFs, systematizing and extending the scattered definitional landscape. Our framework provides a new formal foundation for this powerful primitive, and discovers a number of gaps and challenges when dissecting the concrete properties of OPRFs. An overview of all definitions and shown relations is depicted in Fig. 1. Our game-based approach enables to capture the different levels of security and privacy, and their necessity when designing protocols. This provides a more granular understanding of OPRFs and hopefully simplifies the quest of finding quantum-safe instantiations, which still proves to be a challenge when aiming at full UC security [BKW20, ADDS21, Bas23, BDFH25]. We also analyze the two most prominent OPRF instantiations, HashDH and 2HashDH, particularly focusing on the security properties HashDH can provide without the burden of UC security. In more detail, our work comprises the following contributions.

*Security: Unpredictability and Pseudorandomness.* The intuitive security properties expected from OPRFs are *unpredictability* and *pseudorandomness*. For the former, we first recap the existing notion of *one-more unpredictability*. We stress that this sole available formalization is insufficient for one of the most prominent applications of OPRFs, Privacy Pass: therein, OPRF outputs are employed as access tokens, essentially providing a verification oracle to users. Therefore, we propose *strong one-more unpredictability* that reflects this adversarial ability. Looking ahead, the property also turns out necessary in a generic compiler for

strongly pseudorandom OPRFs. Interestingly, unpredictability is a notion that has not been captured as a stand-alone goal within the UC formalization, and our game-based definition now provides an easier target for OPRFs that are used for such applications.

Pseudorandomness is surprisingly tricky to define due to the blind nature of OPRFs. We restate and adapt three type of definitions that have been proposed in the literature before: *weak*, *one-more* and *strong pseudorandomness*. Weak pseudorandomness is the simplest notion, but is limited to honest-but-curious requesters. The two other notions allow for a malicious requester, but have their own trade-offs: The strongest one reverts to a simulation-based game, and thus likely relies strongly on idealized building blocks in the OPRF construction. One-more pseudorandomness on the other hand requires more tailored hardness assumptions in the security proofs.

*Privacy: Obliviousness.* For privacy, the expected property is *obliviousness*, meaning that the evaluator must not learn anything about the values it blindly computed the PRF for. We capture this privacy property through three obliviousness definitions, with increasing strength. Our first notion *request obliviousness* assumes an honest-but-curious evaluator and implies the three obliviousness notions proposed before [Leh19, TCR<sup>+</sup>22, BRRA24], albeit being simpler than any of the existing models. A stronger notion with an actively malicious evaluator, *multi-key obliviousness*, is also found in the literature [TCR<sup>+</sup>22]. The challenge thereby is how to prevent trivial wins, when the adversary evaluates different requests with different public keys. As OPRFs are classically secret-key primitives, they typically do not come with public verifiability. And in fact, the strongest notion multi-key obliviousness requires some form of *implicit verifiability* that allows the requester to check that they received a correctly computed response. We formally define this verifiability property and show that it is implied by multi-key obliviousness. While prior work [TCR<sup>+</sup>22] successfully builds a multi-key oblivious OPRF by adding implicit verifiability to a request-oblivious construction, it has gone unnoticed that formally, something stronger than request obliviousness is needed for the base construction. Therefore, we introduce an intermediate obliviousness property called *single-key obliviousness*.

*Security Analysis of (2)HashDH.* To showcase our framework, we conduct a security analysis of the ubiquitously used OPRFs HashDH and 2HashDH. HashDH does not achieve UC security, but has the advantage of preserving the group structure, thus enabling features such as key rotation or key homomorphism. We present the first formal analysis of HashDH and show that this simpler construction is still surprisingly strong, as it achieves single-key obliviousness on the privacy side, and strong one-more unpredictability as well as one-more pseudorandomness on the security side. We also provide evidence that HashDH *cannot* satisfy strong pseudorandomness, by proving that such a security proof implies breaking a natural CDH-related assumption. This stronger pseudorandomness notion is achieved by 2HashDH, and we show that applying an outer hash function generally lifts strong one-more unpredictability to strong pseudorandom-

ness, adapting a similar result recently shown by Beullens et al. [BDFH25] in the UC framework.

We also show that neither HashDH nor 2HashDH satisfy our strongest privacy notion of multi-key obliviousness, as they do not come with any verifiability. While this might not be surprising per se, it does show that the UC definitions might not be as well-understood as we would expect them to be. Therein, privacy is often guaranteed towards malicious evaluators, too, indicating equivalence with our strongest privacy notion, yet the 2HashDH construction is used which does not satisfy this property. We show that multi-key obliviousness can be achieved for (2)HashDH via a simple zero-knowledge proof.

*Comparison with Related Work.* Our work performs a thorough survey of existing game-based OPRF definitions [ECS<sup>+</sup>15, Leh19, TCR<sup>+</sup>22, BRRA24, AG24, ADDG24], making implicit knowledge explicit. We relate the final framework to these previous works in depth in the respective section for each property. For a high-level summary of the related work we refer to Appendix A. The focus of our work is to provide a simple and well-analyzed framework for the core security and privacy properties of OPRFs. Extensions to cover more functionality, such as partial obliviousness or threshold evaluation, as well as a formal study to compare our framework to the UC definitions are beyond the scope of our work, and interesting open problems.

## 2 Preliminaries

This section introduces the writing conventions and building blocks used in the rest of the work.

*Game-based Models.* We follow Shoup’s [Sho04] formalization for game-based models, and use Bellare and Rogaway’s [BR04] notation to indicate failure events via a flag `bad`. We make some further simplifications to reduce syntactic clutter: Adversaries are expected to respect domains with their inputs. Variables and public parameters are initialized at the beginning of a definition, and assumed to be accessible as a shared game state. Finally, we use  $\Pr[\text{Game}^X(\lambda)]$  as a standard shorthand for the event that game  $\text{Game}^X$  terminates with output 1.

*Pseudorandom Functions.* A PRF is a function  $F : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ . The computation of  $F$  is efficient and deterministic, and *pseudorandomness* ensures that without knowledge of the key the output is indistinguishable from random.

*Zero-Knowledge Proofs (ZKPs).* They allow to prove statements without disclosing anything else than the fact that the statement is true. We use the Camenisch and Stadler notation [CS97] to denote non-interactive zero-knowledge proofs (NIZK). Therein, proof statements are written in the form of

$$\pi \leftarrow \$ \text{NIZK}\{(\alpha, \beta, \gamma) : y = g^\alpha h^\beta \wedge \tilde{y} = \tilde{g}^\alpha \tilde{h}^\gamma\}(ctx)$$

which expresses a zero-knowledge proof of secret values  $\alpha, \beta, \gamma$  for the well-formedness of  $y, \tilde{y}$  w.r.t. the group elements  $g, h, \tilde{g}, \tilde{h}$ . The proof can additionally be bound to a context message  $ctx$ , which usually comprises all public values the proof should be bound to. Any proof written in such an abstract form can be easily realized through a Schnorr-based instantiation via the Fiat-Shamir transform. We require the NIZK to be *correct*, *zero-knowledge* and *sound*.

*DH-based Assumptions.* The security of the (2)HashDH OPRF constructions is based on several CDH and DDH assumptions we state informally in the following. We consider hardness relative to a group generation algorithm  $\mathsf{GGen}$ , from which initially a prime order group  $(\mathbb{G}, q, g)$  of order  $q$  and with generator  $g$  is sampled and given to the adversary. Appendix D presents the assumptions in detail and discusses their relations to standard assumptions.

- (Gap-)OM-CDH:** Given  $g^x$  for  $x \leftarrow \$ \mathbb{Z}_q$ , random challenges  $Y_i \leftarrow \$ \mathbb{G}$  and  $\ell - 1$  “hints”  $Y_i^x$ , it is computationally infeasible for an efficient adversary to output  $Y_i^x$  for  $\ell$  challenges (even with access to a DDH oracle that upon an input  $A, B, C$  returns 1 if  $\log_g(C) = \log_g(A) \cdot \log_g(B)$ ).
- Hint-CDH:** Given a challenge  $Y^* \leftarrow \$ \mathbb{G}$  and a hint  $Y^x$  for a  $x \leftarrow \$ \mathbb{Z}_q$  and an adversary-chosen base  $Y \in \mathbb{G}$ , it is computationally infeasible for an efficient adversary to output  $(Y^*)^x$ .
- DDH-Mult:** Given  $g^x$  for  $x \leftarrow \$ \mathbb{Z}_q$  and multiple challenges of either two random elements  $(Y, Z \leftarrow \$ \mathbb{G})$ , or  $(Y \leftarrow \$ \mathbb{G}, Y^x)$ , depending on a global  $b \leftarrow \$ \{0, 1\}$ , it is computationally infeasible for an efficient adversary to guess  $b$ .
- OM-DDH:** Given  $g^x$  for  $x \leftarrow \$ \mathbb{Z}_q$  and  $\ell - 1$  queries to a CDH oracle that upon input  $Y$  returns  $Y^x$ , it is computationally infeasible for an efficient adversary to guess  $b_i$  for  $\ell$  challenges of either two random elements  $(Y, Z \leftarrow \$ \mathbb{G})$ , or  $(Y \leftarrow \$ \mathbb{G}, Y^x)$ , where each challenge randomly takes one of the two cases.

### 3 OPRF Syntax

This section presents a syntax of OPRFs, for which we state formal definitions of the core properties in the following two sections. An OPRF is an interactive protocol run between two parties: the *requester* and the *evaluator*. The evaluator holds a PRF secret key  $sk$ . The requester wants to learn the evaluation of the PRF,  $y \leftarrow \mathsf{Eval}(sk, x)$ , for an input  $x$  without revealing any information about  $x$  or  $y$  to the evaluator.

*Setup and KGen.* Before the protocol, a **Setup** algorithm generates the public parameters  $pp$ . The evaluator then generates its key pair as  $(pk, sk) \leftarrow \$ \mathsf{KGen}(pp)$ . Thus, we provide our definition in a *public-key* setting. The public key is required for an implicit form of *verifiability*, allowing the requester to verify whether a blind evaluation was performed honestly. It can simply be set to  $pk = \perp$  when this feature is not needed. However, this verifiability is in fact necessary to obtain any reasonably strong form of privacy for the requester as proven for our framework in Section 5.3.

(*Blind*) *Evaluation*. We make the standard PRF evaluation  $y \leftarrow \text{Eval}(sk, x)$  explicit in our syntax, following [ECS<sup>+</sup>15, Leh19, TCR<sup>+</sup>22, ADDG24], as this will be helpful in the definition of the security model.

The core of the OPRF is the interactive evaluation protocol, and we formalize the round-optimal version which consists of three algorithms **Request**, **BlindEval**, and **Finalize**. Initially, the requester runs  $(\bar{x}, st) \leftarrow \text{Request}(pk, x)$  to obtain a blinded version  $\bar{x}$  of the value  $x$  it wants to query the PRF on. The algorithm also outputs a state  $st$  which will be needed to unblind the response. The requester then sends  $\bar{x}$  to the evaluator who returns  $\bar{y} \leftarrow \text{BlindEval}(sk, \bar{x})$ . Finally, the requester can obtain the unblinded response by running  $y \leftarrow \text{Finalize}(\bar{y}, st)$ . Correctness requires that  $y \leftarrow \text{Eval}(sk, x)$ . A visual overview of the exchange is given in Appendix B.

Note that we already include  $pk$  in the **Request** algorithm, as this is required in some lattice-based schemes, e.g., [ADDG24]. We do not provide  $pk$  again in **Finalize** since it can be passed into the **Finalize** algorithm as part of the state  $st$ . Our modelling also ensures that both algorithms are invoked on the same key.

More formally, an OPRF is defined as follows:

**Definition 1 (OPRF Scheme).** *An oblivious pseudorandom function for input space  $\mathcal{X}$  is a scheme  $\Pi$  with the following algorithms:*

- Setup**( $\lambda$ )  $\rightarrow pp$  : *On input of a security parameter  $\lambda$  generates public parameters  $pp$ . We make  $pp$  only explicit in **KGen**, and assume it as an implicit input in all other algorithms.*
- KGen**( $pp$ )  $\rightarrow (pk, sk)$  : *On input of the public parameters  $pp$  outputs the PRF key  $sk$  and a public verification key  $pk$ .*
- Eval**( $sk, x$ )  $\rightarrow y$  : *Evaluates the underlying PRF non-interactively for secret key  $sk$  and input  $x$ .*
- Request**( $pk, x$ )  $\rightarrow (\bar{x}, st)$  : *Prepares a blind request for input  $x$  to be evaluated by the  $sk$  belonging to  $pk$ , outputs a blinded input  $\bar{x}$  and state  $st$ .*
- BlindEval**( $sk, \bar{x}$ )  $\rightarrow \bar{y}$  : *Evaluates the blind request  $\bar{x}$  with secret key  $sk$ , returning the blind response  $\bar{y}$ .*
- Finalize**( $\bar{y}, st$ )  $\rightarrow y/\perp$  : *Unblinds the blind response  $\bar{y}$  using the state  $st$  (which contains  $pk$  in the implicitly verifiable case). It returns the PRF evaluation  $y$  if it succeeds. Otherwise, it returns  $\perp$ .*

*Correctness.* Correctness provides a guarantee for *honest* execution of the protocol, i.e., honest executions of the oblivious and standard evaluation to have identical results, and is defined in Appendix C.

## 4 Security of OPRFs

This section models the core security properties of an OPRF: unpredictability and pseudorandomness. Unpredictability demands that the requester cannot learn more OPRF evaluations than the number of interactions with the evaluator. This property is sufficient in applications such as Privacy Pass [DGS<sup>+</sup>18],



where requesters run the protocol to obtain one-time tokens they can redeem later for accessing a service. In contrast, applications using the OPRF output as a cryptographic key require the stronger notion of pseudorandomness. While the property is trivial to define for standard PRFs, the blind nature of OPRFs makes this surprisingly challenging in the game-based manner. Our work recaps different ways to address this challenge and analyzes their relation.

We present two variants for unpredictability notions: the known *one-more unpredictability* and a new *strong one-more unpredictability*, that will be necessary for applications that implicitly provide a *sk*-based verification check, such as Privacy Pass. For pseudorandomness we restate and adapt three types of definitions that have been proposed in the literature before: *weak*, *one-more* and *strong pseudorandomness*. Weak pseudorandomness assumes an honest-but-curious requester, while the other two accommodate malicious requesters with different trade-offs in their game design. Finally, we present a thorough analysis of the relations among this set of different notions, as depicted in Fig. 1.

#### 4.1 Unpredictability

Unpredictability seems to be the most well-understood property, with the works by Everspaugh et al. [ECS<sup>+</sup>15] and Lehmann [Leh19] proposing very similar definitions for the variant of *one-more unpredictability*. It clearly resembles one-more unforgeability for blind signatures, but while unforgeability asks the adversary to produce a signature that *validates*, we ask for the unique value that is *correct*.

We define two types of unpredictability: The first is standard one-more unpredictability [ECS<sup>+</sup>15, Leh19]. Interestingly, this definition – the only existing definition of unpredictability at the time of writing – is insufficient for one of the most prominent applications of OPRFs, Privacy Pass. Privacy Pass uses OPRF outputs as one-time access tickets to a service, which implicitly allows users to check the validity of a PRF tuple  $(x, y)$ . This ability is not captured in one-more unpredictability, so our proposed second definition grants the adversary a *sk*-verification oracle. Looking ahead, the novel property will also be needed for our generic strongly pseudorandom OPRF compiler.

We state both security games in Fig. 2, and define unpredictability as follows:

**Definition 2 ((Strong) One-more Unpredictability).** *We call an OPRF protocol  $\Pi$  (strongly) one-more unpredictable if  $\forall$  PPT adversaries  $\mathcal{A}$  the following advantage is negligible:*

$$\text{Adv}_{\Pi, \mathcal{A}}^{(\text{str-})\text{unpred}}(\lambda) := \Pr \left[ \text{Game}_{\Pi, \mathcal{A}}^{(\text{str-})\text{unpred}}(\lambda) \right]$$

**One-more unpredictability (unpred).** Intuitively, the goal is to ensure that an adversary cannot predict a fresh OPRF evaluation without the help of the evaluator. The adversary may interact with the evaluator multiple times to learn some evaluations, but must not win by reusing these evaluations. As in blind signature unforgeability, this is enforced via a one-more style game: The adversary must produce more evaluations than the number of interactions performed.

| $\text{Game}_{\Pi, \mathcal{A}}^{\text{unpred}}(\lambda), \text{Game}_{\Pi, \mathcal{A}}^{\text{str-unpred}}(\lambda)$           | $\mathcal{O}_{\text{BlindEval}}(\bar{x})$            |
|----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| $pp \leftarrow \text{Setup}(\lambda); (pk, sk) \leftarrow \text{KGen}(pp); c_{\text{BE}} := 0$                                   | $c_{\text{BE}} := c_{\text{BE}} + 1$                 |
| $((x_1, y_1), \dots, (x_\ell, y_\ell)) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{BlindEval}}, \mathcal{O}_{\text{Vf}}}(pp, pk)$ | <b>return</b> $\text{BlindEval}(sk, \bar{x})$        |
| <b>if</b> $\ell \leq c_{\text{BE}} \vee \exists \alpha \neq \beta : x_\alpha = x_\beta$ <b>return</b> 0                          | $\mathcal{O}_{\text{Vf}}(x, y)$                      |
| <b>return</b> $\forall \alpha : \text{Eval}(sk, x_\alpha) \stackrel{?}{=} y_\alpha$                                              | <b>return</b> $y \stackrel{?}{=} \text{Eval}(sk, x)$ |

**Fig. 2.** Security games for (strong) one-more unpredictability.

The basic unpredictability game first generates a key pair, which is used to answer any  $\mathcal{O}_{\text{BlindEval}}$  queries. The adversary finally outputs  $\ell$  input-output pairs, which must surpass the number of blind evaluations (counted by  $c_{\text{BE}}$ ) and be distinct.  $\mathcal{A}$  wins if all  $\ell$  evaluations are correct under the set key.

**Strong unpredictability (str-unpred).** In practical applications, malicious requesters might be given another capability we have not modeled so far: If an application prompts users to provide their OPRF evaluation, as, e.g., in Privacy Pass, then the application only accepts correct values and reject others. Thus, the application essentially provides a  $sk$ -based verification function, where the user can provide values  $x, y$  and learns whether  $y = \text{Eval}(sk, x)$ . As OPRFs implement a *deterministic* evaluation function, such verification can only be done in the canonical way, i.e. by computing  $y' \leftarrow \text{Eval}(sk, x)$  and checking whether  $y = y'$ .

Our strong unpredictability definition models this accordingly through an additional  $\text{Vf}$  oracle that takes an input-output pair  $x, y$  and returns a single bit signaling correctness, determined through canonical verification. The recent work by Beullens et al. [BDFH25] also introduces a  $(n, q)$ -unpredictability notion that comes with such a verification oracle. They keep their definition intentionally weak, though, to capture the minimal requirements needed for their 2Hash transform, whereas our notion yields a strong stand-alone property that will suffice for a broad class of applications.

Interestingly, this additional verification ability is not obsolete as it would be for related unforgeable primitives like signatures or MACs, which we show in Section 4.3. For some intuition of why one-more unpredictability does not trivially imply the strong version, observe that we cannot know what exactly an adversary learns from a response given out by  $\mathcal{O}_{\text{BlindEval}}$ . Consequently, we cannot simulate a verification oracle as we would for MACs by approving exactly the evaluations we know  $\mathcal{A}$  obtained during the game.

Adding a non-blind  $\text{Eval}$  oracle to the definition would not change its strength though, as we show in Appendix E.1, so we omit it.

**Unpredictability in the UC framework.** UC ideal functionalities typically assume the OPRF evaluations to be assigned through a random function. Thus the UC definition is aiming at the stronger pseudorandomness property, which implies unpredictability as it eliminates any prediction strategy besides guessing.

While (one-more) unpredictability is then implied by any UC secure construction, it takes away the possibility to aim at a construction that fails to achieve this strong notion, but still guarantees the weaker unpredictability guarantee. As we will see in Section 6.1, the HashDH construction is such an example.

Another notable observation is that, to the best of our knowledge, there is no ideal functionality that permits the requester to *sk*-verify input-output pairs, as we do in our strong unpredictability notion. This makes the existing UC definitions unsuitable for one of the most widely-used applications of OPRFs, Privacy Pass, which provides such verification.

## 4.2 Pseudorandomness

For stronger security, we may not only want evaluations to be unpredictable, but even hard to tell apart from truly random values. This property known as *pseudorandomness* adds versatility, as such OPRF evaluations may be used in place of random values in larger protocols.

Although pseudorandomness is well understood for standard PRFs, applying it to OPRFs surprisingly turned out a challenge. Recall that pseudorandomness for regular PRFs is defined roughly as follows: An adversary is given an oracle that either returns the PRF evaluation of an input or a truly random value. We call the former case the *real world* and the latter the *random world*. Introducing blind evaluations to this game, however, is non-trivial. Since we cannot observe what inputs an adversary hides in a blind request, a **BlindEval** oracle risks to be inconsistent with the random-world challenges output by the standard **Eval** oracle, making the two worlds easily distinguishable. There is no easy fix, and related works take different approaches to this problem, each with their own advantages and disadvantages. Looking ahead, a stronger attacker model is only addressed by either deviating from the above structure or resorting to a simulation-based definition.

We first present the weaker, but significantly simpler notion we call *weak pseudorandomness*, slightly modifying the approach of *OPRF-PR* by Bellare et al. [BRRA24] to use only a single evaluation key instead of many, for consistency within our framework. We then formalize *strong pseudorandomness*, for which we choose the *POPRF* notion from Tyagi et al. [TCR<sup>+</sup>22], and finally *one-more-pseudorandomness* as proposed by Everspaugh et al. [ECS<sup>+</sup>15].

The security games for all three definitions are given in Fig. 3, and we define the various levels of pseudorandomness as follows:

**Definition 3 (Pseudorandomness).** *We call an OPRF protocol  $\Pi$   $\mathbf{x}$ -pseudorandom for  $\mathbf{x} \in \{\text{weak}, \text{str}, \text{om}\}$ , if  $\forall$  PPT adversaries  $\mathcal{A}$ , there exists a simulator  $S = (S.\text{Setup}, S.\text{BlindEval}, S.\text{Prim})$  such that the following advantage is negligible:*

$$\text{Adv}_{\Pi, \mathcal{A}, S}^{\mathbf{x}\text{-prnd}}(\lambda) := 2 \cdot \Pr \left[ \text{Game}_{\Pi, \mathcal{A}, S}^{\mathbf{x}\text{-prnd}}(\lambda) \right] - 1$$

**Weak pseudorandomness (weak-prnd).** The weakest notion of pseudorandomness circumvents the challenge of dealing with blind evaluations by assuming

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{Game}_{\Pi, \mathcal{A}}^{\text{weak-prnd}}(\lambda)$<br>$pp \leftarrow \$ \text{Setup}(\lambda)$<br>$(pk, sk) \leftarrow \$ \text{KGen}(pp)$<br>$f \leftarrow \$ \text{RandFnGen}(pp)$<br>$E := []; \quad b \leftarrow \$ \{0, 1\}$<br>$b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Eval}}, \mathcal{O}_{\text{BlindEval}}}(pp, pk)$<br><b>return</b> $b' \stackrel{?}{=} b$                                                                                                                                                                                                                                                                               | $\mathcal{O}_{\text{Eval}}(x)$<br><b>if</b> $x \notin E$ :<br><b>if</b> $b = 0$ :   // random<br>$E[x] \leftarrow f(x)$<br><b>else</b> :   // real<br>$E[x] \leftarrow \text{Eval}(sk, x)$<br><b>return</b> $E[x]$                                                                                                                                        | $\mathcal{O}_{\text{BlindEval}}(x)$<br><b>if</b> $x \in E$ : <b>return</b> $\perp$<br>$\omega \leftarrow \$ \Omega$ // fix randomness<br>$(\bar{x}, st) \leftarrow \text{Request}(pk, x; \omega)$<br>$\bar{y} \leftarrow \$ \text{BlindEval}(sk, \bar{x})$<br>$E[x] \leftarrow \text{Finalize}(\bar{y}, st)$<br><b>return</b> $(\bar{x}, \bar{y}, E[x], \omega)$ |
| $\text{Game}_{\Pi, \mathcal{A}, \mathcal{S}}^{\text{str-prnd}}(\lambda)$<br>$pp \leftarrow \$ \text{Setup}(\lambda)$<br>$(sts, pk_0) \leftarrow \$ \text{S.Setup}(pp)$<br>$st_{\text{Prim}} \leftarrow \$ \text{Prim.Setup}(pp)$<br>$(pk_1, sk) \leftarrow \$ \text{KGen}(pp)$<br>$f \leftarrow \$ \text{RandFnGen}(pp)$<br>$b \leftarrow \$ \{0, 1\}; \quad c_{\text{BE}}, c_{\text{S}} := 0$<br>$b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Eval}}, \mathcal{O}_{\text{BlindEval}}, \mathcal{O}_{\text{Prim}}}(pp, pk_b)$<br><b>return</b> $b' \stackrel{?}{=} b$                                                                                               | $\mathcal{O}_{\text{BlindEval}}(\bar{x})$<br>$c_{\text{BE}} := c_{\text{BE}} + 1$<br><b>if</b> $b = 0$ :   // random<br>$(\bar{y}, st_{\text{S}}) \leftarrow \$ \text{S.BlindEval}^{\mathcal{O}_{\text{LimEval}}}(\bar{x}, st_{\text{S}})$<br><b>else</b> :   // real<br>$\bar{y} \leftarrow \$ \text{BlindEval}(sk, \bar{x})$<br><b>return</b> $\bar{y}$ | $\mathcal{O}_{\text{Prim}}(x)$<br><b>if</b> $b = 0$ :   // random<br>$(y, st_{\text{S}}) \leftarrow \$ \text{S.Prim}^{\mathcal{O}_{\text{LimEval}}}(x, st_{\text{S}})$<br><b>return</b> $y$<br><b>else</b> :   // real<br><b>return</b> $\text{Prim.Eval}(x, st_{\text{Prim}})$                                                                                  |
| $\mathcal{O}_{\text{Eval}}(x)$<br><b>if</b> $b = 0$ :   // random<br><b>return</b> $f(x)$<br><b>else</b> :   // real<br><b>return</b> $\text{Eval}(sk, x)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | $\mathcal{O}_{\text{LimEval}}(x)$<br>$c_{\text{S}} := c_{\text{S}} + 1$<br><b>if</b> $c_{\text{S}} \leq c_{\text{BE}}$ :<br><b>return</b> $\mathcal{O}_{\text{Eval}}(x)$<br><b>return</b> $\perp$                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                  |
| $\text{Game}_{\Pi, \mathcal{A}}^{\text{om-prnd}}(\lambda)$<br>$pp \leftarrow \$ \text{Setup}(\lambda); \quad (pk, sk) \leftarrow \$ \text{KGen}(pp)$<br>$f \leftarrow \$ \text{RandFnGen}(\lambda)$<br>$c_{\text{BE}} := 0; \quad \mathbf{b} := (); \quad \mathbf{C} := \emptyset$<br>$(i_1, \dots, i_{\ell}, b') \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Chal}}, \mathcal{O}_{\text{BlindEval}}}(pp, pk)$<br><b>if</b> $\exists \alpha : i_{\alpha} \geq  \mathbf{b}  \vee \ell \leq c_{\text{BE}} \vee \exists \alpha \neq \beta : i_{\alpha} = i_{\beta}$ :<br><b>return</b> 0<br><b>return</b> $b' \stackrel{?}{=} \bigoplus_{\alpha=1}^{\ell} b[i_{\alpha}]$ | $\mathcal{O}_{\text{Chal}}(x)$<br><b>if</b> $x \in \mathbf{C}$ : <b>return</b> $\perp$ ; <b>else</b> : $\mathbf{C} := \mathbf{C} \cup \{x\}$<br>$b \leftarrow \$ \{0, 1\}$<br>Append $b$ to $\mathbf{b}$<br><b>if</b> $b = 0$ : <b>return</b> $f(x)$<br><b>else</b> : <b>return</b> $\text{Eval}(sk, x)$                                                  | $\mathcal{O}_{\text{BlindEval}}(\bar{x})$<br>$c_{\text{BE}} := c_{\text{BE}} + 1$<br><b>return</b> $\text{BlindEval}(sk, \bar{x})$                                                                                                                                                                                                                               |

**Fig. 3. Top:** Security game for weak pseudorandomness. The adversary is passive. **Middle:** Security game for strong pseudorandomness. The adversary is active. **Bottom:** Security game for one-more pseudorandomness. The adversary is active.

an honest-but-curious attacker model for the requester. This is captured by forcing the adversary to send  $x$  in plain to the  $\mathcal{O}_{\text{BlindEval}}$ , and blinding happens within the oracle prior to the blind evaluation. Although this notion offers no security against misbehaving requesters, it gives the simplest model of pseudorandomness, and it has been used by Lehmann [Leh19], and Bellare et al. [BRRA24].

For weak pseudorandomness, we adapt Bellare et al.’s definition with small changes. The adversary has access to an **Eval** and a **BlindEval** oracle. The behavior of  $\mathcal{O}_{\text{Eval}}$  depends on a random bit  $b$ : It returns either PRF evaluations ( $b = 1$ ) or random values ( $b = 0$ ). For the latter, we assume there is an algorithm  $\text{RandFnGen}(\lambda)$  that randomly selects a function  $f$  with deterministic, but truly random outputs of matching size.

The blind evaluation oracle  $\mathcal{O}_{\text{BlindEval}}$  always runs the blind OPRF evaluation protocol, but for a plain input value  $x$ . To provide the same view as if the adversary was running the algorithm themselves, we first fix some randomness by sampling  $\omega \leftarrow \Omega$  and then apply the  $\omega$  to a probabilistic algorithm. Thus a probabilistic  $\text{Request}(pk, x)$  becomes a deterministic  $\text{Request}(pk, x; \omega)$  which can be re-computed identically if  $\omega$  is known.

The oracle blinds the input, evaluates the blinded input and finalizes it with the kept state. Finally, it outputs all values  $(\bar{x}, \bar{y}, y, \omega)$ , including the internal randomness  $\omega$ . (We show in Appendix E.2 that omitting the randomness or at least the blinding state  $st$  results indeed in a weaker definition.) As in Bellare et al., at the first query of some  $x$  at either oracle, one fix  $y$  is assigned for the whole game, preventing trivial wins. It is also important to note that the authors disallow re-querying inputs at  $\mathcal{O}_{\text{BlindEval}}$ . At the end of the game, the adversary outputs their guess  $b'$  and wins if  $b = b'$ .

Bellare et al. model their game for multiple evaluators as their work considers concrete security. We leave this feature out for consistency among our definitions, but one can prove that the two versions are equivalent via a hybrid argument.

**Strong pseudorandomness (str-prnd).** We now strengthen the attacker model by allowing an attacker to maliciously blind requests. When allowing blind requests, the adversary may send the same inputs to  $\mathcal{O}_{\text{Eval}}$  and  $\mathcal{O}_{\text{BlindEval}}$ . This is fine for the real world, since standard and blind evaluations are consistent in correct schemes. But in the random world,  $\mathcal{O}_{\text{Eval}}$  assigns truly random values to each input and in  $\mathcal{O}_{\text{BlindEval}}$ , facing potentially information-theoretic blinding, it seems impossible to consistently match the response to the same value.

Tyagi et al. [TCR<sup>+</sup>22] and Lehmann [Leh19] therefore resort to a simulation-based approach for their pseudorandomness property: The simulator takes care of blind evaluations in the random world and restores consistency later in the protocol, before the requester has finished finalizing the evaluation. Its exact strategy is highly OPRF-dependent, but the simulator obviously requires additional idealized powers such as programming a random oracle. We adopt Tyagi et al.’s game as *strong pseudorandomness* for our framework.

The overall structure of the game is similar to weak pseudorandomness and tasks the adversary to determine whether it is in the real or random world.

Now, the  $\mathcal{O}_{\text{BlindEval}}$  oracle does no longer always return real protocol values, but returns simulated ones via  $\text{S.BlindEval}$  in the random world, i.e. when  $b = 0$ . The simulator’s additional powers are modeled via an ideal primitive oracle  $\mathcal{O}_{\text{Prim}}$  that it controls in the random world. Clearly, to restore consistency, the simulator  $\text{S}$  also needs access to the non-blind evaluation, which is modeled by giving the  $\text{S}$  access to  $\mathcal{O}_{\text{LimEval}}$  model. Following Tyagi et al., this oracle is limited to one query per  $\mathcal{O}_{\text{BlindEval}}$  query that  $\text{S}$  processes. This controlled access guarantees that the simulator does not leak more than one evaluation per  $\text{BlindEval}$  call.

While this approach cleverly modifies the established pseudorandomness paradigm, it also inherits the disadvantages of simulation-based security. Security guarantees of a strongly pseudorandom OPRF now depend on a variable simulator and thus become less concrete. Moreover, as we will see later, strong pseudorandomness fails to do what we wanted to achieve with a game-based alternative to UC security: It seems once more to require an idealized outer structure and thus inhibits a proof for HashDH. Finally, since the simulator cannot succeed without additional powers, the definition is bound to the Ideal Primitive model [GT15] – essentially, a generalization of the random oracle model – making proofs less flexible in comparison with the other notions.

**One-more pseudorandomness (om-prnd).** Everspaugh et al. [ECS<sup>+</sup>15] go a different route, avoiding the simulator at the cost of switching to a non-standard game structure. The authors accept that  $\mathcal{A}$  may cross-check challenges between  $\mathcal{O}_{\text{Eval}}$  at  $\mathcal{O}_{\text{BlindEval}}$ . Instead of fixing consistency at  $\text{BlindEval}$ , they draw each  $\mathcal{O}_{\text{Eval}}$  challenge either from the real or random world, count the uses of  $\mathcal{O}_{\text{BlindEval}}$  and then ask  $\mathcal{A}$  to guess the world of *one more* challenge.

In the game, the adversary chooses  $\ell$  distinct challenges themselves to answer, but at least one more than they made calls to  $\text{BlindEval}$ . Their guess is a single bit, which must be the world of each of the  $\ell$  challenges combined via the exclusive or. This trick ensures that from knowing the correct bits for  $\ell - 1$  evaluations, the adversary learns nothing about the final output.

One-more pseudorandomness is perhaps the most straightforward strengthening of one-more unpredictability. On the downside, the one-more paradigm makes the relation to other properties less tight and security analyses likely require tailored versions of computational assumptions.

**Pseudorandomness in the UC framework.** In the UC framework, pseudorandomness is typically captured by having ideal evaluations be some truly random function tables (e.g., [JKKX17, BFH<sup>+</sup>20]), and security is expressed through the existence of an efficient simulator that makes the real and ideal world indistinguishable. This aspect is thus similar to the strong pseudorandomness notion. The UC definition also comes with a one-more aspect, which is most apparent in works that use a “ticket counter” (e.g., [JKK14]). Such a counter increases every time the evaluator agrees to an evaluation, and the requester is allowed to retrieve one OPRF output for each ticket they possess. Intuitively, UC security is closest to strong pseudorandomness due to the reliance on the

simulator in that definition, but also integrates aspects of the one-more notion, and a formal analysis is an interesting open problem.

### 4.3 Relations among Security Properties

We now analyze the formal relations between the various unpredictability and pseudorandomness properties we saw so far.

**Unpredictability Relations.** Our strong one-more unpredictability notion is indeed strictly stronger than the version that does not grant a *sk*-verification oracle. Do reduce redundancy, let us skip the proof for now, in fact, we get the relation as a corollary of Lemmas 4 and 5 below.

**Corollary 1.** *str-unpred is strictly stronger than unpred.*

**Pseudorandomness Relations.** Weak, one-more and strong pseudorandomness form a clear hierarchy. We start by showing that one-more pseudorandomness is strictly stronger than weak pseudorandomness, and then prove that strong pseudorandomness is strictly stronger than the one-more variant. Comparing one-more and weak pseudorandomness, while both games pose real-or-random challenges, we clearly strengthened the adversary in one-more pseudorandomness, where malicious **BlindEval** requests are allowed.

**Lemma 1.** *om-prnd is strictly stronger than weak-prnd.*

*Proof (sketch).* It is easy to see that weak pseudorandomness does not imply one-more pseudorandomness. One can define a scheme that becomes insecure upon rogue requests, e.g., **BlindEval** might reveal the secret key if some flag is set in the request. Such a change does not affect weak pseudorandomness, since the adversary is allowed only honest requests, but an active attacker can exploit this and trivially win the one-more pseudorandomness game.

Proving the inverse implication turns out rather tedious and untight due to the non-standard one-more paradigm, where the adversary must solve a single, fresh challenge instead of multiple challenges from one, consistent, world. The detailed proof is given in Appendix F.1, and the high-level idea is below.

Our goal is to make the weak pseudorandomness attacker  $\mathcal{A}_{\text{weak-prnd}}$  solve a single fresh one-more challenge – so we need to tame the  $q$  challenges  $\mathcal{A}_{\text{weak-prnd}}$  actually receives from  $\mathcal{O}_{\text{Eval}}$ . We do this in  $q$  steps; transitioning from the random to the real case of  $\text{Game}_{\Pi, \mathcal{A}}^{\text{weak-prnd}}$ , in each intermediate game we switch one of  $q$   $\mathcal{O}_{\text{Eval}}$  challenges to *real*. Thus, if some distinguisher is able to distinguish between two adjacent games, they actually solve exactly one real-or-random challenge. So we bound each game hop via one-more pseudorandomness, providing the remaining random challenges with a self-picked random function and the other real challenges from the **BlindEval** oracle of  $\text{Game}_{\Pi, \mathcal{A}}^{\text{om-prnd}}$ .

Moving on, we show that strong pseudorandomness implies one-more pseudorandomness. It remains an open question whether it is a strict strengthening, although Section 6.1 provides evidence that HashDH is one-more, but not strongly pseudorandom, which suggests this relation.

**Lemma 2.** *str-prnd implies om-prnd (in the Ideal Primitive model).*

*Proof (sketch).* Our strategy will be to modify the one-more pseudorandomness game into an impossible-to-solve version where all challenges are random, and show that this game is in fact indistinguishable from the original  $\text{Game}_{H,A}^{\text{om-prnd}}$ , unless  $\text{Game}_{H,A,S}^{\text{str-prnd}}$  can also be won. In detail, we replace any *real* challenges from  $\mathcal{O}_{\text{Eval}}$  to outputs of a second random function  $f_1$ , and let a simulator operate  $\mathcal{O}_{\text{BlindEval}}$  and an ideal primitive  $\mathcal{O}_{\text{Prim}}$  to be consistent with  $f_1$ .

With this setup, we can reduce one-more to strong pseudorandomness as follows:  $\mathcal{A}_{\text{str}}$  simulates one-more pseudorandomness by forwarding  $\mathcal{O}_{\text{BlindEval}}^{\text{om}}$  and  $\mathcal{O}_{\text{Prim}}^{\text{om}}$  queries to their own oracles, and for  $\mathcal{O}_{\text{Eval}}^{\text{om}}$  either runs a random function  $f_0$  or forwards the query to their own  $\mathcal{O}_{\text{Eval}}$ . Depending on  $\mathcal{A}_{\text{str}}$ 's world, either the original  $\text{Game}_{H,A}^{\text{om-prnd}}$  or the random version is simulated. The two cases must be indistinguishable if a scheme is strongly pseudorandom. Moreover, the random game described above does not give  $\mathcal{A}_{\text{om-prnd}}$  any advantage over a random guess. The full proof is given in Appendix F.2.

Additionally, we provide a direct treatment of the relation between strong pseudorandomness and the weaker properties in Appendix F.3, avoiding the lossy intermediate step of one-more pseudorandomness.

**Pseudorandomness vs. Unpredictability.** For classic PRFs, pseudorandomness clearly implies unpredictability. For OPRFs, however, we introduced different levels for both notions, and thus it is interesting to analyze how they relate to each other on the same level and across. Surprisingly, we find some orthogonality, both between the weakest version of each property and between strong one-more unpredictability and one-more pseudorandomness.

**Lemma 3.** *weak-prnd and unpred are orthogonal properties.*

*Proof (sketch).* OPRFs whose evaluations include some non-random part, let us say a fixed first half of zeroes, are clearly not pseudorandom, but may still have a second unpredictable half that makes them fulfill one-more unpredictability. Because of our decision to limit weak pseudorandomness to passive adversaries, it is not stronger than unpredictability though.

The one-more pseudorandomness property overcomes the constraint of weak pseudorandomness and implements an active attacker model, therefore surpassing weak one-more unpredictability. Everspaugh et al. [ECS<sup>+</sup>15] provide a proof sketch of the following lemma, which immediately holds for our definitions, too:

**Lemma 4.** *om-prnd is strictly stronger than unpred.*



When we move to our stronger unpredictability definition, this implication no longer holds. Strong unpredictability added a capability that is not reflected in the one-more pseudorandomness game: a canonical verification oracle for the PRF evaluation, which we can now exploit to separate the two notions.

Designing a scheme that satisfies one-more pseudorandomness but not strong unpredictability is challenging. We present such a scheme,  $\Pi'$ , which builds upon an existing OPRF  $\Pi$ . Our analysis demonstrates that if  $\Pi$  is strongly pseudorandom – implying both one-more pseudorandomness and strong unpredictability – then  $\Pi'$  achieves one-more pseudorandomness, but not strong unpredictability.

Somewhat unusually, this incomparability result relies on the existence of a strong pseudorandom OPRF  $\Pi'$ . This is necessary due to a technical subtlety in the formal proof. Given that Beullens et al. [BDFH25] showed that existence of a strongly unpredictable OPRF implies the existence of a pseudorandom OPRF in the ROM, our assumption is a reasonable one. While showing this incomparability without such an additional assumption is an interesting open problem, our result is still enough to justify that the verification oracle of **str-unpred** makes the unpredictability definition meaningfully stronger.

**Lemma 5.** *If there exists a str-prnd OPRF, then str-unpred and om-prnd are orthogonal properties.*

*Proof (sketch).* As shown in Lemma 3, it is easy to construct an OPRF that is unpredictable but not pseudorandom. For a construction that is one-more pseudorandom, but not strongly unpredictable, we exploit the fact that a  $\text{Game}_{\Pi, \mathcal{A}}^{\text{str-unpred}}$  adversary  $\mathcal{A}_{\text{str-unpred}}$  can distinguish between a correct and an incorrect evaluation, but a  $\text{Game}_{\Pi, \mathcal{A}}^{\text{om-prnd}}$  adversary  $\mathcal{A}_{\text{om-prnd}}$  cannot.

The high-level idea is to embed a special string  $s^*$  into an OPRF which allows to obtain two evaluations from a single request, but we carefully hide  $s^*$  such that it is only observable by  $\mathcal{A}_{\text{str-unpred}}$ . By appending an index  $i$  to a blind request, the  $i^{\text{th}}$  bit of  $s^*$  is leaked in a circumscribed way: The returned response contains the evaluation of the input  $x = s_i^* \| i$ . Thus, one learns the bit  $s_i^*$  only if one can distinguish the evaluation of  $0 \| i$  and of  $1 \| i$ . While  $\mathcal{A}_{\text{str-unpred}}$  may use  $\mathcal{O}_{\text{Vf}}$  to check,  $\mathcal{A}_{\text{om-prnd}}$  is in trouble:  $\mathcal{O}_{\text{BlindEval}}$  requests are costly in the one-more paradigm, and  $\mathcal{O}_{\text{Eval}}$  will in 1/2 of the cases return a useless random value. A concrete construction and its justification is given in Appendix F.4.

On the other hand, strong pseudorandomness does not “penalize”  $\mathcal{O}_{\text{BlindEval}}$  calls in the same way one-more pseudorandomness does – the strong notion allows the adversary to make unlimited calls to the oracle without the challenge getting harder every time. Thus, a verification oracle becomes simulatable in this version of pseudorandomness.

**Lemma 6.** *str-prnd is strictly stronger than str-unpred.*

*Proof (sketch).* Let  $G_0$  be as the strong unpredictability game but with an ideal primitive oracle. We transition to a game  $G_1$  where the OPRF is replaced by a truly random function  $f$ : the verification oracle checks input-output pairs against

this function  $f$ ,  $\mathcal{O}_{\text{BlindEval}}$  and  $\mathcal{O}_{\text{Prim}}$  are run by a simulator. If a scheme  $\Pi$  is strongly pseudorandom, these two games are indistinguishable. Next, in  $G_2$ , we let  $\mathcal{O}_{\text{Vf}}$  answer only those queries  $(x, y)$  correctly that have been output by  $\mathcal{O}_{\text{LimEval}}$  before, otherwise the oracle always returns 0. This is indistinguishable, unless for the unlikely event that the adversary queries a pair  $(x, y)$ , that is correct but was not provided by the game. But now, in  $G_2$ ,  $\mathcal{A}$  is required to guess an additional  $f$  output for which they clearly received no hint from the oracles, so we conclude that any strongly pseudorandom OPRF must also be strongly unpredictable. For the inverse implication, a standard argument can be made that not every unpredictable ORPF is also pseudorandom. We give the full proof in Appendix F.5.

## 5 Privacy of OPRFs

We now turn to the privacy property expected from OPRFs: The evaluator should learn nothing about the input  $x$  or the output  $y$  during the exchange. This is essential for privacy-preserving application such as Privacy Pass, or when the OPRF is used to derive keys from low-entropy inputs such as a password.

We capture this privacy property through three obliviousness definitions, with increasing strength. Our first notion *request obliviousness* captures the same privacy guarantees as has been formalized in three different manners in the works by Lehmann [Leh19], Tyagi et al. [TCR<sup>+</sup>22], as well as Bellare et al. [BRRA24]. This notion assumes an honest-but-curious evaluator though, and we therefore also propose definitions where the adversary can be actively malicious: *Single-key obliviousness* ensures privacy against an adversary that is malicious but does not use rogue keys, and *multi-key obliviousness* protects against malicious adversaries without the latter restriction.

We then observe that *multi-key obliviousness* implies *implicit verifiability*; the requester must be able to check that they received a correctly computed response for a targeted public key  $pk$ . To emphasize this need, we formally define this verifiability property and show that it is implied by multi-key obliviousness and also lifts single-key obliviousness to this stronger form.

### 5.1 Obliviousness

Obliviousness is similar to blindness for blind signatures, and accordingly obliviousness definitions borrow the concept of left-or-right indistinguishability: A malicious evaluator chooses two inputs  $x_0, x_1$ , which are blinded and returned in random order, and they have to tell which blind request belongs to which input. Our three variants of obliviousness definitions are formally defined through security games in Fig. 4 and each notion’s advantage is defined as follows:

To address different adversarial models, we will motivate three definitions, with increasing strength: *request obliviousness* ensures privacy against an honest-but-curious adversary, *single-key obliviousness* ensures privacy against an adversary that is malicious but does not use rogue keys, and *multi-key obliviousness* protects against malicious adversaries without the latter restriction.

| $\text{Game}_{\Pi, \mathcal{A}}^{\text{obliv-req}}(\lambda), \text{Game}_{\Pi, \mathcal{A}}^{\text{obliv-1k}}(\lambda), \text{Game}_{\Pi, \mathcal{A}}^{\text{obliv-k+}}(\lambda)$                                                                                                                                                                                               | $\mathcal{O}_{\text{Finalize}}(\bar{y}', \bar{y}'', i, \text{sk})$                                                                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $pp \leftarrow \text{Setup}(\lambda)$<br>$c_{\text{req}} := 0; b \leftarrow \{0, 1\}$<br>$b' \leftarrow \mathcal{A}^{\text{Request}, \text{Finalize}}(pp)$<br><b>return</b> $b \stackrel{?}{=} b'$                                                                                                                                                                               | <b>if</b> $i > c_{\text{req}}$ : <b>return</b> $\perp$<br>$y_b \leftarrow \text{Finalize}(\bar{y}', st_{i,b})$<br>$y_{1-b} \leftarrow \text{Finalize}(\bar{y}'', st_{i,1-b})$<br><b>if</b> $y_0 \neq \text{Eval}(\text{sk}, x_{i,0}) \vee y_1 \neq \text{Eval}(\text{sk}, x_{i,1})$ :<br><b>return</b> $\perp$<br><b>if</b> $y_b = \perp \vee y_{1-b} = \perp$ : <b>return</b> $\perp$<br><b>return</b> $(y_0, y_1)$ |
| $\mathcal{O}_{\text{Request}}(pk, x_0, x_1)$<br><hr/> $c_{\text{req}} := c_{\text{req}} + 1; \quad x_{c_{\text{req}},0} := x_0; \quad x_{c_{\text{req}},1} := x_1$<br>$(\bar{x}_b, st_{c_{\text{req}},b}) \leftarrow \text{Request}(pk, x_b)$<br>$(\bar{x}_{1-b}, st_{c_{\text{req}},1-b}) \leftarrow \text{Request}(pk, x_{1-b})$<br><b>return</b> $(\bar{x}_b, \bar{x}_{1-b})$ |                                                                                                                                                                                                                                                                                                                                                                                                                      |

**Fig. 4.** Games for request, single-key, and multi-key obliviousness. The first property assumes an honest-but-curious adversary, the other two a malicious adversary. Highlighted code is exclusive to the respective game.

**Definition 4 (Obliviousness).** We call an OPRF protocol  $\Pi$   $x$ -oblivious for  $x \in \{\text{req}, 1k, k+\}$ , if  $\forall$  PPT adversaries  $\mathcal{A}$  the following advantage is negligible:

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{obliv-}x}(\lambda) := 2 \cdot \Pr \left[ \text{Game}_{\Pi, \mathcal{A}}^{\text{obliv-}x}(\lambda) \right] - 1$$

**Request obliviousness (obliv-req).** Our first notion assumes that the evaluator is honest-but-curious. This might be a reasonable assumption in some scenarios. For instance, a requester that trusts the evaluator may still want to ensure that the transcript of the blind evaluation does not leak information to third parties who eavesdrop on their communication.

We find three related works employing the same model, even though, at first glance, they differ strongly in how the challenges look and how much control the adversary has over evaluation keys. Lehmann [Leh19] (in a property called *obliviousness*) provides the attacker with a single request as the left-or-right challenge, while Tyagi et al. [TCR<sup>+</sup>22] (*POPRIV*) as well as Bellare et al. [BRRA24] (*input indistinguishability*<sup>1</sup>) provide the challenges via a transcript oracle, which upon two inputs returns not only the request, but also blind and cleartext evaluation of the given inputs. Concerning the keys used in generating the challenges, Tyagi et al. fix a key pair and send it to the adversary. Bellare et al. allows the adversary to specify for each transcript the evaluation keys; but since they model attackers as semi-honest, these keys should still be generated honestly. Lehmann’s treatment of keys is harder to compare: here, the obliviousness game does not involve evaluation, so evaluation keys are not needed, but the work uses an additional key pair for blinding, out of which the public key is sent to the adversary.

Somewhat surprisingly, the transcript oracles do not result in a stronger definition, so we propose the simpler game titled *request obliviousness*. To see

<sup>1</sup> Input indistinguishability is a generic property for two-party-computation, but plugging in OPRFs with the evaluator as the corrupt party, we get request obliviousness.

why the full transcript is redundant, note how the adversary is in possession of the secret key and can compute both the blind response and the cleartext evaluation themselves – in other words, after receiving the request, for a protocol-following evaluator the rest of the exchange does not bear new information.

Thus, the adversary is merely given a **Request** oracle, where they can repeatedly obtain left-or-right challenges consisting of the blinded inputs. We adopt Bellare et al.’s choice of allowing for dynamically chosen evaluation keys, which seems the strongest variant, and similarly capture honest key generation implicitly in our semi-honest model.

Formally, we can show our definition implies each of the related works. The proofs to the following lemma are given in Appendix F.6.

**Lemma 7.** *obliv-req implies obliviousness in [Leh19], POPRIV1 in [TCR<sup>+</sup>22] and input indistinguishability in [BRRA24].*

**Single-key obliviousness (obliv-1k).** While omitting the finalized evaluations in the game does not matter when considering only honest-but-curious adversaries, this no longer holds when we switch to an active attacker model: The adversary may send maliciously crafted responses, and the finalization failing or returning an incorrect value can then reveal information about the associated input. A malicious evaluator could also handle each request with a different key, and later, by inspecting under which key an evaluation is valid, link it to the evaluation process. We refer to this strategy as a *multi-key* attack.

We partially capture the aforementioned attacks by granting the adversary a **Finalize** oracle that takes two blinded responses and finalizes them. It only returns the final outputs if they are both correct, i.e. valid under the same secret key. The same approach is taken in definitions of signature blindness, where this check is straightforward due to the public verifiability. For OPRF, we (generally) lack a public verification mechanism though, so we force the adversary to specify the secret key used during blind evaluation. Since we are limiting the adversary to one key pair per challenge, we call our property *single-key obliviousness*.

The **Finalize** oracle might appear rather useless, given that it returns nothing but two correct evaluations – which the adversary could compute themselves as  $\text{Eval}(sk, x)$  for any input  $x$ . But in fact,  $\mathcal{O}_{\text{Finalize}}$  provides one bit of extra information: The adversary can test whether some malicious evaluation finalizes into the correct value with respect to the  $i^{\text{th}}$  request. Thus, single-key obliviousness covers bugs in the **Finalize** algorithm that could leak some information on the output. Additionally, active attacks on the **Request** algorithm are addressed. These two details fill an important gap between request obliviousness and the strongest obliviousness notion, as we will see in Section 5.3.

**Multi-key obliviousness (obliv-k+).** The single-key definition still implicitly requires some honest behavior. In particular, it takes multi-key attacks out of the equation by enforcing the adversary to use the same key for both evaluations. Thus, our strongest notion of obliviousness aims to prevent multi-key attacks.

Tyagi et al. [TCR<sup>+</sup>22] propose such a stronger property as *POPRIV2*, where they relax the restrictions within  $\mathcal{O}_{\text{Finalize}}$  to refuse garbage data that makes the finalization fail, but do not impose a single key pair. We adopt Tyagi et al.’s definition and call it *multi-key obliviousness*.

However, this property is so strong that standard OPRFs such as HashDH or 2HashDH do not achieve it. Intuitively, in these constructions the requester simply has no way of ensuring that a consistent key is used. The only fix is to introduce a mechanism of implicit verifiability to allow *Finalize* to refuse rogue key evaluations. Indeed, any multi-key oblivious OPRF is also verifiable. We prove this relation in Section 5.3 after formally introducing verifiability.

**Obliviousness in the UC framework.** Obliviousness is typically captured in an ideal functionality by giving the input  $x$  and output  $y$  exclusively to the requester and no other party (e.g., [JKKX16]). The only information the evaluator gets is that a function query is made. Once the evaluator approves the query, the functionality directly responds with a random value to the requester. The exact level of obliviousness captured by the ideal functionalities is not entirely clear. Many UC definitions follow the approach by Jarecki et al. [JKKX16], where corrupt evaluators can point to a different function table for each query, that is then consistently filled with random values. The adversary still does not learn any information on the requester’s input or output. Thus, the functionality gives a corrupt evaluator the freedom to use inconsistent keys, but still requires their outputs to be indistinguishable from random values. Analyzing a clear connection between the various UC functionalities and our three levels of obliviousness is an interesting open problem. In particular, if the UC functionality can be shown to be equivalent to our multi-key obliviousness notion, then the 2HashDH instantiation that is typically used for the functionality would *not* be secure. Moreover, asking for random outputs even for a malicious evaluator once again suggests an outer random oracle is mandatory – whereas we are able to also prove a verifiable HashDH multi-key oblivious (Corollary 2).

## 5.2 Verifiability

We saw in the previous section how malicious responses may violate the requester’s privacy, e.g., plain evaluations become linkable through a multi-key attack. This is because in standard OPRFs, the requester essentially has to trust the evaluator about the blind evaluation; they have no means of knowing what the evaluator did as long as the finalization does not fail.

To provide protection against malicious evaluators, we introduce an extension: implicit verifiability. This introduces a mechanism for the requester to verify that the blind evaluation was performed honestly with the targeted *public key*. The verification is implicitly performed during *Finalize*, so in a verifiable scheme, we expect *Finalize* to fail upon any dishonest response if the result is not correct w.r.t. the public key the requester has chosen in their *Request* algorithm.

We capture implicit verifiability in the game shown in Fig. 5, where a malicious evaluator tries to generate a response that yields an incorrect result with

| $\text{Game}_{\Pi, \mathcal{A}}^{\text{vf}}(\lambda)$                                                       | $\mathcal{O}_{\text{Request}}(pk, x)$                             | $\mathcal{O}_{\text{Finalize}}(\bar{y}, i)$            |
|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|--------------------------------------------------------|
| $pp \leftarrow \text{Setup}(\lambda); \quad c_{\text{req}} := 0$                                            | $c_{\text{req}} := c_{\text{req}} + 1$                            | <b>if</b> $i > c_{\text{req}}$ : <b>return</b> $\perp$ |
| $(sk, j, \bar{y}) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Request}}, \mathcal{O}_{\text{Finalize}}}(pp)$ | $pk_{c_{\text{req}}} := pk; \quad x_{c_{\text{req}}} := x$        | <b>return</b> $\text{Finalize}(\bar{y}, st_i)$         |
| $y' \leftarrow \text{Finalize}(\bar{y}, st_j)$                                                              | $(\bar{x}, st_{c_{\text{req}}}) \leftarrow \text{Request}(pk, x)$ |                                                        |
| <b>if</b> $\text{public}(sk) \neq pk_j \vee y' = \perp$ : <b>return</b> 0                                   | <b>return</b> $\bar{x}$                                           |                                                        |
| <b>return</b> $y' \stackrel{?}{\neq} \text{Eval}(sk, x_j)$                                                  |                                                                   |                                                        |

**Fig. 5.** Security game for verifiability.

respect to the target key. The game allows the adversary to pick a challenge request and target key pair themselves, and to test their responses at a **Finalize** oracle. At the end of the game, the adversary selects a  $j^{\text{th}}$  request as their challenge, outputs their rogue response  $\bar{y}$  and reveals the purported evaluation key  $sk$ . It is checked that the public counterpart of  $sk$  is indeed the public key  $pk_j$  that the  $j^{\text{th}}$  request was bound to, and that the finalization of  $\bar{y}$  does indeed not fail. If these conditions are met and the resulting value is still incorrect, the adversary wins. For checking correctness of the public key, we introduce a function  $\text{public}(sk) \rightarrow pk'$  which computes the corresponding public key from  $sk$ , and checks it against the provided value  $pk$ . Thus, our definition implicitly assumes unique public keys, which holds trivially for DL-based constructions, but will be challenging for lattice-based constructions. Asking the adversary to output the secret key might appear like a strong request. This can be realized by using an extractable zero-knowledge proof as part of the public keys in the construction. We have opted for our model for the sake of simplicity.

**Definition 5 (Verifiability).** *We call an OPRF protocol  $\Pi$  verifiable, if  $\forall$  PPT adversaries  $\mathcal{A}$  the following advantage is negligible:*

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{vf}}(\lambda) := \Pr \left[ \text{Game}_{\Pi, \mathcal{A}}^{\text{vf}}(\lambda) \right]$$

Significant prior work on implicit verifiability comes from Albrecht et al. [ADDG24, AG24] and Tyagi et al. [TCR<sup>+</sup>22], from whom our definition takes inspiration. Albrecht et al. weaken the adversary slightly by forcing them to commit to a challenge before getting access to the oracles, and do not pose any limitations on the evaluation keys, which unfortunately leaves room for a trivial attack (shown in Appendix I) where the attacker announces a key pair with public and secret key that are not matching. Tyagi et al. define a slightly different notion *uniqueness*, which ensures that malicious evaluators cannot make the requester accept two distinct evaluations to the same input. The game structure and adversarial abilities are similar, and informally, verifiability implies uniqueness, since the adversary can *exclusively* output the honest response. Uniqueness implies verifiability only as long as we enforce well-formedness of the keys – otherwise, it might be that the adversary, using a malformed key, is able to provide a single, incorrect response, breaking verifiability, but has no way of computing any other response, not even the correct one.

**Verifiability in the UC framework.** Two examples of UC-based works defining a verifiable OPRF are Jarecki et al. [JKK14] and Boneh et al. [BKW20]. Both capture verifiability in a subtle way: Evaluators must initially register a function table from which during the interaction any evaluation is drawn. This seems a more limited guarantee than what  $\text{Game}_{H,A}^{\text{vf}}$  expresses: Jarecki et al. state explicitly that pseudorandomness cannot be ensured, so while a malicious evaluator cannot dynamically forge evaluations, they still have some head space and could register a faulty table initially. In contrast, the game-based notion promises that a requester who (1) honestly executes their own algorithms and (2) verifies the blind evaluation will receive the unique correct PRF evaluation at the end.

### 5.3 Relations among Privacy Notions

Our obliviousness notions successively increase the privacy requirements, where request obliviousness only ensures the input is securely blinded, single-key obliviousness adds a key-restricted **Finalize** oracle, and multi-key obliviousness gives full freedom in key usage. Unsurprisingly, this results in a clear hierarchy among the definitions. The proof of Lemma 8 is given in Appendix F.7.

**Lemma 8.** *obliv-k+ is strictly stronger than obliv-1k, which in turn is strictly stronger than obliv-req.*

Multi-key obliviousness demands that a malign response might trigger an error during finalization, but can never change the result in any way that would allow linking. Thus, the only blind responses any evaluator can create are *expected* ones. The similarity of this requirement to implicit verifiability is apparent, and as already noted by Tyagi et al. [TCR<sup>+</sup>22], multi-key obliviousness indeed requires a verifiable scheme. For preciseness, we add that this holds only for correct schemes: Consider an OPRF where **Finalize** simply outputs a completely random value. This OPRF is multi-key oblivious – there is no way to link the outputs to the blind evaluation – without being verifiable.

**Lemma 9.** *Correctness and obliv-k+ imply vf.*

The inverse is not true, i.e. implicit verifiability is not sufficient for multi-key obliviousness. Verifiability provides only a guarantee for honest blind evaluation; requester privacy could still be completely broken if the blinding or finalization algorithms are flawed. This is where our newly introduced single-key obliviousness property comes into play: combining implicit verifiability and request obliviousness is explicitly not sufficient to achieve the strongest obliviousness notion, since it lacks active security of the blinding and does not cover the **Finalize** algorithm, but single-key obliviousness provides the missing guarantee.

**Lemma 10.** *obliv-req and vf do not imply obliv-k+.*

**Lemma 11.** *obliv-1k and vf imply obliv-k+ (in the Knowledge of the Secret Key Model [Bol03]).*

The Knowledge of the Secret Key (KOSK) model of Lemma 11 is needed due to a technicality: We reduce to single-key-obliviousness and verifiability, where adversaries must explicitly output secret keys. Therefore, we need access to the multi-key-obliviousness adversary’s secret key, which the KOSK model solves generically. We formally prove Lemmas 9 to 11 and show that implicit verifiability does not imply multi-key obliviousness in Appendices F.8 to F.11.

As a final note, for some applications neither multi-key obliviousness nor verifiability might be desirable, because the evaluator must use different keys for protocol-internal reasons and the requester should not be able to tell which key is used for what evaluation.

## 6 Security and Privacy of the (2)HashDH OPRFs

To showcase our framework, we conduct a security analysis of two widely used OPRFs. HashDH and 2HashDH achieve almost all properties, with HashDH lacking strong pseudorandomness and both constructions requiring an additional proof of honest behavior for implicit verifiability and multi-key obliviousness.

### 6.1 Analysis of HashDH

The non-blind DH-based PRF  $F(sk, x) := H(x)^{sk}$  was first introduced by Naor et al. [NPR99], and is secure under the DDH assumption if  $H$  is a random oracle. The PRF is easily made interactive by blinding  $H(x)$  exponentially<sup>2</sup>.

**Definition 6 (HashDH).** *The HashDH OPRF is defined as follows, with group generation  $(\mathbb{G}, q, g) \leftarrow \$ \text{GGen}(\lambda)$  and hash function  $H : \{0, 1\}^* \rightarrow \mathbb{G}$ .*

**Setup** $(\lambda) : (\mathbb{G}, q, g) \leftarrow \$ \text{GGen}(\lambda); \text{ return } (\mathbb{G}, q, g)$   
**KGen** $(pp) : \text{ parse } (\mathbb{G}, q, g) = pp; sk \leftarrow \$ \mathbb{Z}_q; pk := \perp; \text{ return } (pk, sk)$   
**Request** $(pk, x) : r \leftarrow \$ \mathbb{Z}_q \setminus \{0\}; \bar{x} := H(x)^r; st := r; \text{ return } (\bar{x}, st)$   
**BlindEval** $(sk, \bar{x}) : \bar{y} := \bar{x}^{sk}; \text{ return } \bar{y}$   
**Finalize** $(\bar{y}, st) : \text{ parse } r = st; y := \bar{y}^{(1/r)}; \text{ return } y$

Correctness is straightforward and shown in Appendix G.1. Note that we require the blinding exponent  $r$  explicitly to be non-zero, as otherwise the scheme is no longer perfectly correct. Despite its simplicity, HashDH achieves strong security and privacy guarantees. We start with unpredictability and pseudorandomness; here the only missing property is strong pseudorandomness, for which HashDH lacks an outer idealized primitive.

<sup>2</sup> For both HashDH and 2HashDH we use exponential blinding. While multiplicative blinding has performance advantages, it opens new attack vectors [JKX21, Don22].



**Unpredictability.** Everspaugh et al. [ECS<sup>+</sup>15] show one-more unpredictability for a pairing-based version of HashDH, and we show that this proof can be adapted for HashDH as well. The security of HashDH is proven under the OM-CDH assumption, which tasks the adversary to solve  $q + 1$  CDH challenges after being able to learn at most  $q$  CDH solutions of their choice. For strong unpredictability, the CDH assumption is not enough to simulate the canonical verification oracle, which essentially requires solving DDH for each verification query. In order to satisfy this extra need, we prove the strong one-more unpredictability under the **gap-OM-CDH** assumption, which extends the OM-CDH assumption with such a DDH oracle. Note that using a gap assumption in non-gap groups is a common approach to prove the security of (2)HashDH or related constructions [CLN15, TCR<sup>+</sup>22, JKK14]. The full proof of Theorem 1 is given in Appendix G.2.

**Theorem 1.** *If  $H$  is a random oracle, then HashDH is:*

- *unpred if the OM-CDH assumption holds for GGen,*
- *str-unpred if the gap-OM-CDH assumption holds for GGen.*

**Weak pseudorandomness.** Our weakest pseudorandomness notion now requires that the OPRF outputs must be indistinguishable from random. This notion assumes an honest-but-curious requester, which essentially simplifies the proof to showing that the non-blind version is secure. The proof requires a little more care though. Intuitively, a DDH assumption typically lets the adversary distinguish between a real world, in which they receive DH pairs  $Y, Y^x$ , and a random world, in which they receive an independent, random pair  $Y, Z$ . In contrast, during the weak pseudorandomness game, the adversary also has the **BlindEval** oracle, which always hands out “real” DH values. We handle this via an intermediate game hop between the real and the random world: We initially supply both  $\mathcal{O}_{\text{Eval}}$  and  $\mathcal{O}_{\text{BlindEval}}$  queries with DDH challenges, and then move to a game where only  $\mathcal{O}_{\text{BlindEval}}$  is returning DDH challenges, whereas  $\mathcal{O}_{\text{Eval}}$  samples random responses. Finally, to simplify the proof, we follow Bellare et al. [BRR24] and employ a variation of the classic DDH assumption with multiple challenges that is equivalent to DDH. The full proof is given in Appendix G.4.

**Theorem 2.** *HashDH is weak-prnd if  $H$  is a random oracle and the DDH-mult assumption holds for GGen.*

**One-More-Pseudorandomness.** We show that HashDH is also one-more pseudorandom by adapting the proof of Everspaugh et al. [ECS<sup>+</sup>15]. They gave a proof for a pairing-based variant of HashDH via a tailored one-more DDH assumption, which we adapt to standard groups in Appendix D. This assumption provides multiple real-or-random challenges and a CDH helper oracle. The adversary therein wins if they can break more DDH challenges than they made queries to the CDH oracle. While this assumption allows for a simple proof that we give in Appendix G.3, it leaves us with a specialized assumption, and further investigation of the hardness of that assumption is desirable.

**Theorem 3.** *HashDH is om-prnd if  $H$  is a random oracle and the OM-DDH assumption holds for GGen.*

**Strong pseudorandomness.** We have already mentioned that the simulation-based nature of strong pseudorandomness imposes similarly strong constraints as a UC definition, in particular, it seems to impose the need for an *outer* idealized primitive. We now provide strong evidence that HashDH is indeed not strongly pseudorandom due to the lack of the outer hash. We prove this by showing that a simulator needed in the strong pseudorandomness proof would allow to break a reasonable CDH-related assumption. While our assumption is novel, we show that it is implied by the DL assumption in the AGM in Appendix D.1.

**Theorem 4.** *If hint-CDH holds, HashDH is not str-prnd.*

*Proof (sketch).* The proof is rooted in the observation that a simulator needed in the strong pseudorandomness proof for HashDH obtains a request blinded by an unknown  $r$  and a randomly chosen evaluation  $y$ , and then has to generate some response  $\bar{y}$  such that  $y^r = \bar{y}$ . Essentially, it is solving a CDH challenge for the exponent  $r$  and the base  $y$ .

Concretely, the following simple attack cannot be averted by any efficient simulator (without invalidating a CDH-based assumption): An adversary  $\mathcal{A}_{\text{prnd}}$  queries the ideal primitive oracle  $\mathcal{O}_{\text{Prim}}$  (modeling the hash function) on some input  $x$ , and then sends a blinded request to  $\mathcal{O}_{\text{BlindEval}}$ , and checks whether the result is consistent with  $\mathcal{O}_{\text{Eval}}(x)$ .

In the real world, the two oracles' outputs are always consistent, but in the random world, they differ unless the simulator solves a CDH-type challenge. We construct a CDH adversary that forwards the CDH challenge to the simulator upon its  $\mathcal{O}_{\text{LimEval}}$  query. To simulate the blinded response correctly, the simulator must raise this challenge to the unknown  $r$ , enabling the CDH adversary to win.

As a technicality, we have to consider that the simulator sees  $\bar{x}$ , which is the value returned from  $\mathcal{O}_{\text{Prim}}$  raised to  $r$ . We accommodate this detail by resorting to a new hint-CDH assumption, which is introduced in Appendix D.1. The full proof of Theorem 4 is given in Appendix G.5.

Turning to our privacy properties, the standard construction of HashDH satisfies request obliviousness and single-key obliviousness.

**Request obliviousness.** Request and single-key obliviousness are achieved directly from the exponential blinding. We start with the weaker property, which does assume an honest-but-curious evaluator, and is expressed through the indistinguishability of honest requests. We follow the proof by Bellare et al. [BRRA24] showing how the blinding is almost, but not perfectly hiding.

**Theorem 5.** *If  $H$  is a random oracle, then HashDH is obliv-req.*

The exponential blinding yields perfect privacy, as it produces a uniformly random group element. However, as Bellare et al. [BRR24] point out, the hash function introduces a subtlety: If the  $H$  returns the identity element 1 for some  $x_i$ , then any blinding  $H(x_i)^r = 1$  fails to hide the input. Fortunately, this exception happens rarely for a secure hash function and large group size, which is what we capture through the random oracle assumption for convenience. The actual requirements are more relaxed: We need neither programmability nor observability, not even true randomness, it only should be hard to find a map to 1. Standard hash-to-curve algorithms with appropriate group sizes  $q$  fulfill this condition: The probability of a random input being mapped to 1 is roughly  $1/q$  and finding such an input is deemed “computationally infeasible” [FHSS+23]. The full proof is in Appendix G.6.

**Single-key obliviousness.** In single-key obliviousness, we also consider malicious adversaries and model access to a **Finalize** oracle that checks whether two blind responses finalize correctly under a given secret key. We argue that for the case of HashDH, this attacker model is equivalent to request obliviousness, and thus single-key obliviousness is implied by request obliviousness. The full proof of the following theorem is in Appendix G.7.

**Theorem 6.** *If  $H$  is a random oracle, then HashDH is obliv-1k.*

*Proof (sketch).* We want to show that an adversary  $\mathcal{A}_{\text{obliv-1k}}$  has identical abilities to an adversary in the request obliviousness game. First,  $\mathcal{O}_{\text{Request}}$  is identical, and  $\mathcal{A}_{\text{obliv-1k}}$  gains no advantage from becoming active, either: They could send a rogue  $pk$ , but the key is not used in HashDH’s **Request** algorithm. Secondly, there is the additional  $\mathcal{O}_{\text{Finalize}}$  in  $\mathcal{A}_{\text{obliv-1k}}$ , which tells whether two given responses result each in the correct evaluation of the associated input. However, observe that HashDH’s **Finalize** is a bijection. Consequently, the only  $\bar{y}$  that will finalize into the correct one is the *honest* blind evaluation. But  $\mathcal{A}_{\text{obliv-1k}}$  can check trivially themselves whether  $\bar{y}$  is the honest blind evaluation of some  $\bar{x}$  by checking whether  $\bar{x}^{sk} = \bar{y}$  – so there is nothing to be learned from  $\mathcal{O}_{\text{Finalize}}$ .

**No multi-key obliviousness.** Standard HashDH achieves neither multi-key obliviousness nor implicit verifiability: A blind evaluation is not bound to a key and the **Finalize** algorithm accepts arbitrary input. Requester privacy is therefore fully compromised by a malicious evaluator. In particular, using the multi-key attack, the evaluator can later link every plain evaluation they see to the specific OPRF session by checking under which key it verifies. We describe this attack formally in Appendix G.8.

**Theorem 7.** *HashDH is not obliv-k+ and not vf.*

**Vf-HashDH: achieving multi-key obliviousness and verifiability.** We established in Section 5.3 how multi-key obliviousness requires a mechanism of

implicit verification for the requester. Therefore, we need to modify HashDH accordingly, using the well-established approach of adding a zero-knowledge proof of correct evaluation. We first modify key generation to now output a public key  $pk := g^{sk}$ . In addition, the evaluator must provide a proof  $\pi$  of correct computation of  $\bar{y}$  as:  $\pi \leftarrow \text{NIZK}\{(sk) : pk = g^{sk} \wedge \bar{y} = \bar{x}^{sk}\}(g, pk, \bar{x}, \bar{y})$ . The requester verifies the proof while finalizing. To this end, **Finalize** needs the public values  $pk$  and  $\bar{x}$ , so we store both in the state  $st$  during **Request**. This has already been used in a number of works, e.g., [JKK14, CLN15, DGS<sup>+</sup>18]. We thus do not claim any novelty for this extension, but merely want to point out its necessity for achieving the stronger – and intuitively expected – privacy guarantees.

**Definition 7 (Vf-HashDH).** *The vf-HashDH OPRF is defined analogously to HashDH, using an additional NIZK as follows.*

**Setup**( $\lambda$ ) :  $(\mathbb{G}, q, g) \leftarrow \text{GGen}(\lambda)$ ; **return**  $(\mathbb{G}, q, g)$   
**KGen**( $pp$ ) : *parse*  $(\mathbb{G}, q, g) = pp$ ;  $sk \leftarrow \mathbb{Z}_q$ ;  $pk := g^{sk}$ ; **return**  $(pk, sk)$   
**Request**( $pk, x$ ) :  $r \leftarrow \mathbb{Z}_q \setminus \{0\}$ ;  $\bar{x} := H(x)^r$ ;  $st := (pk, \bar{x}, r)$ ; **return**  $(\bar{x}, st)$   
**BlindEval**( $sk, \bar{x}$ ) :  $\bar{y} := \bar{x}^{sk}$ ;  $\pi \leftarrow \text{NIZK}\{(sk) : pk = g^{sk} \wedge \bar{y} = \bar{x}^{sk}\}(g, pk, \bar{x}, \bar{y})$ ,  
 $\bar{y}' := (\bar{y}, \pi)$ ; **return**  $\bar{y}'$   
**Finalize**( $\bar{y}', st$ ) : *parse*  $(pk, \bar{x}, r) = st$  and  $(\bar{y}, \pi) = \bar{y}'$ ; *verify*  $\pi$  w.r.t.  $(g, pk, \bar{x}, \bar{y})$ .  
*If*  $\pi$  *fails*, **return**  $\perp$ ; *else*  $y := \bar{y}^{(1/r)}$ ; **return**  $y$

Vf-HashDH achieves the same properties as HashDH if the NIZK is correct, sound and zero-knowledge. For single-key obliviousness, which we use in the following, we provide some argumentation in Appendix G.9.

**Theorem 8.** *Vf-HashDH is obliv-1k if  $H$  is a random oracle.*

Next, we show how the additional proof of honest blind evaluation corrects the unwanted malicious linkability of the standard HashDH OPRF.

**Theorem 9.** *Vf-HashDH is vf if the NIZK is sound.*

Verifiability follows directly from the soundness of the employed NIZK scheme, and we state the simple proof in Appendix G.10. Multi-key obliviousness then follows according to Lemma 11.

**Corollary 2.** *Vf-HashDH is obliv-k+ if  $H$  is a random oracle and the NIZK is sound.*

## 6.2 Analysis of 2HashDH & Generic Compiler

We now analyze 2HashDH, which is the stronger variant of HashDH. It was proposed by Jarecki et al. [JKK14] as a UC-secure OPRF, by adding an outer hash to HashDH. We also observe that adding this outer hash yields a generic transform that allows to lift any strongly one-more unpredictable OPRF into a strongly pseudorandom one.

**Security and Privacy of 2HashDH.** 2HashDH is defined through the blind evaluation of  $2\text{HashDH.Eval}(sk, x) := H_2(x, H_1(x)^{sk})$ . It achieves all security and privacy properties of our framework, although multi-key obliviousness and implicit verifiability once more require the extension by a proof of honest blind evaluation. In Appendix H, we formally state the construction as well as its verifiable variant before giving a full security analysis. We point to and adapt existing game-based proofs where possible. Notably, our work closes the gap for the yet unaddressed properties (strong) one-more unpredictability, single-key obliviousness, and verifiability, showing the following:

**Theorem 10.** *If  $H_1$  is a random oracle, then 2HashDH is*

- str-prnd, om-prnd, weak-prnd, and str-unpred if  $H_2$  is a random oracle and gap-OM-CDH holds for GGen
- unpred if  $H_2$  is a random oracle and OM-CDH holds for GGen
- obliv-req
- obliv-1k if  $H_2$  is collision resistant.

*With a secure NIZK, vf-2HashDH achieves all of the above and is additionally*

- vf and obliv-k+ if the NIZK is sound.

**Generic Transform.** The pseudorandomness proofs of 2HashDH (see Appendix H.3) and 3HashSDHI OPRF by Tyagi et al. [TCR<sup>+</sup>22] give rise to a generic construction of pseudorandom OPRFs. In fact, Beullens et al. [BDFH25] recently demonstrated that a universally composable OPRF can be obtained by applying an outer hash to an unpredictable OPRF. Their unpredictability notion was kept weak on purpose to examine minimal properties needed for that transform. We now show that a similar result holds for transforming a strongly unpredictable OPRF into a strongly pseudorandom one. Thus, our result complements [BDFH25] in the game-based model and holds for an underlying OPRF with a meaningful stand-alone property. We begin by presenting the generic construction.

**Definition 8.** *Let  $\Pi_{\text{unpred}}$  be an OPRF and  $H$  be a hash function mapping from the output space of  $\Pi_{\text{unpred}}$  to strings of length  $n$ . Let  $\Pi_{\text{prnd}}$  be identical to  $\Pi_{\text{unpred}}$  except the following algorithms:*

```

Eval'(sk, x) : return H(x, Eval(sk, x))
Request'(pk, x) : ( $\bar{x}$ , st)  $\leftarrow$  Request(pk, x); return ( $\bar{x}$ , (st, x))
Finalize'( $\bar{y}$ , st) : parse (st', x) = st;  $y' \leftarrow$  Finalize( $\bar{y}$ , st'); if  $y' = \perp$  : return  $\perp$ ;
else return H(x, y')

```

Intuitively,  $\Pi_{\text{prnd}}$  is secure in the ROM as the outputs of  $H$  are indistinguishable from truly random values to an adversary that is unlikely to *predict* the input value  $H$  is invoked on. This is exactly what the input  $\text{Eval}(sk, x)$  of the underlying strongly unpredictable OPRF will provide.

The privacy properties of  $\Pi_{\text{prnd}}$  can be analyzed by relying on the corresponding properties of the underlying  $\Pi_{\text{unpred}}$  as well. We show this for the multi-key obliviousness explicitly, the other implications follow similarly. The full proof of Theorem 11 is given in Appendix J.

**Theorem 11.** *If  $\Pi_{\text{unpred}}$  is str-unpred and obliv-k+, and  $H$  is a random oracle, then  $\Pi_{\text{prnd}}$  is str-prnd and obliv-k+.*

**Acknowledgments.** This research was partially funded by the HPI Research School on Systems Design. We sincerely thank Joseph Jaeger for his valuable feedback and generous support in revising the first version of this paper. In particular, he pointed out issues in the proofs of Lemma 5 and Lemma 15 and suggested a correction, which we have incorporated into this version.

## References

- ADDG24. Martin R. Albrecht, Alex Davidson, Amit Deo, and Daniel Gardham, *Crypto dark matter on the torus*, Advances in Cryptology – EUROCRYPT 2024 (Cham) (Marc Joye and Gregor Leander, eds.), Springer Nature Switzerland, 2024, pp. 447–476.
- ADDS21. Martin R. Albrecht, Alex Davidson, Amit Deo, and Nigel P. Smart, *Round-optimal verifiable oblivious pseudorandom functions from ideal lattices*, Public-Key Cryptography – PKC 2021 (Cham) (Juan A. Garay, ed.), Springer International Publishing, 2021, pp. 261–289.
- AG24. Martin R. Albrecht and Kamil Doruk Gur, *Verifiable oblivious pseudorandom functions from lattices: Practical-ish and thresholdisable*, Advances in Cryptology – ASIACRYPT 2024 (Singapore) (Kai-Min Chung and Yu Sasaki, eds.), Springer Nature Singapore, 2024, pp. 205–237.
- Bas23. Andrea Basso, *A post-quantum round-optimal oblivious PRF from isogenies*, Cryptology ePrint Archive, Paper 2023/225, 2023.
- BBC<sup>+</sup>24. Carsten Baum, Jens Berlips, Walther Chen, Ivan B. Damgård, Kevin M. Esvelt, Leonard Foner, Dana Gretton, Martin Kysel, Ronald L. Rivest, Lawrence Roy, Francesca Sage-Ling, Adi Shamir, Vinod Vaikuntanathan, Lynn Van Hauwe, Theia Vogel, Benjamin Weinstein-Raun, Daniel Wicks, Stephen Wooster, Andrew C. Yao, and Yu Yu, *Efficient maliciously secure oblivious exponentiations*, IACR Communications in Cryptology **1** (2024), no. 3 (English).
- BDFH25. Ward Beullens, Lucas Dodgson, Sebastian Faller, and Julia Hesse, *The 2hash oprf framework and efficient post-quantum instantiations*, Advances in Cryptology – EUROCRYPT 2025 (Cham) (Serge Fehr and Pierre-Alain Fouque, eds.), Springer Nature Switzerland, 2025, pp. 332–362.
- BDG<sup>+</sup>24. Matilda Backendal, Hannah Davis, Felix Günther, Miro Haller, and Kenneth G. Paterson, *A formal treatment of end-to-end encrypted cloud storage*, Advances in Cryptology – CRYPTO 2024, Springer Nature Switzerland, 2024, pp. 40–74.
- BFH<sup>+</sup>20. Carsten Baum, Tore Frederiksen, Julia Hesse, Anja Lehmann, and Avishay Yanai, *Pesto: Proactively secure distributed single sign-on, or how to trust a hacked server*, 2020 IEEE European Symposium on Security and Privacy (EuroS&P), IEEE, September 2020.

- BFP21. Balthazar Bauer, Georg Fuchsbauer, and Antoine Plouviez, *The one-more discrete logarithm assumption in the generic group model*, Advances in Cryptology – ASIACRYPT 2021 (Cham) (Mehdi Tibouchi and Huaxiong Wang, eds.), Springer International Publishing, 2021, pp. 587–617.
- BKLW24. Daniel Bourdreux, Hugo Krawczyk, Kevin Lewi, and Christopher A. Wood, *The OPAQUE Augmented PAKE Protocol*, Internet-Draft draft-irtf-cfrg-opaque-18, November 2024, Work in Progress.
- BKW20. Dan Boneh, Dmitry Kogan, and Katharine Woo, *Oblivious pseudorandom functions from isogenies*, Advances in Cryptology – ASIACRYPT 2020 (Cham) (Shiho Moriai and Huaxiong Wang, eds.), Springer International Publishing, 2020, pp. 520–550.
- Bol03. Alexandra Boldyreva, *Threshold signatures, multisignatures and blind signatures based on the gap-diffie-hellman-group signature scheme*, Public Key Cryptography — PKC 2003 (Berlin, Heidelberg) (Yvo G. Desmedt, ed.), Springer Berlin Heidelberg, 2003, pp. 31–46.
- BR04. Mihir Bellare and Phillip Rogaway, *Code-based game-playing proofs and the security of triple encryption.*, IACR Cryptology ePrint Archive **2004** (2004), 331.
- BRRA24. M. Bellare, R. Ranjan, D. Riepel, and A. Aldakheel, *The concrete security of two-party computation: Simple definitions, and tight proofs for psi and oprfs*, Advances in Cryptology – ASIACRYPT 2024 (Singapore) (Kai-Min Chung and Yu Sasaki, eds.), Springer Nature Singapore, 2024, pp. 328–362.
- Can01. R. Canetti, *Universally composable security: a new paradigm for cryptographic protocols*, Proceedings 42nd IEEE Symposium on Foundations of Computer Science, 2001, pp. 136–145.
- CATZ24. Rutchathon Chairattana-Apirom, Stefano Tessaro, and Chenzhi Zhu, *Pairing-free blind signatures from cdh assumptions*, Advances in Cryptology – CRYPTO 2024 (Cham) (Leonid Reyzin and Douglas Stebila, eds.), Springer Nature Switzerland, 2024, pp. 174–209.
- CDCGS19. Jan Camenisch, Angelo De Caro, Esha Ghosh, and Alessandro Sorniotti, *Oblivious prf on committed vector inputs and application to deduplication of encrypted data*, Financial Cryptography and Data Security: 23rd International Conference, FC 2019, Frigate Bay, St. Kitts and Nevis, February 18–22, 2019, Revised Selected Papers (Berlin, Heidelberg), Springer-Verlag, 2019, p. 337–356.
- CDVW24. Sofia Celi, Alex Davidson, Steven Valdez, and Christopher A. Wood, *Privacy Pass Issuance Protocols*, RFC 9578, June 2024.
- CHL22. Sílvia Casacuberta, Julia Hesse, and Anja Lehmann, *Sok: Oblivious pseudorandom functions*, 2022 IEEE 7th European Symposium on Security and Privacy (EuroS&P), 2022, pp. 625–646.
- CLN15. Jan Camenisch, Anja Lehmann, and Gregory Neven, *Optimal distributed password verification*, Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, CCS’15, ACM, October 2015, pp. 182–194.
- CS97. Camenisch, Jan and Stadler, Markus, *Proof systems for general statements about discrete logarithms*, Tech. report, 1997.
- DFHSW23. Alex Davidson, Armando Faz-Hernandez, Nick Sullivan, and Christopher A. Wood, *Oblivious Pseudorandom Functions (OPRFs) Using Prime-Order Groups*, RFC 9497, December 2023.

- DGS<sup>+</sup>18. Alex Davidson, Ian Goldberg, Nick Sullivan, George Tankersley, and Filippo Valsorda, *Privacy pass: Bypassing internet challenges anonymously*, Proceedings on Privacy Enhancing Technologies **2018** (2018), no. 3, 164–180.
- DHL22. Poulami Das, Julia Hesse, and Anja Lehmann, *Dpase: Distributed password-authenticated symmetric-key encryption, or how to get many keys from one password*, Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security, ASIA CCS '22, ACM, May 2022.
- DL24. Dennis Dayanikli and Anja Lehmann, *(strong) apake revisited: Capturing multi-user security and salting*, 2024 IEEE 9th European Symposium on Security and Privacy (EuroS&P), 2024, pp. 415–439.
- Don22. Minglang Dong, *Multi-point hashdh oprf using multiplicative blinding with application to private set intersection*, IACR Cryptol. ePrint Arch. **2022** (2022), 1482.
- DY05. Yevgeniy Dodis and Aleksandr Yampolskiy, *A verifiable random function with short proofs and keys*, Public Key Cryptography - PKC 2005 (Berlin, Heidelberg) (Serge Vaudenay, ed.), Springer Berlin Heidelberg, 2005, pp. 416–431.
- ECS<sup>+</sup>15. Adam Everspaugh, Rahul Chaterjee, Samuel Scott, Ari Juels, and Thomas Ristenpart, *The Pythia PRF Service*, 24th USENIX Security Symposium (USENIX Security 15) (Washington, D.C.), USENIX Association, August 2015, pp. 547–562.
- ESTX24. Muhammed F. Esgin, Ron Steinfeld, Erkan Tairi, and Jie Xu, *Leopard: Towards practical post-quantum oblivious prfs via interactive lattice problems*, IACR Cryptol. ePrint Arch. **2024** (2024), 1615.
- FHH<sup>+</sup>24. Sebastian Faller, Tobias Handirk, Julia Hesse, Máté Horváth, and Anja Lehmann, *Password-protected key retrieval with(out) hsm protection*, Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security (New York, NY, USA), CCS '24, Association for Computing Machinery, 2024, p. 2445–2459.
- FHSS<sup>+</sup>23. Armando Faz-Hernandez, Sam Scott, Nick Sullivan, Riad S. Wahby, and Christopher A. Wood, *Hashing to Elliptic Curves*, RFC 9380, August 2023.
- FHV13. Sebastian Faust, Carmit Hazay, and Daniele Venturi, *Outsourced pattern matching*, International Journal of Information Security **17** (2013), 327 – 346.
- FIPR05. Michael J. Freedman, Yuval Ishai, Benny Pinkas, and Omer Reingold, *Keyword search and oblivious pseudorandom functions*, Theory of Cryptography (Berlin, Heidelberg) (Joe Kilian, ed.), Springer Berlin Heidelberg, 2005, pp. 303–324.
- FKL18. Georg Fuchsbauer, Eike Kiltz, and Julian Loss, *The algebraic group model and its applications*, Advances in Cryptology – CRYPTO 2018 (Cham) (Hovav Shacham and Alexandra Boldyreva, eds.), Springer International Publishing, 2018, pp. 33–62.
- GT15. Peter Gaži and Stefano Tessaro, *Secret-key cryptography from ideal primitives: A systematic overview*, 2015 IEEE Information Theory Workshop (ITW), 2015, pp. 1–5.
- HHM<sup>+</sup>24. Lena Heimberger, Tobias Hennerbichler, Fredrik Meisingseth, Sebastian Ramacher, and Christian Rechberger, *Oprfs from isogenies: Designs and analysis*, Proceedings of the 19th ACM Asia Conference on Computer and Communications Security, ASIA CCS '24, ACM, July 2024.



- HL08. Carmit Hazay and Yehuda Lindell, *Efficient protocols for set intersection and pattern matching with security against malicious and covert adversaries*, Theory of Cryptography (Berlin, Heidelberg) (Ran Canetti, ed.), Springer Berlin Heidelberg, 2008, pp. 155–175.
- JKK14. Stanisław Jarecki, Aggelos Kiayias, and Hugo Krawczyk, *Round-optimal password-protected secret sharing and t-pake in the password-only model*, Advances in Cryptology – ASIACRYPT 2014, Springer Berlin Heidelberg, 2014, pp. 233–253.
- JKKX16. Stanisław Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu, *Highly-efficient and composable password-protected secret sharing (or: How to protect your bitcoin wallet online)*, 2016 IEEE European Symposium on Security and Privacy (EuroS&P), IEEE, March 2016.
- JKKX17. Stanisław Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu, *Toppss: Cost-minimal password-protected secret sharing based on threshold oprf*, Applied Cryptography and Network Security (Cham) (Dieter Gollmann, Atsuko Miyaji, and Hiroaki Kikuchi, eds.), Springer International Publishing, 2017, pp. 39–58.
- JKX18. Stanisław Jarecki, Hugo Krawczyk, and Jiayu Xu, *Opaque: An asymmetric pake protocol secure against pre-computation attacks*, Advances in Cryptology – EUROCRYPT 2018 (Cham) (Jesper Buus Nielsen and Vincent Rijmen, eds.), Springer International Publishing, 2018, pp. 456–486.
- JKX21. Stanisław Jarecki, Hugo Krawczyk, and Jiayu Xu, *On the (in)security of the diffie-hellman oblivious prf with multiplicative blinding*, Public-Key Cryptography – PKC 2021 (Cham) (Juan A. Garay, ed.), Springer International Publishing, 2021, pp. 380–409.
- JL09. Stanisław Jarecki and Xiaomin Liu, *Efficient oblivious pseudorandom function with applications to adaptive ot and secure computation of set intersection*, Theory of Cryptography (Berlin, Heidelberg) (Omer Reingold, ed.), Springer Berlin Heidelberg, 2009, pp. 577–594.
- JL10. ———, *Fast secure computation of set intersection*, Security and Cryptography for Networks (Berlin, Heidelberg) (Juan A. Garay and Roberto De Prisco, eds.), Springer Berlin Heidelberg, 2010, pp. 418–435.
- KCM24. Novak Kaluderovic, Nan Cheng, and Aikaterini Mitrokotsa, *A post-quantum distributed oprf from the legendre prf*, IACR Cryptol. ePrint Arch. **2024** (2024), 544.
- KMP<sup>+</sup>17. Vladimir Kolesnikov, Naor Matania, Benny Pinkas, Mike Rosulek, and Ni Trieu, *Practical multi-party private set intersection from symmetric-key techniques*, Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (New York, NY, USA), CCS '17, Association for Computing Machinery, 2017, p. 1257–1272.
- Leh19. Anja Lehmann, *ScrambleDB: Oblivious (Chameleon) Pseudonymization-as-a-Service*, Proceedings on Privacy Enhancing Technologies **2019** (2019), no. 3, 289–309.
- NPR99. Moni Naor, Benny Pinkas, and Omer Reingold, *Distributed pseudorandom functions and kdcs*, Advances in Cryptology — EUROCRYPT '99 (Berlin, Heidelberg) (Jacques Stern, ed.), Springer Berlin Heidelberg, 1999, pp. 327–346.
- Sho04. Victor Shoup, *Sequences of games: a tool for taming complexity in security proofs*, IACR Cryptol. ePrint Arch. **2004** (2004), 332.

- TCR<sup>+</sup>22. Nirvan Tyagi, Sofia Celi, Thomas Ristenpart, Nick Sullivan, Stefano Tessaro, and Christopher A. Wood, *A fast and simple partially oblivious prf, with applications*, Advances in Cryptology – EUROCRYPT 2022 (Cham) (Orr Dunkelman and Stefan Dziembowski, eds.), Springer International Publishing, 2022, pp. 674–705.

## A Related Work

In this section, we summarize the related work that we have discussed throughout the main body of our work. Tab. 1 gives an overview of the related works that introduce and use game-based definitions of OPRFs. The only other works we are aware of that use game-based definitions (this time, not introducing any new games) are by Heimberger et al. [HHM<sup>+</sup>24] and Esgin et al. [ESTX24]. This demonstrates the scarcity of game-based works on OPRFs and the inconsistency in formalization, with almost all works coming up with their own formalizations. In the following, we give a summary of the existing game-based definitions.

|                       | [ECS <sup>+</sup> 15] | [Leh19] | [TCR <sup>+</sup> 22] | [ADDG24] | [AG24] | [BRRA24]        |
|-----------------------|-----------------------|---------|-----------------------|----------|--------|-----------------|
| Unpredictability      | ★ <sup>(H)</sup>      | ★       |                       |          | ○      |                 |
| Pseudorandomness      | ★ <sup>(H)</sup>      | ★       | ★ <sup>2H</sup>       | ○        | ○      | ★ <sup>2H</sup> |
| Obliviousness         |                       | ★       | ★ <sup>(2H)</sup>     | ○        | ○      | ★ <sup>2H</sup> |
| Partial obliviousness | ★                     |         | ★                     | ○        |        |                 |
| Verifiability         |                       |         |                       | ★        | ○      |                 |
| Threshold             |                       |         |                       |          | ★      |                 |

**Table 1.** Overview of related works proposing and/or using game-based security properties for OPRFs.

★ new definition      ○ use existing definition      <sup>H</sup> (derivable) proof for HashDH      <sup>2H</sup> (derivable) proof for 2HashDH

*Existing Game-Based Definitions.* Firstly, Everspaugh et al. [ECS<sup>+</sup>15] propose an OPRF-as-a-service system called *Pythia*. Their construction aims to be verifiable and is the first occurrence of partial obliviousness, but only the latter feature is formally treated. Concretely, they use a pairing-based version of HashDH. They are the first to propose a game-based analysis of OPRFs and define one-more unpredictability, which was in the following adopted by other works, and one-more pseudorandomness.

Lehmann [Leh19] treats as part of the pseudonymization service *ScrambleDB* a specialized three-party, convertible OPRF. Here, the requesting party is split into two: One party sends an input, the other receives the evaluation. This setting gives rise to multiple additional security requirements for different corruption

and collusion scenarios, but the standard properties one-more unpredictability, pseudorandomness, and obliviousness, are also treated. Further, definitions where the requester and receiver can be different parties, become equivalent to definitions for a standard OPRF if the requester and receiver are the same entity. For instance, *collusion-resistance* aims at pseudorandomness in the presence of corrupt and colluding requester and receiver, which corresponds to pseudorandomness with an actively corrupt requester in the 2-party setting. Additionally, *convertibility* allows to transform evaluations to be valid under a different key, using some form key rotation.

Tyagi et al. [TCR<sup>+</sup>22] present a new OPRF construction *3HashSDHI*, which combines properties of 2HashDH and the Dodis-Yampolskiy OPRF [DY05]. They provide security games with different attacker models and build on top of previous works. A strong notion of pseudorandomness as well as two versions of obliviousness are presented, where all definitions implicitly assume partial obliviousness.

Albrecht et al. [ADDG24] construct a partially oblivious PRF from lattice assumptions. They make use of the aforementioned security definitions and introduce a new game for verifiability.

Albrecht and Gur [AG24] improve efficiency of a previous lattice-based OPRF by Albrecht et al. [ADDS21]. They slightly modify the previous verifiability definition, and propose a new definition for threshold computation of OPRFs.

In a more theoretical work, Bellare et al. [BRRA24] reflect on security definitions for two-party-computation (2PC) protocols that favor concrete results and argue game-based definitions are the appropriate choice. As a justification, the authors carefully establish a relation between simulation-based and game-based definitions and determine conditions under which the two are equivalent. For OPRFs specifically, they propose a pseudorandomness definition and develop a generic *input indistinguishability* property which corresponds to obliviousness if an OPRF is plugged in as the 2PC protocol.

Beullens et al. [BDFH25] present the generic OPRF construction *2HashOPRF* which yields a promising candidate for a practical post-quantum OPRF with malicious security. While the authors perform the main analysis in the UC framework, they also define unpredictability for function families in a game-based fashion.

*Game-based Analysis of (2)HashDH.* The two most common classical OPRF constructions HashDH and 2HashDH have so far received limited attention in the game-based model (as shown in Tab. 1), and the few contributions unfortunately use incomparable properties.

2HashDH, which is well-understood in the UC framework, is shown by Bellare et al. [BRRA24] pseudorandom and oblivious against honest-but-curious attackers. Tyagi et al. [TCR<sup>+</sup>22] provide only proof sketches for 2HashDH in analogy to their OPRF 3HashSDHI, covering a stronger pseudorandomness and obliviousness property.

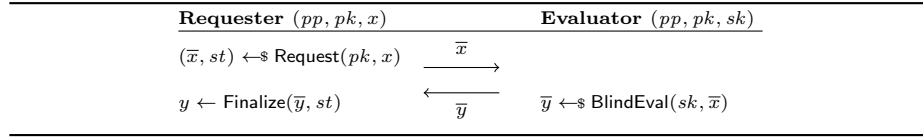
For HashDH, we find no direct proofs, which is surprising since there is no UC-based analysis, either. However, the proofs by Everspaugh et al. [ECS<sup>+</sup>15]

for their pairing-based version of HashDH can be adapted in a straightforward way to standard cyclic groups. The authors cover one-more unpredictability as well as one-more pseudorandomness.

## B Overview of the OPRF exchange

Here we give a visual overview of the OPRF exchange between requester and evaluator in Fig. 6.

Prior to the exchange, the **Setup** and **KGen** algorithms are executed to generate public parameters and the evaluator’s keys, respectively.



**Fig. 6.** Interactive OPRF evaluation.

## C Correctness

We state the missing correctness definition from Section 3. Correctness ensures consistency between the interactive and non-interactive computation of an OPRF, with the simplest definition being the following:

**Definition 9 (Correctness).** *An OPRF  $\Pi$  is correct, if*

$$\forall \lambda, pp \leftarrow \text{Setup}(\lambda), (pk, sk) \leftarrow \text{KGen}(pp), x \in \mathcal{X}, (\bar{x}, st) \leftarrow \text{Request}(pk, x) : \\ \text{Finalize}(\text{BlindEval}(sk, \bar{x}), st) = \text{Eval}(sk, x)$$

Note that some OPRF constructions might rely on computational assumptions that cannot ensure correctness in all cases but have to account for rare failures even during honest execution. For this case, Bellare et al. [BRRA24] propose a notion of *imperfect* correctness captured in a game, expressing how hard it is to find inputs that make the protocol fail. In this work, we only encounter perfectly correct OPRF constructions for simplicity. Our results can be generalized for imperfectly correct schemes easily.

## D Computational Assumptions

This section contains the full version of all hardness assumptions needed throughout our work.

We give the security games to the computational assumptions in Fig. 7 and define CDH-type and DDH-type assumptions separately as follows.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                              |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{Game}_{\mathcal{A}, \text{GGen}}^{\text{OM-CDH}}(\lambda), \text{Game}_{\mathcal{A}, \text{GGen}}^{\text{gap-OM-CDH}}(\lambda)$                                                                                                                                                                                                                                                                                                                                                                        | $\mathcal{O}_{\text{Chal}}$                                                                                                                                                                                                                                   | $\mathcal{O}_{\text{DDH}}(A, B, C)$                                                                                                                                                                                          |
| $(\mathbb{G}, q, g) \leftarrow \text{GGen}(\lambda)$<br>$x \leftarrow \mathbb{Z}_q$<br>$c_{\text{Chal}} := 0; c_{\text{CDH}} := 0$<br>$(i_1, Z_1), \dots, (i_\ell, Z_\ell) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Chal}}, \mathcal{O}_{\text{CDH}}, \mathcal{O}_{\text{DDH}}}((\mathbb{G}, q, g), g^x)$<br><b>if</b> $\ell \leq c_{\text{CDH}} \vee \exists \alpha \neq \beta : i_\alpha = i_\beta$ : <b>return</b> 0<br><b>return</b> $\forall \alpha : Z_\alpha \stackrel{?}{=} Y_{i_\alpha}^x$         | $c_{\text{Chal}} := c_{\text{Chal}} + 1$<br>$Y_{c_{\text{Chal}}} \leftarrow \mathbb{G}$<br><b>return</b> $Y_{c_{\text{Chal}}}$<br>$\mathcal{O}_{\text{CDH}}(Y)$<br>$c_{\text{CDH}} := c_{\text{CDH}} + 1$<br><b>return</b> $Y^x$                              | $a := \log_g(A)$<br>$b := \log_g(B)$<br>$c := \log_g(C)$<br><b>return</b> $c \stackrel{?}{=} ab$                                                                                                                             |
| $\text{Game}_{\mathcal{A}, \text{GGen}}^{\text{hint-CDH}}(\lambda)$                                                                                                                                                                                                                                                                                                                                                                                                                                           | $\text{Game}_{\mathcal{A}, \text{GGen}}^{\text{DDH-mult}}(\lambda)$                                                                                                                                                                                           | $\mathcal{O}_{\text{Chal}}$                                                                                                                                                                                                  |
| $(\mathbb{G}, q, g) \leftarrow \text{GGen}(\lambda)$<br>$x \leftarrow \mathbb{Z}_q; Y^* \leftarrow \mathbb{G}$<br>$Y \leftarrow \mathcal{A}((\mathbb{G}, q, g))$<br><b>assert</b> $Y \in \mathbb{G}$<br>$Z^* \leftarrow \mathcal{A}(Y^x, Y^*)$<br><b>return</b> $Z^* \stackrel{?}{=} (Y^*)^x$                                                                                                                                                                                                                 | $(\mathbb{G}, q, g) \leftarrow \text{GGen}(\lambda)$<br>$b \leftarrow \{0, 1\}; x \leftarrow \mathbb{Z}_q$<br>$b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Chal}}}((\mathbb{G}, q, g), g^x)$<br><b>return</b> $b' \stackrel{?}{=} b$                        | $y \leftarrow \mathbb{Z}_q$<br><b>if</b> $b = 0$ : $z \leftarrow \mathbb{Z}_q$<br><b>else</b> : $z := xy$<br><b>return</b> $(g^y, g^z)$                                                                                      |
| $\text{Game}_{\mathcal{A}, \text{GGen}}^{\text{OM-DDH}}(\lambda)$                                                                                                                                                                                                                                                                                                                                                                                                                                             | $\mathcal{O}_{\text{Chal}}(i)$                                                                                                                                                                                                                                | $\mathcal{O}_{\text{CDH}}(Y)$                                                                                                                                                                                                |
| $(\mathbb{G}, q, g) \leftarrow \text{GGen}(\lambda)$<br>$\mathbf{b} := (); x \leftarrow \mathbb{Z}_q; c_{\text{tgt}} := 0$<br>$(i_1, \dots, i_\ell, b') \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Chal}}}((\mathbb{G}, q, g), g^x)$<br><b>if</b> $\exists \alpha : i_\alpha \geq  \mathbf{b}  \vee \ell \leq c_{\text{CDH}}$ : <b>return</b> 0<br><b>if</b> $\exists \alpha \neq \beta : i_\alpha = i_\beta$ : <b>return</b> 0<br><b>return</b> $b' \stackrel{?}{=} \bigoplus_{\alpha=1}^{\ell} b[i_\alpha]$ | <b>if</b> $i > c_{\text{tgt}}$ : <b>return</b> $\perp$<br>$b \leftarrow \{0, 1\}$<br>Append $b$ to $\mathbf{b}$<br><b>if</b> $b = 0$ : $Z \leftarrow \mathbb{G}$<br>// DH pair from $i^{\text{th}}$ target<br><b>else</b> : $Z := Y_i^x$<br><b>return</b> $Z$ | $c_{\text{CDH}} := c_{\text{CDH}} + 1$<br><b>return</b> $Y^x$<br>$\mathcal{O}_{\text{Target}}$<br>$c_{\text{tgt}} := c_{\text{tgt}} + 1$<br>$Y_{c_{\text{tgt}}} \leftarrow \mathbb{G}$<br><b>return</b> $Y_{c_{\text{tgt}}}$ |

**Fig. 7.** Games for the OM-CDH, gap-OM-CDH, hint-CDH, DDH-mult and OM-DDH computational assumptions.

**Definition 10 (CDH).** A problem  $x$ -CDH for  $x \in \{\text{OM}, \text{gap-OM}, \text{hint}\}$  is considered computationally hard for  $\text{GGen}$  if  $\forall$  PPT-adversaries  $\mathcal{A}$  the following advantage is negligible:

$$\text{Adv}_{\mathcal{A}, \text{GGen}}^{x\text{-cdh}}(\lambda) := \Pr \left[ \text{Game}_{\mathcal{A}, \text{GGen}}^{x\text{-CDH}}(\lambda) \right]$$

**Definition 11 (DDH).** A problem  $x$ -DDH for  $x \in \{\text{mult}, \text{OM}\}$  is considered computationally hard for  $\text{GGen}$  if  $\forall$  PPT-adversaries  $\mathcal{A}$  the following advantage is negligible:

$$\text{Adv}_{\mathcal{A}, \text{GGen}}^{x\text{-ddh}}(\lambda) := 2 \cdot \Pr \left[ \text{Game}_{\mathcal{A}, \text{GGen}}^{x\text{-DDH}}(\lambda) \right] - 1$$

*One-More CDH.* OM-CDH is a standard variant of CDH, formalized for example by Chairattana-Apirom et al. [CATZ24]. Here, instead of solving a single CDH challenge, the adversary gets to see the answer to  $\ell - 1$  challenges, and must only solve a final, fresh challenge themselves. It is assumed to be hard in any group where CDH is hard [Bol03]. Bauer et al. [BFP21] show the assumption to be secure in the Generic Group Model (GGM).

*Gap-One-More CDH.* The variant gap-OM-CDH was introduced by Jarecki et al. [JKK14]. In comparison to the aforementioned OM-CDH, the adversary is made stronger by being given the ability to verify DH pairs via a DDH oracle. This requires groups where the DDH problem is easy, but the CDH problem is assumed to be hard. Tyagi et al. [TCR<sup>+</sup>22] argue that the same bounds Bauer et al. [BFP21] showed for the security of OM-CDH should also hold for the gap version.

*Hint CDH.* hint-CDH is a new assumption. The adversary gets a single DH pair as a hint instead of a one-more CDH oracle. But in contrast to standard assumptions, we allow the adversary to choose the group element to be exponentiated for the hint, themselves. After receiving the hint, the adversary is given a single CDH challenge in the second game phase. We provide a reduction to the standard DL assumption in the Algebraic Group Model in Appendix D.1.

*DDH-mult.* DDH-mult is a simple extension of the classic DDH problem, where instead of a single DDH challenge, the adversary gets to see many, which are all either random or DH pairs. Bellare et al. [BRRA24] show a very similar version to the formalization of DDH-mult in Fig. 7 to be equivalent to DDH, from which equivalence of our DDH-mult is trivially derivable.

*One-More DDH.* Everspaugh et al. [ECS<sup>+</sup>15] introduce a tailored bilinear OM-DDH assumption in their work to prove one-more pseudorandomness of a bilinear HashDH variant. Here, the adversary again receives many DDH challenges, but this time each of the challenges can be from a different world, and the adversary ultimately has to guess correctly the world of  $\ell$  distinct challenges. As an aid, a CDH oracle is provided, to which the adversary may make  $\ell - 1$

queries. OM-DDH is a rather non-standard assumption, and the structural similarity to the one-more pseudorandomness game is apparent. For our work, we adapt Everspaugh et al.'s bilinear definition to our standard (i.e., non bilinear) group setting. This transformation is straightforward: Whereas Everspaugh et al. generate challenges in the target group  $\mathbb{G}_T$  we generate them simply in  $\mathbb{G}$ . Moreover, we only give one target oracle for obtaining random elements in  $\mathbb{G}$ , whereas Everspaugh et al. granted two, for  $\mathbb{G}_1$  and  $\mathbb{G}_2$ , respectively.

### D.1 Hardness of the hint-CDH Assumption

In order to show that HashDH is not strongly pseudorandom, we introduced the hint-CDH assumption, which to the best of our knowledge is new. In comparison with the regular CDH assumption, the adversary gets to see a hint before facing the challenge: They may select an arbitrary group element  $Y$  and receive the CDH response  $Y^x$ . The modified CDH game is given in Fig. 7.

| $\text{Game}_{\mathcal{A}, \text{GGen}}^{\text{DL}}(\lambda)$ | $\mathcal{A}_{\text{DL}}((\mathbb{G}, q, g), Y^*)$                                               |
|---------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| $(\mathbb{G}, q, g) \leftarrow \text{GGen}(\lambda)$          | $(Y, (z_1)) \leftarrow \mathcal{A}_{\text{hint-CDH}}((\mathbb{G}, q, g))$                        |
| $x \leftarrow \mathbb{Z}_q; \quad Y^* := g^x$                 | $c \leftarrow \mathbb{Z}_q \quad // \text{ Random challenge for } \mathcal{A}_{\text{hint-CDH}}$ |
| $x' \leftarrow \mathcal{A}((\mathbb{G}, q, g), Y^*)$          | <b>if</b> $c = 0$ : <b>abort</b>                                                                 |
| <b>return</b> $x' \stackrel{?}{=} x$                          | $(Z, (z'_1, z'_2, z'_3)) \leftarrow \mathcal{A}_{\text{hint-CDH}}((Y^*)^{z_1}, (Y^*)^c)$         |
|                                                               | <b>return</b> Solve for $x$ and return                                                           |

**Fig. 8. Left:** Game for the DL problem. **Right:** Adversary  $\mathcal{A}_{\text{DL}}$  for the reduction between DL and hint-CDH.

To establish some confidence in the hardness of this assumption, we show in the following a reduction to the DL problem in the AGM [FKL18]. Let us first give a standard definition of the DL problem; the game is shown in Fig. 8.

**Definition 12 (DL).** *The DL problem is considered hard for GGen if  $\forall$  PPT-adversaries  $\mathcal{A}$  the following advantage is negligible:*

$$\text{Adv}_{\mathcal{A}, \text{GGen}}^{\text{dl}}(\lambda) := \Pr \left[ \text{Game}_{\mathcal{A}, \text{GGen}}^{\text{DL}}(\lambda) \right]$$

**Lemma 12.** *If the DL problem is hard, then the hint-CDH problem is also hard in the AGM.*

*Proof.* The AGM assumes that adversary are unable to find group elements on their own, but can only construct them from elements they have previously seen. Concretely, using a group  $\mathbb{G}$  of order  $q$ , this is formalized by forcing the adversary to give a representation  $\mathbf{z} = (z_i, \dots, z_t) \in \mathbb{Z}_q^t$  of each group element  $Z$  they output, such that  $Z = \prod_i^t L_i^{z_i}$  where  $\mathbf{L} = (L_1, \dots, L_t) \in \mathbb{G}^t$  are all group elements previously seen by the adversary.

Let  $\mathcal{A}_{\text{DL}}$  be an adversary playing  $\text{Game}_{\mathcal{A}, \text{GGen}}^{\text{DL}}$  and  $\mathcal{A}_{\text{hint-CDH}}$  an *algebraic* adversary in  $\text{Game}_{\mathcal{A}, \text{GGen}}^{\text{hint-CDH}}$ . For the sake of a contradiction, let us assume there exists a non-negligible function  $g(\lambda)$  such that

$$\Pr \left[ \text{Game}_{\mathcal{A}_{\text{hint-CDH}}, \text{GGen}}^{\text{hint-CDH}}(\lambda) \right] = g(\lambda)$$

We let  $\mathcal{A}_{\text{DL}}$  simulate the hint-CDH game and use the output of  $\mathcal{A}_{\text{hint-CDH}}$  to win the DL game as follows (shown in Fig. 8): At the beginning,  $\mathcal{A}_{\text{DL}}$  simply forwards the group parameters to  $\mathcal{A}_{\text{hint-CDH}}$ . The latter must now choose the base  $Y$  for their CDH hint and simultaneously output a representation for  $Y$  from the known group elements. But the only group element given to  $\mathcal{A}_{\text{hint-CDH}}$  is the generator  $g$ , therefore we know  $\mathcal{A}_{\text{hint-CDH}}$  will send  $(Y, (z_1))$  such that  $Y = g^{z_1}$ .  $\mathcal{A}_{\text{DL}}$  answers this challenge using their own DL challenge  $Y^*$ , by computing  $(Y^*)^{z_1}$ . This is exactly  $Y^x$ , since  $(Y^*)^{z_1} = (g^x)^{z_1} = (g^{z_1})^x = Y^x$ . In addition to the hint,  $\mathcal{A}_{\text{DL}}$  creates a CDH challenge  $(Y^*)^c$  with some  $c \leftarrow \mathbb{Z}_q$ . We get an abort condition in  $1/q$  of the cases since we need  $c$  to be non-zero, but this happens only with negligible probability.

To solve the challenge and win the hint-CDH game,  $\mathcal{A}_{\text{hint-CDH}}$  must compute  $((Y^*)^c)^x$ . They return their attempt  $Z$  and its representation  $(z'_1, z'_2, z'_3)$  from the now three known elements  $g, (Y^*)^{z_1}, (Y^*)^c$ . If  $Z$  is indeed the required  $((Y^*)^c)^x$ , then this representation leaks the discrete logarithm of  $Y^*$ : We know that  $((Y^*)^c)^x = g^{z'_1} \cdot ((Y^*)^{z_1})^{z'_2} \cdot ((Y^*)^c)^{z'_3}$ , or in a simplified form,  $g^{c \cdot x^2} = g^{z'_1} \cdot g^{x \cdot z_1 \cdot z'_2} \cdot g^{x \cdot c \cdot z'_3}$ . From the exponents, we therefore get the equivalence  $cx^2 \equiv z'_1 + xz_1z'_2 + xcz'_3 \pmod{q}$ , where all variables except  $x$  are known and  $c \neq 0$ . If any  $z_i = 0$  then that summand is removed, and  $\mathcal{A}_{\text{DL}}$  can easily compute  $x$  from the remaining terms.

As a result, we conclude that

$$\Pr \left[ \text{Game}_{\mathcal{A}_{\text{DL}}, \text{GGen}}^{\text{DL}}(\lambda) \right] = \Pr \left[ \text{Game}_{\mathcal{A}_{\text{hint-CDH}}, \text{GGen}}^{\text{hint-CDH}}(\lambda) \right] + 1/q$$

But that contradicts the assumption that DL is hard, so we conclude that no such successful  $\mathcal{A}_{\text{hint-CDH}}$  can exist.

## E Discussion of dismissed property variants

### E.1 Unpredictability with a standard evaluation oracle

Note that in our definition of one-more unpredictability, we only provided a `BlindEval` oracle to the malicious requester— adding a second oracle for standard evaluation might seem an intuitive extension. Indeed, some OPRF constructions allow a requester to obtain evaluations non-blindly if they deviate from the protocol: for instance, in HashDH, the `BlindEval` algorithm consists of raising the input to the evaluator's key. A requester could skip the blinding, simply sending the hash of their input – and would receive a valid PRF output as a response. For illustration, we show an alternative unpredictability definition with an additional non-blind evaluation oracle in Fig. 9.



| $\text{Game}_{\Pi, \mathcal{A}}^{\text{unpred}}(\lambda), \text{Game}_{\Pi, \mathcal{A}}^{\text{unpred-eval}}(\lambda)$          | $\mathcal{O}_{\text{Eval}}(x)$                |
|----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| $pp \leftarrow \text{Setup}(\lambda); (pk, sk) \leftarrow \text{KGen}(pp)$                                                       | $c_{\text{Ev}} := c_{\text{Ev}} + 1$          |
| $c_{\text{Ev}} := 0$                                                                                                             | <b>return</b> $\text{Eval}(sk, x)$            |
| $(x_1, y_1), \dots, (x_\ell, y_\ell) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Eval}}, \mathcal{O}_{\text{BlindEval}}}(pp, pk)$ | $\mathcal{O}_{\text{BlindEval}}(\bar{x})$     |
| <b>if</b> $\ell \leq c_{\text{Ev}} \vee \exists \alpha \neq \beta : x_\alpha = x_\beta$ <b>return</b> 0                          | $c_{\text{Ev}} := c_{\text{Ev}} + 1$          |
| <b>return</b> $\forall \alpha : \text{Eval}(sk, x_\alpha) \stackrel{?}{=} y_\alpha$                                              | <b>return</b> $\text{BlindEval}(sk, \bar{x})$ |

**Fig. 9.** Standard one-more unpredictability and alternative definition with an oracle for standard evaluation.

Fortunately, it turns out that the alternative definition is equivalent to  $\text{Game}_{\Pi, \mathcal{A}}^{\text{unpred}}$ , given *correctness*. The proof is straightforward, so we provide only a brief sketch below.

**Lemma 13.** *For a correct OPRF protocol  $\Pi$ , one-more unpredictability according to  $\text{Game}_{\Pi, \mathcal{A}}^{\text{unpred}}$  and  $\text{Game}_{\Pi, \mathcal{A}}^{\text{unpred-eval}}$  are equivalent.*

*Proof (sketch).* It is trivial to show that **unpred-eval** implies **unpred** – winning conditions are unchanged and the adversary is granted an additional oracle.

The reverse implication is straightforward, too. Assume there is an efficient adversary  $\mathcal{A}_{\text{unpred-eval}}$  that wins  $\text{Game}_{\Pi, \mathcal{A}}^{\text{unpred-eval}}$  with more than negligible probability. We can construct a reduction  $\mathcal{A}_{\text{unpred}}$  that wins  $\text{Game}_{\Pi, \mathcal{A}}^{\text{unpred}}$  with the same probability by answering both  $\mathcal{O}_{\text{BlindEval}}$  and  $\mathcal{O}_{\text{Eval}}$  queries using the blind evaluation oracle from the **unpred** game. From correctness follows that the values obtained via blind and non-blind evaluation are identical. Once  $\mathcal{A}_{\text{unpred-eval}}$  terminates,  $\mathcal{A}_{\text{unpred}}$  forwards their output. Since the adversaries make the same total number of calls to oracles, their winning probability is identical.  $\square$

## E.2 Returning the randomness from the honest blind evaluation oracle in weak pseudorandomness

Recall that weak pseudorandomness includes a **BlindEval** oracle that generates a full blind evaluation transcript for a given input. When developing weak pseudorandomness, we stated that we must return the randomness used in the **Request** algorithm to provide the attacker with the same view as if they participated in the protocol run themselves.

Let us now justify this design decision by illustrating how a malicious requester is weakened by being withheld the state  $st$ . First, we state in Fig. 10 a **BlindEval** oracle that skips outputting the randomness.

A first observation is that knowing  $st$  allows an adversary to check whether  $\text{Finalize}(\bar{y}, st)$  is equal to  $y$ , which makes them intuitively stronger when it comes to distinguishing real blind evaluations in the pseudorandomness context.

| $\mathcal{O}_{\text{BlindEval}}(x)$                |
|----------------------------------------------------|
| <b>if</b> $x \in \mathbf{E}$ <b>return</b> $\perp$ |
| $(\bar{x}, st) \leftarrow \text{Request}(pk, x)$   |
| $\bar{y} \leftarrow \text{BlindEval}(sk, \bar{x})$ |
| $E[x] \leftarrow \text{Finalize}(\bar{y}, st)$     |
| <b>return</b> $(\bar{x}, \bar{y}, E[x])$           |

**Fig. 10.** Alternative honest **BlindEval** oracle where randomness is not returned.

But we can even construct an OPRF that has an obvious security problem while still fulfilling weak pseudorandomness with the alternative  $\mathcal{O}_{\text{BlindEval}}$ . Consider a (highly fabricated) OPRF scheme that makes use of some encryption scheme with keys compatible to the base OPRF. The **Request** algorithm sends the state  $st$  to the evaluator by encrypting it under  $pk$  and appending it to the request. The evaluator uses  $st$  to one-time-pad their secret key and appends the result to the blind evaluation.

Obviously, any requester running **Request** themselves knows the state  $st$  and learns the secret key from a **BlindEval** response, so this scheme is very insecure. However, in a weak pseudorandomness game that computes the transcript non-interactively and returns only  $\bar{x}, \bar{y}$ , and  $y$ , the state remains hidden from the adversary – we would have to call the above construction weakly pseudorandom.

## F Full proofs of property relations

### F.1 Lemma 1: One-more pseudorandomness is strictly stronger than weak pseudorandomness

It follows the full proof of Lemma 1.

*Proof.* We start with the easier part of showing a weakly pseudorandom, but not one-more pseudorandom, before showing the inverse implication.

$\text{om-prnd} \not\Leftarrow \text{weak-prnd}$  We show how an OPRF behaving “weirdly” upon malicious blind requests can be exploited in **om-prnd** but not by the honest-but-curious **weak-prnd** adversary.

Consider the following modifications to a weakly pseudorandom OPRF  $\Pi$ :

```

Request'(pk, x) : return (Request(pk, x), 0)
BlindEval'(sk,  $\bar{x}$ ) : parse ( $\bar{x}'$ , flag) =  $\bar{x}$ ;
    if flag = 0: return (BlindEval(sk,  $\bar{x}'$ ), 0|sk|);
    else : return (BlindEval(sk,  $\bar{x}'$ ), sk)
Finalize'( $\bar{y}$ , st) : parse ( $\bar{y}'$ ,  $\kappa$ ) =  $\bar{y}$ ; return Finalize( $\bar{y}'$ , st)

```

The resulting scheme  $\Pi'$  is still weakly pseudorandom, since for an honestly executed **Request'** the flag will always be 0 and  $\bar{y}$  will simply have some zeros added. However,  $\Pi'$  can hardly be considered secure against a malicious requester, who can switch **flag** to 1 and obtain the entire secret key.

$\text{om-prnd} \Rightarrow \text{weak-prnd}$  The inverse implication turns out rather tedious due to the non-standard one-more paradigm.

As a warm-up, let us limit **weak-prnd** to a single real-or-random challenge instead of the **Eval** oracle. Then we can do a straightforward reduction to one-more pseudorandomness: we provide this challenge from the **om-prnd**  $\mathcal{O}_{\text{Chal}}$  oracle. Otherwise, we answer any call  $\mathcal{A}_{\text{weak-prnd}}$  makes to  $\mathcal{O}_{\text{BlindEval}}$  by calling the **om-prnd**  $\mathcal{O}_{\text{BlindEval}}$  oracle and querying  $\mathcal{O}_{\text{Chal}}$  in parallel, recording the “world” of

this challenge. At the end, we will have recorded the right world for each of the challenges we queried during the game, and one fresh challenge solution from  $\mathcal{A}_{\text{weak-prnd}}$ .

To achieve the same reduction for the regular weak pseudorandomness game, we repeat this strategy for  $c$  distinct challenges. Let  $G_0$  be the **weak-prnd** game in the random world, where all  $q$  outputs of  $\mathcal{O}_{\text{Eval}}$  are random. We do  $q$  game hops until  $G_c$ , where all challenges are real, as in the real world of **weak-prnd**.

| Adversary $\mathcal{A}_{i,\text{om-prnd}}^{\mathcal{O}_{\text{Chal}}, \mathcal{O}_{\text{BlindEval}}}(pp, pk)$                      | $\mathcal{O}_{\text{Eval}}^i(x)$                                  | $\mathcal{O}_{\text{BlindEval}}^{\text{weak}}(x)$            |
|-------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|--------------------------------------------------------------|
| $f' \leftarrow \text{RandFnGen}()$                                                                                                  | <b>if</b> $x \notin E$ :                                          | <b>if</b> $x \in E$ : <b>return</b> $\perp$                  |
| $b', c_{\text{Chal}} := 0$                                                                                                          | $c_{\text{Chal}} := c_{\text{Chal}} + 1$                          | $\omega \leftarrow \Omega$                                   |
| $E, \text{chals} := []$                                                                                                             | // $i - 1$ real outputs                                           | $(\bar{x}, st) \leftarrow \text{Request}(pk, x; \omega)$     |
| $b^* \leftarrow \mathcal{A}_{\text{weak-prnd}}^{\mathcal{O}_{\text{Eval}}^i, \mathcal{O}_{\text{BlindEval}}^{\text{weak}}}(pp, pk)$ | <b>if</b> $c_{\text{Chal}} < i$ :                                 | $\bar{y} \leftarrow \mathcal{O}_{\text{BlindEval}}(\bar{x})$ |
| <b>return</b> $(1, \dots,  \text{chals}  + 1, b' \oplus b^*)$                                                                       | $E[x] \leftarrow \mathcal{O}_{\text{BlindEval}}^{\text{weak}}(x)$ | $E[x] \leftarrow \text{Finalize}(\bar{y}, st)$               |
|                                                                                                                                     | // the fresh challenge!                                           | $\text{chals}[x] \leftarrow \mathcal{O}_{\text{Chal}}(x)$    |
|                                                                                                                                     | <b>elseif</b> $c_{\text{Chal}} = i$ :                             | <b>if</b> $E[x] \neq \text{chals}[x]$ :                      |
|                                                                                                                                     | $E[x] \leftarrow \mathcal{O}_{\text{Chal}}(x)$                    | $b' := b' \oplus 1$                                          |
|                                                                                                                                     | // $q - i$ random outputs                                         | <b>return</b> $\bar{x}, \bar{y}, E[x], \omega$               |
|                                                                                                                                     | <b>else</b> : $E[x] \leftarrow f'(x)$                             |                                                              |
|                                                                                                                                     | <b>return</b> $E[x]$                                              |                                                              |

**Fig. 11.** Adversary  $\mathcal{A}_{i,\text{om-prnd}}$  for the proof of Lemma 1. Note that variable  $i$  is defined implicitly per  $\mathcal{A}_{i,\text{om-prnd}}$ .

Intuitively, from  $G_{i-1}$  to  $G_i$ , the  $i^{\text{th}}$  evaluation will transition from random to real, and we can bound the game hop via **om-prnd**. Concretely, we build an adversary  $\mathcal{A}_{i,\text{om-prnd}}$  (Fig. 11) that sets this  $i^{\text{th}}$   $\mathcal{O}_{\text{Eval}}$  output to a challenge from the one-more pseudorandomness game, such that  $G_{i-1}$  is simulated if the challenge is *random*, and  $G_i$  is simulated if the challenge is real. If some distinguisher can tell between the two games, we win **om-prnd** in our reduction.

The rest of  $G_{i-1}$  and  $G_i$  is simulated as follows: for  $\mathcal{O}_{\text{BlindEval}}$ , adversary  $\mathcal{A}_{i,\text{om-prnd}}$  calls their own  $\mathcal{O}_{\text{BlindEval}}$  for the blind evaluation and generates the rest of the transcript honestly; simultaneously, they retrieve a one-more challenge as in the warm-up. For  $\mathcal{O}_{\text{Eval}}$ , there are three cases: the first  $i - 1$  challenges must be real, so  $\mathcal{A}_{i,\text{om-prnd}}$  proceeds as for the **BlindEval** oracle. The  $i^{\text{th}}$  input is answered with  $\mathcal{O}_{\text{Chal}}$ . All remaining random challenges are created with a truly random function  $f'$ . There is no issue in using a different random function than  $\mathcal{O}_{\text{Chal}}$  uses: a collision between the  $i^{\text{th}}$  challenge and subsequent random outputs is as high in our simulation as it would be if all random values came from a single function. Observe also how the  $E$  dictionary maintained by  $\mathcal{A}_{i,\text{om-prnd}}$  ensures that they never double-query any input to their  $\mathcal{O}_{\text{Chal}}$ , which is forbidden in **om-prnd**.

Finally, note that our reduction might fail despite a correct simulation, if  $\mathcal{A}_{i,\text{om-prnd}}$  at some point obtains a challenge from  $\mathcal{O}_{\text{Chal}}$  that is identical to

$\text{Eval}(sk, x)$  but actually was an output from  $f$ , due a collision of the two functions. In this case the adversary computes  $b'$  incorrectly. Let us disregard the possibility of multiple such collisions. We upper-bound the probability of such an error, if a total of  $(c_{\text{BE}})_i$  calls are made to  $\mathcal{O}_{\text{BlindEval}}$  by  $\mathcal{A}_{i, \text{om-prnd}}$  and the output space of the OPRF has a size of  $n$ , by  $(c_{\text{BE}})_i/n$ .

For a one-more pseudorandom scheme we bound a single game hop as follows:

$$|\Pr[G_{i-1, \Pi, \mathcal{A}}(\lambda) = 1] - \Pr[G_{i, \Pi, \mathcal{A}}(\lambda) = 1]| - (c_{\text{BE}})_i/n \leq \text{Adv}_{\Pi, \mathcal{A}_{i, \text{om-prnd}}}^{\text{om-prnd}}(\lambda)$$

Winning *weak-prnd* equals distinguishing  $G_0$  and  $G_c$ , so we can bound the *weak-prnd* advantage by summing the  $q$  game hops, concluding the implication.

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{weak-prnd}}(\lambda) \leq c \cdot \text{Adv}_{\Pi, \mathcal{A}_{i, \text{om-prnd}}}^{\text{om-prnd}}(\lambda) + \sum (c_{\text{BE}})_i/n$$

□

## F.2 Lemma 2: Strong pseudorandomness implies one-more pseudorandomness

*Proof.* Our strategy will be to modify, using strong pseudorandomness, the *om-prnd* into an impossible-to-solve version where all challenges are random. The proof inherits the Ideal Primitive Model from the strong pseudorandomness property. This model is essentially a generalization of the Random Oracle Model, proving security under the assumption that some building block of a protocol is replaced by an idealized, e.g., truly random, version [GT15].

Let  $G_0$  be the original *om-prnd* game, but with an ideal primitive oracle. In  $G_1$  (Fig. 12), the real challenges are replaced by outputs of a second truly function  $f'$ . The *BlindEval* oracle and the ideal primitive are now operated by a simulator, which has limited access to  $f'$  outputs – just like the simulator in the strong pseudorandomness game has limited access to evaluations – so it is able to restore consistency between the simulated blind evaluations and  $f'$ .

The two games are indistinguishable by strong pseudorandomness: in a reduction  $\mathcal{A}_{\text{str}}$  we simulate either game as follows. We simply forward queries to  $\mathcal{O}_{\text{BlindEval}}$  and  $\mathcal{O}_{\text{Prim}}$  to the corresponding *str-prnd* oracles. For  $\mathcal{O}_{\text{Chal}}$ , we flip a coin and either return a value of a random function  $f$ , or call the *str-prnd* evaluation oracle. Thus, if *str-prnd* is in the random world, we perfectly simulate  $G_0$ , if it is in the real world, we perfectly simulate  $G_1$ . Thus, we bound the game hop by strong pseudorandomness of the scheme.

Finally, looking at  $G_1$ , we are left with a game where challenges come from two random distributions. While by calling  $\mathcal{O}_{\text{BlindEval}}$  an adversary may obtain some  $f'$  outputs, the  $\mathcal{O}_{\text{LimEval}}$  ticketing mechanism strictly limits the number of such  $f'$  values a simulator is able to leak. Hence, upon  $\ell - 1$  calls to  $\mathcal{O}_{\text{BlindEval}}$ , it is infeasible to know for  $\ell$  challenges whether they were produced by  $f$  or by  $f'$ . This allows us to bound the advantage of any one-more pseudorandomness adversary  $\mathcal{A}_{\text{om-prnd}}$  as follows:

$$\begin{aligned} \text{Adv}_{\Pi, \mathcal{A}_{\text{om-prnd}}}^{\text{om-prnd}}(\lambda) &= |\Pr[G_{0, \Pi, \mathcal{A}}(\lambda) = 1] - \Pr[G_{1, \Pi, \mathcal{A}, \mathcal{S}_{\text{str}}}(\lambda) = 1]| + \text{Adv}_{\Pi, \mathcal{A}, \mathcal{S}_{\text{str}}}^{G_1}(\lambda) \\ &\leq \text{Adv}_{\Pi, \mathcal{A}_{\text{str}}, \mathcal{S}_{\text{str}}}^{\text{prnd}}(\lambda) + 0 \end{aligned}$$

| $G_{1,\Pi,\mathcal{A},\mathcal{S}}(\lambda)$                                                                          | $\mathcal{O}_{\text{Chal}}(x)$          | $\mathcal{O}_{\text{LimEval}}(x)$                                                                       |
|-----------------------------------------------------------------------------------------------------------------------|-----------------------------------------|---------------------------------------------------------------------------------------------------------|
| $pp \leftarrow \mathcal{S}.\text{Setup}(\lambda)$                                                                     | <b>if</b> $x \in \mathcal{C}$ :         | $c_S := c_S + 1$                                                                                        |
| $(st_S, pk) \leftarrow \mathcal{S}.\text{Setup}(pp)$                                                                  | <b>return</b> $\perp$                   | <b>if</b> $c_S \leq c_{\text{BE}}$ : <b>return</b> $f'(x)$                                              |
| $f, f' \leftarrow \mathcal{S}.\text{RandFnGen}(\lambda)$                                                              | <b>else</b> :                           | <b>return</b> $\perp$                                                                                   |
| $c_{\text{BE}}, c_S := 0; \quad \mathbf{b} := (); \quad \mathcal{C} := \emptyset$                                     | $\mathcal{C} := \mathcal{C} \cup \{x\}$ | $\mathcal{O}_{\text{BlindEval}}(\bar{x})$                                                               |
| $(i_1, \dots, i_\ell, b') \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Chal}}, \mathcal{O}_{\text{BlindEval}}}(pp, pk)$ | $b \leftarrow \{0, 1\}$                 | $c_{\text{BE}} := c_{\text{BE}} + 1$                                                                    |
| <b>if</b> $\exists \alpha : i_\alpha \geq  \mathbf{b}  \vee \ell \leq c_{\text{BE}}$                                  | Append $b$ to $\mathbf{b}$              | $(\bar{y}, st_S) \leftarrow \mathcal{S}.\text{BlindEval}^{\mathcal{O}_{\text{LimEval}}}(\bar{x}, st_S)$ |
| $\vee \exists \alpha \neq \beta : i_\alpha = i_\beta :$                                                               | <b>if</b> $b = 0$ :                     | <b>return</b> $\bar{y}$                                                                                 |
| <b>return</b> 0                                                                                                       | <b>return</b> $f(x)$                    | $\mathcal{O}_{\text{Prim}}(x)$                                                                          |
| <b>return</b> $b' = \bigoplus_{\alpha=1}^{\ell} b[i_\alpha]$                                                          | <b>else</b> :                           | $(y, st_S) \leftarrow \mathcal{S}.\text{Prim}^{\mathcal{O}_{\text{LimEval}}}(x, st_S)$                  |
|                                                                                                                       | <b>return</b> $f'(x)$                   | <b>return</b> $y$                                                                                       |

Fig. 12. Game  $G_1$  for the relation of strong and one-more pseudorandomness.

□

### F.3 Strong pseudorandomness is stronger than one-more unpredictability and weak pseudorandomness, direct proofs

Here, we provide the lemmas and proofs for the relation between **str-prnd** and **unpred** as well as **str-prnd** and **weak-prnd**, avoiding the intermediate step of the non-standard one-more pseudorandomness notion given in Section 4.3.

**Lemma 14.** *str-prnd is strictly stronger than unpred.*

*Proof.* We start by demonstrating how an unpredictable OPRF need not be pseudorandom with a simple example that we will refer back to in the following.

**str-prnd**  $\not\Rightarrow$  **unpred** To enforce an unpredictable construction that is distinguishable from random at a glance we add some fixed string to an unpredictable evaluation:

Assume  $\Pi$  is an unpredictable OPRF scheme. We construct  $\Pi'$  which appends a zero bit to the evaluation:

$\text{Eval}'(sk, x) : \mathbf{return} \text{Eval}(sk, x) \parallel 0$   
 $\text{Finalize}'(\bar{y}, st) : \mathbf{return} \text{Finalize}(\bar{y}, st) \parallel 0$

No efficient adversary is able to predict fresh evaluations of  $\Pi'$ , since it contains evaluations of the unpredictable  $\Pi$ . However,  $\Pi'$  is not strongly pseudorandom (or any type of pseudorandom) since the last bit is biased.

**str-prnd**  $\Rightarrow$  **unpred** If a scheme is strongly pseudorandom, we can replace evaluations in the unpredictability game by random values without the adversary noticing – but then predicting fresh values is evidently hard.

We translate this intuition into a proof, consisting of a single game hop. Let  $G_0$  be the original **unpred** game in the Ideal Primitive Model, i.e., with

| $G_{1,\Pi,\mathcal{A},\mathcal{S}}(\lambda)$                                                                         | $\mathcal{O}_{\text{BlindEval}}(\bar{x})$                                                                          | $\mathcal{O}_{\text{LimEval}}(x)$             |
|----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| $pp \leftarrow \text{\$ Setup}(\lambda)$                                                                             | $c_{\text{BE}} := c_{\text{BE}} + 1$                                                                               | $c_{\text{S}} := c_{\text{S}} + 1$            |
| $(st_{\text{S}}, pk) \leftarrow \text{\$ S.Setup}(pp)$                                                               | $(\bar{y}, st_{\text{S}}) \leftarrow \text{\$ S.BlindEval}^{\mathcal{O}_{\text{LimEval}}}(\bar{x}, st_{\text{S}})$ | <b>if</b> $c_{\text{S}} \leq c_{\text{BE}}$ : |
| $c_{\text{BE}}, c_{\text{S}} := 0$                                                                                   | <b>return</b> $\bar{y}$                                                                                            | <b>return</b> $f(x)$                          |
| $f \leftarrow \text{\$ RandFnGen}(\lambda)$                                                                          |                                                                                                                    | <b>return</b> $\perp$                         |
| $(x_i, y_i)_{i=1}^{\ell} \leftarrow \mathcal{A}^{\mathcal{O}_{\text{BlindEval}}, \mathcal{O}_{\text{Prim}}}(pp, pk)$ | $\mathcal{O}_{\text{Prim}}(x)$                                                                                     |                                               |
| <b>if</b> $\ell \leq c_{\text{BE}} \vee \exists \alpha \neq \beta : x_{\alpha} = x_{\beta}$ :                        | $(y, st_{\text{S}}) \leftarrow \text{\$ S.Prim}^{\mathcal{O}_{\text{LimEval}}}(x, st_{\text{S}})$                  |                                               |
| <b>return</b> 0                                                                                                      | <b>return</b> $y$                                                                                                  |                                               |
| <b>return</b> $\forall \alpha : f(x_{\alpha}) \stackrel{?}{=} y_{\alpha}$                                            |                                                                                                                    |                                               |

**Fig. 13.** Game  $G_1$  for the relation of **str-prnd** and **unpred**.

an additional  $\mathcal{O}_{\text{Prim}}$ . We transition to a game  $G_1$  where all **BlindEval** and ideal primitive queries are answered by a simulator  $\mathcal{S} = (\text{\$ Setup}, \text{\$ S.BlindEval}, \text{\$ S.Prim})$ . The simulator has access to a  $\mathcal{O}_{\text{LimEval}}$  oracle that allows it to obtain the output of a random function  $f$  for each  $\mathcal{O}_{\text{BlindEval}}$  query the user makes. The resulting game is shown in Fig. 13.

For a strongly pseudorandom OPRF and instantiating  $G_1$  with the corresponding **str-prnd** simulator  $\mathcal{S}_{\text{prnd}}$ ,  $G_0$  and  $G_1$  are indistinguishable – if any distinguisher could tell apart the  $\mathcal{S}_{\text{prnd}}$ -generated public key, ideal primitive outputs or blind evaluations from the real ones, then they would also win strong pseudorandomness.

Finally, we argue that  $G_1$  is hard to win: here, the “evaluations” are truly random outputs of  $f$ . An adversary can only have learned  $\ell - 1$  such values from the **BlindEval** oracle, since the simulator operating the oracle is only allowed to fetch one  $f$  output per oracle call. The probability of guessing an additional random pair  $(x, y)$  is evidently negligible with regards to the output domain size of the OPRF. Therefore, we may bound the advantage of any  $\mathcal{A}_{\text{unpred}}$  in the **unpred** game by a reduction  $\mathcal{A}_{\text{prnd}}$  to **str-prnd** and the negligible advantage of any adversary  $\mathcal{A}'$  in  $G_1$ :

$$\begin{aligned} \text{Adv}_{\Pi, \mathcal{A}_{\text{unpred}}}^{\text{unpred}}(\lambda) &= |\Pr[G_{0,\Pi,\mathcal{A}}(\lambda) = 1] - \Pr[G_{1,\Pi,\mathcal{A},\mathcal{S}_{\text{prnd}}}(\lambda) = 1]| + \text{Adv}_{\Pi, \mathcal{A}', \mathcal{S}_{\text{str}}}^{G_1}(\lambda) \\ &\leq \text{Adv}_{\Pi, \mathcal{A}_{\text{str}}, \mathcal{S}_{\text{str}}}^{\text{prnd}}(\lambda) + \text{negl}(\lambda) \end{aligned}$$

□

**Lemma 15.** *For correct schemes, **str-prnd** is strictly stronger than **weak-prnd**.*

*Proof.* We skip showing how weak pseudorandomness does not imply strong pseudorandomness and refer to the proof of Lemma 1 for an example scheme that becomes insecure once the adversary is actively malicious.

To show that **str-prnd** implies **weak-prnd**, we will intuitively make use of the strong notion’s simulator to replace the real evaluations in **weak-prnd** by truly random ones – after that, the game is impossible to win, since it asks an adversary to distinguish two uniformly random distributions, and the adversary can never query the same input from both distributions.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $G_{0,\Pi,\mathcal{A}} = \text{Game}_{\Pi,\mathcal{A}}^{\text{weak-prnd}}, G_{1,\Pi,\mathcal{A}}(\lambda)$<br>$pp \leftarrow \text{Setup}(\lambda)$<br>$st_{\text{Prim}} \leftarrow \text{Prim.Setup}(pp)$<br>$(pk, sk) \leftarrow \text{KGen}(pp)$<br>$f \leftarrow \text{RandFnGen}(pp)$<br>$E := []; \quad b \leftarrow \{0, 1\}$<br>$b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Eval}}, \mathcal{O}_{\text{BlindEval}}, \mathcal{O}_{\text{Prim}}}(pp, pk)$<br><b>return</b> $b' \stackrel{?}{=} b$                                                                                                    | $\mathcal{O}_{\text{Eval}}(x)$<br><b>if</b> $x \notin E$ :<br><b>if</b> $b = 0$ :<br>$E[x] \leftarrow f(x)$<br><b>else</b> :<br>$E[x] \leftarrow \text{Eval}(sk, x)$<br><b>return</b> $E[x]$<br><hr/> $\mathcal{O}_{\text{Prim}}(x)$<br><b>return</b> $\text{Prim.Eval}(x, st_{\text{Prim}})$                                                                   | $\mathcal{O}_{\text{BlindEval}}(x)$<br><b>if</b> $x \in E$ : <b>return</b> $\perp$<br>$\omega \leftarrow \Omega$<br>$(\bar{x}, st) \leftarrow \text{Request}(pk, x; \omega)$<br>$\bar{y} \leftarrow \text{BlindEval}(sk, \bar{x})$<br>$E[x] \leftarrow \text{Finalize}(\bar{y}, st)$<br><b>if</b> $E[x] \neq \text{Eval}(sk, x)$ : <b>abort</b><br><b>return</b> $(\bar{x}, \bar{y}, E[x], \omega)$                                                                                                                                      |
| $G_{2,\Pi,\mathcal{A},S}, G_{3,\Pi,\mathcal{A},S}(\lambda)$<br>$pp \leftarrow \text{Setup}(\lambda)$<br>$(st_S, pk) \leftarrow \text{S.Setup}(pp)$<br>$f, f' \leftarrow \text{RandFnGen}(pp)$<br>$c_{\text{BE}}, c_S := 0$<br>$E := []; \quad b \leftarrow \{0, 1\}; \quad X := \emptyset$<br>$b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Eval}}, \mathcal{O}_{\text{BlindEval}}, \mathcal{O}_{\text{Prim}}}(pp, pk)$<br><b>return</b> $b' \stackrel{?}{=} b$<br><hr/> $\mathcal{O}_{\text{Prim}}(x)$<br>$(y, st_S) \leftarrow \text{S.Prim}^{\mathcal{O}_{\text{LimEval}}}(x, st_S)$<br><b>return</b> $y$ | $\mathcal{O}_{\text{Eval}}(x)$<br><b>if</b> $x \notin E$ :<br><b>if</b> $b = 0$ :<br>$E[x] \leftarrow f(x)$<br><b>else</b> :<br>$E[x] \leftarrow f'(x)$<br><b>return</b> $E[x]$<br><hr/> $\mathcal{O}_{\text{LimEval}}(x)$<br>$c_S := c_S + 1$<br><b>if</b> $c_S \leq c_{\text{BE}}$ :<br>$X := X \cup \{x\}$<br><b>return</b> $f'(x)$<br><b>return</b> $\perp$ | $\mathcal{O}_{\text{BlindEval}}(x)$<br><b>if</b> $x \in E$ : <b>return</b> $\perp$<br>$c_{\text{BE}} := c_{\text{BE}} + 1$<br>$\omega \leftarrow \Omega$<br>$(\bar{x}, st) \leftarrow \text{Request}(pk, x; \omega)$<br>$(\bar{y}, st_S) \leftarrow \text{S.BlindEval}^{\mathcal{O}_{\text{LimEval}}}(\bar{x}, st_S)$<br>$E[x] \leftarrow \text{Finalize}(\bar{y}, st)$<br><b>if</b> $E[x] \neq f'(x)$ : <b>abort</b><br><b>if</b> $x \notin X$ :<br><b>bad</b> := 1<br><b>abort</b><br><b>return</b> $(\bar{x}, \bar{y}, E[x], \omega)$ |
| $\mathcal{A}_{\text{str}}^{\mathcal{O}_{\text{Eval}}, \mathcal{O}_{\text{BlindEval}}, \mathcal{O}_{\text{Prim}}}(pp, pk)$<br>$\hat{f} \leftarrow \text{RandFnGen}(pp)$<br>$b_{\text{str}} \leftarrow \{0, 1\}$<br>$E := []$<br>$\text{Start } \mathcal{A}^{\mathcal{O}_{\text{Eval}}, \mathcal{O}_{\text{BlindEval}}, \mathcal{O}_{\text{Prim}}}^{\text{weak}}(pp, pk)$<br><hr/> $\mathcal{O}_{\text{Prim}}^{\text{weak}}(x)$<br><b>return</b> $\mathcal{O}_{\text{Prim}}(x)$                                                                                                                                 | $\mathcal{O}_{\text{Eval}}^{\text{weak}}(x)$<br><b>if</b> $x \notin E$ :<br><b>if</b> $b_{\text{str}} = 0$ :<br>$E[x] \leftarrow \hat{f}(x)$<br><b>else</b> :<br>$E[x] \leftarrow \mathcal{O}_{\text{Eval}}(x)$<br><b>return</b> $E[x]$                                                                                                                         | $\mathcal{O}_{\text{BlindEval}}^{\text{weak}}(x)$<br><b>if</b> $x \in E$ : <b>return</b> $\perp$<br>$c_{\text{BE}} := c_{\text{BE}} + 1$<br>$\omega \leftarrow \Omega$<br>$(\bar{x}, st) \leftarrow \text{Request}(pk, x; \omega)$<br>$\bar{y} \leftarrow \mathcal{O}_{\text{BlindEval}}(\bar{x})$<br>$E[x] \leftarrow \text{Finalize}(\bar{y}, st)$<br><b>if</b> $E[x] \neq \mathcal{O}_{\text{Eval}}(x)$ : <b>abort</b><br><b>return</b> $(\bar{x}, \bar{y}, E[x], \omega)$                                                            |

**Fig. 14. Top and center:** Game transitions for the proof of Lemma 15, where  $G_2$  and  $G_3$  use a simulator  $S = (\text{S.Setup}, \text{S.BlindEval}, \text{S.Prim})$ . **Bottom:** Reduction  $\mathcal{A}_{\text{str}}$  to strong pseudorandomness.

Assume  $\Pi$  is strongly pseudorandom with respect to simulator  $S_{\text{str}}$ . We start from  $G_0$  which is weak pseudorandomness, but in the Ideal Primitive Model, so there is an added  $\mathcal{O}_{\text{Prim}}$  which simply returns  $\text{Prim.Eval}(x, st_{\text{Prim}})$  upon an input  $x$ . All games are shown in Fig. 14.

$G_1$ . First, we add a check to  $\mathcal{O}_{\text{BlindEval}}$ : if the output evaluation of  $x$  is not the correct evaluation, we abort the game.

$G_0 \rightarrow G_1$ . In the check,  $E[x]$  is always the end result of an honest blind evaluation – the only other place where  $E[x]$  could have been defined is  $\mathcal{O}_{\text{Eval}}$ , but we reject such ‘challenge’ inputs at the beginning of  $\mathcal{O}_{\text{BlindEval}}$ . Therefore, by correctness, the finalized value  $E[x]$  will always be the evaluation, so there is no difference between the two games.

$G_2$ . Next, we do a bigger change to the game: we replace all occurrences of ‘real’ OPRF evaluations by a second random function  $f'$ . In  $\mathcal{O}_{\text{Eval}}$ , this is straightforward, in  $\mathcal{O}_{\text{BlindEval}}$ , we employ a simulator to generate a suitable blind response, who is able to operate the ideal primitive and has limited access to the function  $f'$  via a  $\mathcal{O}_{\text{LimEval}}$  oracle. Additionally, the check we added in  $G_1$  now checks that  $E[x]$  matches  $f'(x)$ .

$G_1 \rightarrow G_2$ . For this transition we make use of strong pseudorandomness. We define an adversary  $\mathcal{A}_{\text{str}}$  that plays the strong pseudorandomness game with simulator  $S_{\text{str}}$  that simulates either  $G_1$  or  $G_2$ , and if the two are distinguishable, then  $\mathcal{A}_{\text{str}}$  would also win the **str-prnd** game. Roughly,  $\mathcal{A}_{\text{str}}$  uses their own oracles to simulate the ‘real’ evaluations of both games, and selects yet another random function  $\hat{f}$  for the ‘ideal’ world of both games. The reduction is also shown in Fig. 14. In detail,  $\mathcal{A}_{\text{str}}$  selects their own random function  $\hat{f}$  and their own bit  $b_{\text{str}}$  to simulate the world-dependent behavior of  $\mathcal{O}_{\text{Eval}}$ . Then, in  $\mathcal{O}_{\text{Eval}}^{\text{weak}}$ , they return  $\hat{f}(x)$  if  $b_{\text{str}} = 0$  and forward the request to  $\mathcal{O}_{\text{Eval}}$  otherwise. In  $\mathcal{O}_{\text{Prim}}^{\text{weak}}$ ,  $\mathcal{A}_{\text{str}}$  simply forwards all requests to their own oracle, and in  $\mathcal{O}_{\text{BlindEval}}^{\text{weak}}$ , it follows all the steps of the original oracle, but calls the strong  $\mathcal{O}_{\text{BlindEval}}$  for the blind evaluation and the strong  $\mathcal{O}_{\text{Eval}}$  for the final check of correctness of  $E[x]$ . Observe how we thus perfectly simulate either  $G_1$  or  $G_2$ . If the strong oracles return real-world values, then we achieve  $G_1$ , since all the honest algorithms are run as expected. If the strong oracles return ideal-world values, then we achieve  $G_2$ , which might be a bit harder to see: the simulated  $\mathcal{O}_{\text{Eval}}^{\text{weak}}$ , depending on  $b_{\text{str}}$ , returns either outputs of a random  $\hat{f}$  or of the strong game’s  $f'$ , both random functions as expected. The  $\mathcal{O}_{\text{BlindEval}}^{\text{weak}}$  oracle receives responses  $\bar{y}$  from a simulator, which internally has access to the strong game’s  $\mathcal{O}_{\text{LimEval}}$  (that returns outputs of  $f'$ , just as in  $G_2$ ) and also programs  $\mathcal{O}_{\text{Prim}}$ . We conclude that the difference between  $G_1$  and  $G_2$  can be bounded by the advantage of  $\mathcal{A}_{\text{str}}$ :

$$|\Pr[G_{1,\Pi,\mathcal{A}}(\lambda)] - \Pr[G_{2,\Pi,\mathcal{A},S_{\text{str}}}(\lambda)]| \leq \text{Adv}_{\Pi,\mathcal{A}_{\text{str}},S_{\text{str}}}^{\text{str-prnd}}(\lambda)$$



$G_3$ . Lastly, we want to ensure that the simulator leaks nothing beyond the evaluations of  $x$  that were sent to  $\mathcal{O}_{\text{BlindEval}}$ , such that we can argue that the adversary never sees an evaluation on the same input from both  $f$  and  $f'$ . We record what inputs are queried to  $\mathcal{O}_{\text{LimEval}}$  into an additional set  $X$  and, at the end of  $\mathcal{O}_{\text{BlindEval}}$ , check whether  $x$  is included in  $X$ . If not, we abort the game.

$G_2 \rightarrow G_3$ . Due to the prior check  $E[x] = f'(x)$ , we know that the simulator provided a correct evaluation of  $f'$ . If it happens that  $x \notin X$ , then the simulator never queried  $\mathcal{O}_{\text{LimEval}}$  on this  $x$ , but still somehow knew the  $f'(x)$ . Since  $f'$  is uniformly random, the probability of guessing  $f'(x)$  while  $x \notin X$  is only  $1/n$  for an output space of size  $n$ .  $G_2$  and  $G_3$  behave identically unless the simulator succeeds in such a guess. Using the Difference Lemma:

$$|\Pr[G_{2,\Pi,\mathcal{A},S}] - \Pr[G_{3,\Pi,\mathcal{A},S}]| \leq \Pr[\text{bad} = 1] = c_{\text{BE}}/n$$

In the final game,  $b = 0$  and  $b = 1$  now have  $\mathcal{O}_{\text{Eval}}$  return values from two different, but both uniformly random distributions  $f$  (for  $b = 0$ ) or  $f'$  (for  $b = 1$ ). It is infeasible to distinguish the distributions themselves, so the only hope of any adversary can be to obtain via  $\mathcal{O}_{\text{BlindEval}}$  the 'true' function evaluation  $f'(x)$  for some  $x$  and compare with the corresponding challenge from  $\mathcal{O}_{\text{Eval}}$ .

But the game makes sure adversaries can never obtain  $f'$  from  $\mathcal{O}_{\text{BlindEval}}$  for any challenge input, in three steps: (1) no input can be double-queried at  $\mathcal{O}_{\text{BlindEval}}$ , and vice versa inputs that have been sent to  $\mathcal{O}_{\text{BlindEval}}$  before are not assigned a fresh challenge at  $\mathcal{O}_{\text{Eval}}$ . (2) the check  $E[x] \stackrel{?}{=} f'(x)$  ensures the output  $E[x]$  is exactly the evaluation of the given  $x$ . (3) the simulator, who ultimately must output this  $E[x]$ , is limited to one  $f'$  query per blind evaluation by the ticketing mechanism in  $\mathcal{O}_{\text{LimEval}}$ , and the  $x \in X$  check ensures the simulator used its only  $\mathcal{O}_{\text{LimEval}}$  call *exactly* for  $x$ . Since we are sure the simulator queries nothing but the intended  $x$  at  $\mathcal{O}_{\text{LimEval}}$  every time, we also know the simulator is unable to reveal additional  $f'$  evaluations.

Overall, the evaluations the adversary sees from  $f$  and  $f'$  have no overlap, and the distribution of both functions are identical, so the probability of winning  $G_3$  must be exactly  $1/2$ . This leaves us with the following advantage for the initial weak-prnd game:

$$\text{Adv}_{\Pi,\mathcal{A}}^{\text{weak-prnd}}(\lambda) \leq \text{Adv}_{\Pi,\mathcal{A}_{\text{str}},\mathcal{S}_{\text{str}}}^{\text{str-prnd}}(\lambda) + c_{\text{BE}}/n$$

□

**F.4 Lemma 5: If there exists a strongly pseudorandom OPRF, then strong unpredictability and one-more pseudorandomness are orthogonal properties.**

*Proof.* Let us first introduce the OPRF construction  $\Pi'$ , roughly described in Section 4.3, that achieves **om-prnd**, but not **str-unpred**, by building on some strongly pseudorandom OPRF  $\Pi$ .

**KGen'**(): choose  $s^* \leftarrow \{0,1\}^{9\lambda}$  and generate key pair  $(sk', pk') \leftarrow \text{KGen}()$ ;  
**return**  $((sk', s^*), pk')$   
**Eval'** $(sk, x)$ : parse  $(sk', s^*) = sk$ ; **return**  $\text{Eval}(sk', x)$   
**Request'** $(pk, x)$ :  $(st, \bar{x}) \leftarrow \text{Request}(pk, x)$ ; **return**  $(st, (\bar{x}, 0))$   
**BlindEval'** $(sk, \bar{x})$ : parse  $(\bar{x}', i) = \bar{x}$ ;  $(sk', s^*) = sk$ ; set  $\bar{y} \leftarrow \text{BlindEval}(sk', \bar{x}')$ ;  
 if  $i = 0$ : **return**  $\bar{y}$ ;  
 else if  $i = s^*$ : **return**  $(\bar{y}, \text{Eval}(sk', \text{'extra'}))$ ;  
 else if  $i \in [9\lambda]$ : **return**  $\text{Eval}(sk', s_i^* || i)$  for  $s_i^*$  being the  $i^{th}$  bit of  $s^*$ ;  
 else: **return**  $\perp$

The remaining algorithms of  $\Pi'$  are identical to the those of  $\Pi$ .  $\Pi'$  extends the secret key of  $\Pi$  with a random string  $s^*$  which will be used to determine the behavior of the blind evaluation algorithm **BlindEval'**. Additionally, the blinded request is extended with an integer  $i$  which changes the behavior of **BlindEval'**. In particular,  $i$  triggers the correct blind evaluation of the OPRF. If  $i = s^*$ , then the blind evaluation leaks an extra evaluation of the OPRF for the request 'extra' alongside the regular blind evaluation. Triggering this step is the ultimate goal of a pseudorandomness/unpredictability adversary to receive the extra OPRF evaluation. **BlindEval'** is designed to leak information about  $s^*$  bit by bit through queries with  $i \in [9\lambda]$ . However, this leakage is only helpful to obtain a *one-more* evaluation in the presence of a canonical verification oracle for the OPRF. As a result,  $\Pi'$  is secure under one-more pseudorandomness but not under strong unpredictability.

Note also how the construction requires the existence of a strongly pseudorandom OPRF, but is not strongly pseudorandom itself, as the additionally leaked evaluation cannot be simulated by a simulator with limited access to evaluations.

*$\Pi'$  is not str-unpred.* An adversary can extract extra information about  $s^*$  by making a query for  $i \in [9\lambda]$ . Such a query leaks the evaluation of the OPRF on  $s_i^* || i$  where  $s_i^*$  is the  $i^{th}$  bit of  $s^*$ . Observe how a strong unpredictability adversary can distinguish whether  $s_i^* = 0$  or  $s_i^* = 1$  using their Vf oracle. As a result, a strong unpredictability adversary can learn  $s^*$  with  $9\lambda$  blind evaluation queries, each also giving a distinct and valid evaluation of the OPRF. Finally, by querying with  $s^*$ , the adversary obtains two OPRF evaluations in a single query. This gives the adversary  $9\lambda + 2$  OPRF evaluations in  $9\lambda + 1$  queries, thereby breaking the strong unpredictability property.

*$\Pi'$  is om-prnd.* A one-more pseudorandomness adversary, however, cannot exploit the extra OPRF evaluation. To learn whether  $s_i^* = 0$  or  $s_i^* = 1$  without a Vf oracle, there are intuitively two strategies: either query one of the possible inputs  $s_i^* || 0, s_i^* || 1$  at **BlindEval**, or query them at  $\mathcal{O}_{\text{Chal}}$ , and match the result with the  $\text{Eval}(sk, s_i^* || i)$  obtained earlier. Our proof will argue that the adversary either cannot make a query with  $s^*$  and obtain an extra evaluation, or if this happens, the adversary double-queries  $\mathcal{O}_{\text{BlindEval}}$  for some input, so the additional OPRF evaluation leaked through  $s^*$  no longer provides any advantage. Below,

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $G_{2,\Pi',\mathcal{A},S}(\lambda)$ <hr/> $pp \leftarrow \$ \text{Setup}(\lambda)$ $(st_S, pk) \leftarrow \$ \text{S.Setup}^{\mathcal{O}_{\text{LimEval}}}(pp)$ $f, f' \leftarrow \$ \text{RandFnGen}(\lambda)$ $c_{\text{BE}}, c_{\text{BE}}^*, c_S, c_{s^*} := 0; \quad \mathbf{b} := ()$ $(i_1, \dots, i_\ell, b') \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Chal}}, \mathcal{O}_{\text{BlindEval}}}(pp, pk)$ $\text{if } \exists \alpha : i_\alpha \geq  \mathbf{b}  \vee \ell \leq c_{\text{BE}} \vee \exists \alpha \neq \beta : i_\alpha = i_\beta :$ $\quad \text{return } 0$ $\text{return } b' \stackrel{?}{=} \bigoplus_{\alpha=1}^{\ell} b[i_\alpha]$ | $\mathcal{O}_{\text{BlindEval}}(\bar{x})$ <hr/> $c_{\text{BE}} := c_{\text{BE}} + 1$ $\text{Parse } (\bar{x}', i) = \bar{x}; \quad (sk', s^*) = sk$ $\text{if } i = 0^{8\lambda} \vee i = s^* :$ $\quad c_{\text{BE}}^* := c_{\text{BE}}^* + 1$ $(\bar{y}, st_S) \leftarrow \$ \text{S.BlindEval}(\bar{x}, st_S)$ $\text{if } i = 0^{8\lambda} : \text{return } \bar{y}$ $\text{if } i = s^* :$ $\quad \text{if } c_{s^*} > 0 : \text{abort}$ $\quad c_{s^*} := c_{s^*} + 1$ $\quad \text{return } (\bar{y}, f'(\text{'extra'}))$ |
| $\mathcal{O}_{\text{Chal}}(x)$ <hr/> $\text{if } x \in \mathcal{C} : \text{return } \perp; \text{ else } \mathcal{C} := \mathcal{C} \cup \{x\}$ $b \leftarrow \$ \{0, 1\}; \quad \text{Append } b \text{ to } \mathbf{b}$ $\text{if } b = 0 : \text{return } f(x)$ $\quad \text{return } f(x)$ $\text{else} : \text{return } f'(x)$                                                                                                                                                                                                                                                                                                                                | $\mathcal{O}_{\text{LimEval}}(x)$ <hr/> $c_S := c_S + 1$ $\text{if } c_S \leq c_{\text{BE}}^* :$ $\quad \text{return } f'(x)$ $\text{return } \perp$<br>$\mathcal{O}_{\text{Prim}}(x)$ <hr/> $(y, st_S) \leftarrow \$ \text{S.Prim}(x, st_S)$ $\text{return } y$                                                                                                                                                                                                                                                                  |

**Fig. 15.** Game  $G_2$  for the relation of strong unpredictability and one-more pseudorandomness.

we formally prove that if  $\Pi$  is strongly pseudorandom, then  $\Pi'$  is one-more pseudorandom. Let  $G_0$  be identical to the original one-more pseudorandomness game for  $\Pi'$ .

$G_1$ . This game aborts if the adversary makes more than one  $\mathcal{O}_{\text{BlindEval}}$  query containing  $s^*$ . More formally,  $G_1$  initializes an additional counter  $c_{s^*} := 0$  at the beginning and  $c_{s^*}$  is incremented for each  $\mathcal{O}_{\text{BlindEval}}$  query with  $s^*$ . Furthermore,  $\mathcal{O}_{\text{BlindEval}}$  contains an additional abort condition which occurs when  $c_{s^*} > 1$ . This is just a conceptual limitation on the adversary to simplify the further steps of the proof. Notice that querying  $\mathcal{O}_{\text{BlindEval}}$  with  $s^*$  only gives an extra evaluation in the first call, as the further calls keep returning the previously leaked evaluation of 'extra'. Given any adversary  $\mathcal{B}$  which wins making multiple queries with  $s^*$ , we can build an adversary  $\mathcal{A}$  making a single call with  $s^*$  and the remaining calls are made with  $i = 0$ . Thus,  $\Pr[G_{1,\Pi,\mathcal{A}}(\lambda)] = \Pr[G_{0,\Pi,\mathcal{B}}(\lambda)]$ .

$G_2$ . This game switches to the truly random evaluations of the OPRF  $\Pi$  by relying on its strong pseudorandomness as described in Figure 15. In particular, the PRF evaluations in  $\mathcal{O}_{\text{Chal}}$  and blind evaluations in  $\mathcal{O}_{\text{BlindEval}}$  are simulated by using truly random values and the simulator of  $\Pi$ . The simulator is allowed exactly one call to  $\mathcal{O}_{\text{LimEval}}$  per response it has to generate, via a counter  $c_{\text{BE}}^*$ .

$G_1 \rightarrow G_2$ . This change is indistinguishable under the strong pseudorandomness of  $\Pi$ . We can build a **str-prnd** adversary  $\mathcal{A}_{\text{str-prnd}}$  by using a distinguisher between the two games as follows.  $\mathcal{A}_{\text{str-prnd}}$  receives the challenge public key and passes it to the adversary  $\mathcal{A}$ .  $\mathcal{A}_{\text{str-prnd}}$  makes blind evaluation queries for simulating

$\mathcal{O}_{\text{BlindEval}}$  evaluations and evaluation queries for simulating  $f'$  evaluations. If the strong pseudorandomness game runs the real-world, then  $\mathcal{A}_{\text{str-prnd}}$  simulates  $G_1$ . Otherwise,  $\mathcal{A}_{\text{str-prnd}}$  simulates  $G_2$ . Thus, we conclude that

$$\Pr[G_{2,\Pi',\mathcal{A},\mathcal{S}_{\text{str-prnd}}}(\lambda) = 1] \leq \Pr[G_{1,\Pi',\mathcal{A}}(\lambda) = 1] + \text{Adv}_{\Pi,\mathcal{A}_{\text{str-prnd}},\mathcal{S}_{\text{str-prnd}}}^{\text{str-prnd}}(\lambda).$$

The remainder of the proof has a similar proof strategy to the proof of the implication between strong and one-more pseudorandomness. There, once the OPRF evaluations are simulated with random values, we were able to ensure that the adversary has to distinguish the output of the truly random value for some input and the randomly chosen output by  $\mathcal{O}_{\text{Chal}}$ . This was because the adversary learns fewer evaluations of the truly random function than the  $\ell$  challenges they solve, due to the winning condition. Here, ensuring that this holds is less trivial. In particular, if the adversary can make a blind evaluation query with  $s^*$ , it may use the extra OPRF evaluation revealed to overcome the limitation in this proof strategy.

Our proof will investigate the adversary's probability to get an extra evaluation in two cases. In these cases, the adversary is either unlikely to obtain an extra evaluation, or obtaining an extra evaluation will cause a situation that the extra evaluation is not enough to break one-more pseudorandomness. We first introduce some extra notation to describe our next game hops.

We define three types of blind evaluation queries where  $q_{s^*} \in \{0,1\}$  is the number of queries with the correct  $s^*$  value.  $q_{\text{bit}}$  is the number of blind evaluation queries with  $i \in [9\lambda]$ . The number of remaining blind evaluation queries are equal to  $q_{\text{rest}}$ . In total, we have  $q_{s^*} + q_{\text{bit}} + q_{\text{rest}} = c_{\text{BE}}$ . We further split  $q_{\text{bit}}$  into  $\ell_{\text{be}} + \ell_{\text{chal}}$  s.t.  $\ell_{\text{be}}$  corresponds to the number of integers  $i$  where  $\mathcal{O}_{\text{LimEval}}$  (in one of the  $q_{\text{rest}}$   $\mathcal{O}_{\text{BlindEval}}$  calls) leaked the evaluation of  $b||i$  for  $b \in \{0,1\}$ .  $\ell_{\text{chal}}$  corresponds to the number of bits of  $s^*$  which are not leaked by  $\mathcal{O}_{\text{BlindEval}}$  yet. For these remaining  $\ell_{\text{chal}}$  bits, the adversary can make  $\mathcal{O}_{\text{Chal}}$  queries which reveal the corresponding bit of  $s^*$  with the probability  $1/2$  for each bit.

$G_3$ . This game aborts if  $\ell_{\text{be}} \leq \lambda$  and  $q_{s^*} = 1$ . This means that the adversary was able to guess  $s^*$  correctly by making no more than  $\lambda$   $\ell_{\text{be}}$ -type queries. Aborting in this condition guarantees that the adversary cannot learn more than  $c_{\text{BE}}$  evaluations of  $f'$  when  $\ell_{\text{be}} \leq \lambda$ .

$G_2 \rightarrow G_3$ . We will show that in this case, the adversary will get few bits of  $s^*$ , such that they are unlikely to guess the rest and get an extra evaluation. For each of the  $\ell_{\text{be}}$ -type queries, the adversary is guaranteed to learn one bit of  $s^*$ . For the  $\ell_{\text{chal}}$ -type bits where the adversary may query  $\mathcal{O}_{\text{Chal}}$ , they learn each bit only with probability  $1/2$ : the adversary may send  $0||i$  and  $1||i$  to  $\mathcal{O}_{\text{Chal}}$ , but there is a  $1/2$  probability that the oracle returns a random value for the query that contained the correct  $s^*$  bit, and the adversary learns nothing. Since it is random which  $s^*$  bits can be learned this way, the best strategy for the adversary is intuitively to make an attempt at  $\mathcal{O}_{\text{Chal}}$  for each  $s^*$  bit, and then 'spend' their up to  $\lambda$   $\ell_{\text{be}}$  queries for  $\lambda$  more bits they have not learned yet.

To limit formally the number of bits leaked, we argue that the total number of leaked bits of  $s^*$  is no more than  $7\lambda - 6\lambda$  from  $\mathcal{O}_{\text{Chal}}$  and another  $\lambda$  from  $\ell_{be}$ . First, we consider bits leaked through  $\ell_{chal}$ -type queries. Following from the Bernoulli distribution, the probability that such queries leak  $k$  bits is  $\binom{9\lambda}{k} \cdot 2^{-9\lambda}$ . Then the probability that  $\ell_{chal}$ -type queries leaked more than  $6\lambda$  bits is equal to  $\sum_{k=6\lambda+1}^{9\lambda} \binom{9\lambda}{k} \cdot 2^{-9\lambda}$ . Using Chernoff's bound, one can show that this probability is less than  $2^{-\lambda}$ .

Thus, in total, the adversary learns at most  $7\lambda$  bits with overwhelming probability. This means that the adversary has no information on the remaining  $\lambda$  bits, and they can guess  $s^*$  only with probability  $2^{-\lambda}$  for each  $\mathcal{O}_{\text{BlindEval}}$  query. For the number of  $\mathcal{O}_{\text{BlindEval}}$  queries  $c_{\text{BE}}$ , the adversary makes a query with the correct  $s^*$  only with probability  $c_{\text{BE}}/2^\lambda$ . As a result, we conclude that

$$\Pr[G_{3,\Pi',\mathcal{A},\mathcal{S}_{\text{str-prnd}}}(\lambda) = 1] \leq \Pr[G_{2,\Pi',\mathcal{A},\mathcal{S}_{\text{str-prnd}}}(\lambda) = 1] + (c_{\text{BE}} + 1)/2^\lambda.$$

$G_4$ . This game hop adds a limitation for the case when  $\ell_{be} > \lambda$ . Particularly,  $G_4$  aborts when  $\ell_{be} > \lambda$  and there does not exist a  $\ell_{be}$ -type query such that  $\mathcal{O}_{\text{LimEval}}$  leaked  $b\|i$  and  $s_i = b$ . Intuitively, this condition considers the case that the adversary always gets distinct evaluations from the queries when  $i = 0$  and when  $i \in [9\lambda]$ .

$G_3 \rightarrow G_4$ . For  $\ell_{be}$ -type queries, the evaluation of  $s_i\|i$  is leaked. Furthermore, the evaluation of some  $b\|i$  may be leaked if the simulator queries  $\mathcal{O}_{\text{LimEval}}$ . However, as the adversary does not know  $s_i$ , the true  $s_i\|i$  and a query  $b\|i$  are distinct only with probability  $1/2$ . For  $\ell_{be} > \lambda$ , the abort condition only occurs with probability less than  $2^{-\lambda}$ , so

$$\Pr[G_{4,\Pi',\mathcal{A},\mathcal{S}_{\text{str-prnd}}}(\lambda)] \leq \Pr[G_{3,\Pi',\mathcal{A},\mathcal{S}_{\text{str-prnd}}}(\lambda)] + 1/2^\lambda.$$

Finally we bound the adversary's advantage in  $G_4$ . We argue that, in  $G_4$ , the adversary cannot learn more than  $c_{\text{BE}}$  random function evaluations. Thus, the adversary has to guess for a fresh challenge whether it is an evaluation of  $f$  or  $f'$  which can be done only with probability  $1/2$ .

Remember that  $c_{\text{BE}} = \ell_{be} + \ell_{chal} + q_{s^*} + q_{rest}$ . First we consider the case that  $\ell_{be} \leq \lambda$ . By  $G_3$ , we know that  $q_{s^*} = 0$  in this case. The other type of queries can leak at most one  $f'$  evaluation per query, so the game leaks no more than  $c_{\text{BE}}$  evaluations in this case.

Next we consider the case that  $\ell_{be} > \lambda$ . In this case, the adversary can make a single  $q_{s^*}$ -type query, obtaining 2 evaluations in return. However, by  $G_4$ , the  $\ell_{be}$ -type queries leak at most  $\ell_{be} - 1$  distinct evaluations. Thus, the adversary obtains at most  $\ell_{be} - 1 + \ell_{chal} + q_{s^*} + 1 + q_{rest} = c_{\text{BE}}$  evaluations.

As a result, the adversary must guess for a randomly chosen output whether it comes from  $f$  or  $f'$  to win in  $G_4$ , so  $\Pr[G_{4,\Pi',\mathcal{A},\mathcal{S}_{\text{str-prnd}}}(\lambda)] = 1/2$  and

$$\text{Adv}_{\Pi',\mathcal{A}}^{\text{om-prnd}}(\lambda) \leq 2 \cdot \left( \text{Adv}_{\Pi,\mathcal{A}_{\text{str-prnd}},\mathcal{S}_{\text{str-prnd}}}^{\text{str-prnd}}(\lambda) + (c_{\text{BE}} + 2)/2^\lambda \right).$$

□

| $G_1, \Pi, \mathcal{A}, \mathcal{S}$                                                                               | $G_2, \Pi, \mathcal{A}, \mathcal{S}(\lambda)$ | $\mathcal{O}_{\text{BlindEval}}(\bar{x})$                                                                          | $\mathcal{O}_{\text{LimEval}}(x)$                   |
|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| $pp \leftarrow \$ \text{Setup}(\lambda); f \leftarrow \$ \text{RandFnGen}(pp)$                                     |                                               | $c_{\text{BE}} := c_{\text{BE}} + 1$                                                                               | $c_{\text{S}} := c_{\text{S}} + 1$                  |
| $(st_{\text{S}}, pk) \leftarrow \$ \text{S.Setup}(pp)$                                                             |                                               | $(st_{\text{S}}, \bar{y}) \leftarrow \$ \text{S.BlindEval}^{\mathcal{O}_{\text{LimEval}}}(\bar{x}, st_{\text{S}})$ | <b>if</b> $c_{\text{S}} \leq c_{\text{BE}}$ :       |
| $\mathbf{E} := []; c_{\text{BE}}, c_{\text{S}} := 0$                                                               |                                               | <b>return</b> $\bar{y}$                                                                                            | $\mathbf{E}[x] := f(x)$                             |
| $(x_i, y_i)_{i=1}^{\ell} \leftarrow \mathcal{A}^{\mathcal{O}_{\text{BlindEval}}, \mathcal{O}_{\text{Vf}}}(pp, pk)$ |                                               |                                                                                                                    | <b>return</b> $f(x)$                                |
| <b>if</b> $\ell \leq c_{\text{BE}} \vee \exists \alpha \neq \beta : x_{\alpha} = x_{\beta}$ :                      |                                               | $\mathcal{O}_{\text{Prim}}(x)$                                                                                     | <b>return</b> $\perp$                               |
| <b>return</b> 0                                                                                                    |                                               | $(st_{\text{S}}, y) \leftarrow \$ \text{S.Prim}^{\mathcal{O}_{\text{LimEval}}}(x, st_{\text{S}})$                  | $\mathcal{O}_{\text{Vf}}(x, y)$                     |
| <b>return</b> $\forall \alpha : f(x_{\alpha}) \stackrel{?}{=} y_{\alpha}$                                          |                                               | <b>return</b> $y$                                                                                                  | $\mathbf{return} \ y \stackrel{?}{=} f(x)$          |
|                                                                                                                    |                                               |                                                                                                                    | <b>if</b> $x \in \mathbf{E}$ :                      |
|                                                                                                                    |                                               |                                                                                                                    | $\mathbf{return} \ \mathbf{E}[x] \stackrel{?}{=} y$ |
|                                                                                                                    |                                               |                                                                                                                    | <b>else</b> : <b>return</b> 0                       |

**Fig. 16.** Games  $G_1$  and  $G_2$  for the implication between strong pseudorandomness and strong unpredictability. Boxed code appears only in  $G_1$ , colored code only in  $G_2$ .

### F.5 Lemma 6: Strong pseudorandomness is strictly stronger than strong unpredictability

Showing the implication between pseudorandomness properties and strong one-more unpredictability, the only hurdle to overcome is the verification oracle which our pseudorandomness properties lack. Intuitively, for strong pseudorandomness this does not present an issue as the verification oracle can be simulated by querying the regular  $\mathcal{O}_{\text{Eval}}$  to test a particular input-output pair. The real-or-random paradigm is no obstacle to this strategy since for any strongly pseudorandom scheme, it can be argued the two worlds are indistinguishable, and simulating the verification oracle with truly random values is unnoticeable by the adversary. Since truly random values are clearly hard to guess, the verification oracle is unlikely to be ever be called on an input-output pair that is correct, but was not sent to the adversary by the game itself. In other words, the additional oracle is unlikely to leak new information, so we can replace it by a useless variant.

We make this intuition formal in a simple 2-hop proof.

*Proof.* Let  $G_0$  be the regular strong one-more unpredictability game but we use the ideal primitive model, providing an oracle  $\mathcal{O}_{\text{Prim}}$ .

$G_1$ . In  $G_1$ , shown in Fig. 16, we modify the verification oracle so that it no longer checks an input-output pair of the OPRF, but checks if  $y$  is the correct output of  $x$  with respect to a truly random function  $f$ . To make the  $\mathcal{O}_{\text{BlindEval}}$  responses consistent with  $f$ , we simultaneously introduce a simulator. This simulator replaces the key generation during setup, providing some public key  $pk$ , programs the random oracle, and takes care of blind evaluations. It maintains some state  $st_{\text{S}}$  for consistency, and does not have direct access to  $f$ , but limited access via an oracle  $\mathcal{O}_{\text{LimEval}}$ , which works as in the strong pseudorandomness game: for each **BlindEval** call the adversary makes, the simulator is granted one  $\mathcal{O}_{\text{LimEval}}$  call that returns a value from  $f$ .

$G_0 \rightarrow G_1$ . We proceed to show that if some OPRF  $\Pi$  is strongly pseudorandom, then  $G_0$  and  $G_1$  are indistinguishable. Let  $\mathcal{A}_{\text{str-prnd}}$  be an adversary in the strong pseudorandomness game. They use their oracles to simulate either  $G_0$  or  $G_1$  as follows: Once they receive  $pk_b$  and the parameters  $pp$ , they forward them to the adversary  $\mathcal{A}_{\text{str-unpred}}$  of the strong unpredictability game. Every call by  $\mathcal{A}_{\text{str-unpred}}$  to  $\mathcal{O}_{\text{BlindEval}}$  or  $\mathcal{O}_{\text{Prim}}$ , they forward to their own oracles. For any query  $(x, y)$  to  $\mathcal{O}_{\text{Vf}}$ , they call return  $y \stackrel{?}{=} \mathcal{O}_{\text{Eval}}(x)$ . Thus, if the strong pseudorandomness game is in the real world ( $b = 1$ ),  $G_0$  is perfectly simulated, and with the random world ( $b = 0$ ),  $G_1$  is perfectly simulated. If any distinguisher is able to distinguish between the two games, this would therefore allow  $\mathcal{A}_{\text{str-prnd}}$  to win:

$$\left| \Pr[\text{Game}_{\Pi, \mathcal{A}}^{\text{str-unpred}}(\lambda) = 1] - \Pr[G_{1, \Pi, \mathcal{A}, S_{\text{str-prnd}}}(\lambda) = 1] \right| \leq \text{Adv}_{\Pi, \mathcal{A}_{\text{str-prnd}}, S_{\text{str-prnd}}}^{\text{str-prnd}}(\lambda)$$

$G_2$ .  $G_1$  is evidently already hard to win, since the adversary needs to guess the output of a truly random function. We still need to argue that the verification does not help for such a guess. In  $G_2$ , we therefore modify  $\mathcal{O}_{\text{Vf}}$  such that it does not leak new information that the adversary does not already know: instead of checking  $y \stackrel{?}{=} f(x)$ , we output 1 only if a matching pair was returned by  $\mathcal{O}_{\text{LimEval}}$  before and 0 otherwise.

$G_1 \rightarrow G_2$ . In  $G_2$ , we answer  $\mathcal{O}_{\text{Vf}}$  correctly for those  $x$  already queried to  $\mathcal{O}_{\text{LimEval}}$ . For all other  $(x, y)$  we always return 0, which might be incorrect – but then the adversary sent a valid OPRF evaluation that they certainly did not obtain from the simulator. Guessing a correctly the output of a function  $f$  with an output space of size  $n$  has a probability of  $1/n$ , so with  $c_{\text{Vf}}$  calls to  $\mathcal{O}_{\text{Vf}}$  the probability that we answer incorrectly is at most  $c_{\text{Vf}}/n$ .

Now, in  $G_2$ ,  $\mathcal{O}_{\text{Vf}}$  clearly does not leak evaluations beyond what has been output by  $\mathcal{O}_{\text{LimEval}}$ . As the adversary needs to output  $\ell$  pairs  $(x, y)$  with at most  $\ell - 1$  pairs revealed by  $\mathcal{O}_{\text{LimEval}}$ , for at least one of these pairs the adversary received no information at all, so their probability of providing  $\ell$  correct pairs is at most  $1/n$ . We bound the advantage of any adversary to win strong one-more unpredictability as follows:

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{str-unpred}}(\lambda) \leq \text{Adv}_{\Pi, \mathcal{A}_{\text{str-prnd}}, S_{\text{str-prnd}}}^{\text{str-prnd}}(\lambda) + (c_{\text{Vf}} + 1)/n$$

If  $\Pi$  is strongly pseudorandom, then this advantage is negligible and the scheme must also be strongly one-more unpredictable.

Finally, we need to argue that the inverse relation does not hold, for which any scheme that is unpredictable, but has non-random-looking outputs suffices, as shown before, e.g., for Lemma 14.  $\square$

## F.6 Lemma 7: Request obliviousness implies honest-but-curious obliviousness from related works

Here we show that our obliviousness definition implies all similar game-based notions that have appeared in the literature so far, despite our observation (see

Section 5.1) that the models by [TCR<sup>+</sup>22] and [BRRA24] might seem stronger at first.

|                                                                                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{Game}_{H, \mathcal{A}}^{\text{obliv-leh}}(\lambda)$ <hr/> $pp \leftarrow \text{Setup}(\lambda)$<br>$(pk, sk) \leftarrow \text{KGen}(pp)$<br>$b \leftarrow \{0, 1\}$<br>$(x_0, x_1, st_{\mathcal{A}}) \leftarrow \mathcal{A}(pk)$<br>$\bar{x}_b \leftarrow \text{Request}(pk, x_b)$<br>$b' \leftarrow \mathcal{A}(\bar{x}_b, st_{\mathcal{A}})$<br><b>return</b> $b \stackrel{?}{=} b'$ | $\text{Game}_{H, \mathcal{A}}^{\text{obliv-tcr}}(\lambda)$ <hr/> $pp \leftarrow \text{Setup}(\lambda)$<br>$(pk, sk) \leftarrow \text{KGen}(pp)$<br>$b \leftarrow \{0, 1\}$<br>$b' \leftarrow \mathcal{A}^{\text{OTrans}}(pp, pk, sk)$<br><b>return</b> $b \stackrel{?}{=} b'$ <hr/> $\mathcal{OTrans}(x_0, x_1)$ <hr/> <b>foreach</b> $d \in \{0, 1\}$ :<br>$(\bar{x}_d, st_d) \leftarrow \text{Request}(pk, x_d)$<br>$\bar{y}_d \leftarrow \text{BlindEval}(sk, \bar{x}_d)$<br>$y_d \leftarrow \text{Finalize}(\bar{y}_d, st_d)$<br>$\tau := (\bar{x}_b, \bar{y}_b, y_0)$<br>$\tau' := (\bar{x}_{1-b}, \bar{y}_{1-b}, y_1)$<br><b>return</b> $(\tau, \tau')$                           | $\text{Game}_{H, \mathcal{A}}^{\text{obliv-brra}}(\lambda)$ <hr/> $pp \leftarrow \text{Setup}(\lambda)$<br>$b' \leftarrow \mathcal{A}^{\text{ORun}}(pp)$<br><b>return</b> $b \stackrel{?}{=} b'$ <hr/> $\mathcal{ORun}(x_0, x_1, (pk, sk))$ <hr/> $\omega \leftarrow \Omega$<br>$(\bar{x}_b, st_b) \leftarrow \text{Request}(pk, x_b)$<br>$\bar{y}_b \leftarrow \text{BlindEval}(sk, \bar{x}_b; \omega)$<br><b>return</b> $((\bar{x}_b, \bar{y}_b), \omega)$ |
| $\mathcal{A}_{\text{obliv-req}}^{\text{Request}}(pp) \quad // \text{ (Leh)}$ <hr/> $(pk, sk) \leftarrow \text{KGen}(pp)$<br>$(x_0, x_1, st_{\mathcal{A}}) \leftarrow \mathcal{A}_{\text{leh}}(pk)$<br>$(\bar{x}_b, \bar{x}_{1-b}) \leftarrow \mathcal{ORequest}(pk, x_0, x_1)$<br>$b' \leftarrow \mathcal{A}_{\text{leh}}(\bar{x}_b, st_{\mathcal{A}})$<br><b>return</b> $b'$                 | $\mathcal{A}_{\text{obliv-req}}^{\text{Request}}(pp) \quad // \text{ (TCR)}$ <hr/> $(pk, sk) \leftarrow \text{KGen}(\lambda)$<br>$b' \leftarrow \mathcal{A}_{\text{tcr}}^{\text{OTrans}}(pp, pk, sk)$<br><b>return</b> $b'$ <hr/> $\mathcal{OTrans}(x_0, x_1)$ <hr/> $(\bar{x}_b, \bar{x}_{1-b}) \leftarrow \mathcal{ORequest}(pk, x_0, x_1)$<br>$\bar{y}_b \leftarrow \text{BlindEval}(sk, \bar{x}_b)$<br>$\bar{y}_{1-b} \leftarrow \text{BlindEval}(sk, \bar{x}_{1-b})$<br>$y_0 \leftarrow \text{Eval}(sk, x_0)$<br>$y_1 \leftarrow \text{Eval}(sk, x_1)$<br>$\tau := (\bar{x}_b, \bar{y}_b, y_0)$<br>$\tau' := (\bar{x}_{1-b}, \bar{y}_{1-b}, y_1)$<br><b>return</b> $(\tau, \tau')$ | $\mathcal{A}_{\text{obliv-req}}^{\text{Request}}(pp) \quad // \text{ (BRRA)}$ <hr/> $b' \leftarrow \mathcal{A}_{\text{brra}}^{\text{ORun}}(pp)$<br><b>return</b> $b'$ <hr/> $\mathcal{ORun}(x_0, x_1, (pk, sk))$ <hr/> $\omega \leftarrow \Omega$<br>$(\bar{x}_b, \bar{x}_{1-b}) \leftarrow \mathcal{ORequest}(pk, x_0, x_1)$<br>$\bar{y}_b \leftarrow \text{BlindEval}(sk, \bar{x}_b; \omega)$<br><b>return</b> $((\bar{x}_b, \bar{y}_b), \omega)$          |

**Fig. 17. Top:** Related works honest-but-curious obliviousness definitions, simplified, syntax adapted. **Bottom:** Adversaries for proving `obliv-req` implies the related work definitions. Left: [Leh19], center: [TCR<sup>+</sup>22], right: [BRRA24], adapted to OPRFs.

More precisely, we prove the following three statements:

**Lemma 16.** *obliv-req implies [Leh19]’s obliviousness.*

**Lemma 17.** *obliv-req implies [TCR<sup>+</sup>22]’s POPRIV1(for correct schemes).*

**Lemma 18.** *obliv-req implies [BRRA24]’s input indistinguishability for OPRFs.*

We repeat the obliviousness definitions by Lehmann [Leh19], Tyagi et al. [TCR<sup>+</sup>22], and Bellare et al. [BRRA24] in a simplified form in Fig. 17. In each case, OPRFs are considered oblivious if adversaries do not learn the bit  $b$ :



**Definition 13 (Related works: obliviousness).** *We call an OPRF protocol  $\Pi$   $\text{obliv-x}$  for  $x \in \{\text{leh}, \text{tcr}, \text{brra}\}$ , if  $\forall$  PPT adversaries  $\mathcal{A}$  the following advantage is negligible:*

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{obliv-x}}(\lambda) := 2 \cdot \Pr \left[ \text{Game}_{\Pi, \mathcal{A}}^{\text{obliv-x}}(\lambda) \right] - 1$$

**Lemma 16:  $\text{obliv-req}$  implies  $\text{obliv-leh}$**

*A note about key generation in  $\text{obliv-leh}$ .* As mentioned in Section 5.1, Lehmann’s work employs a slightly different key syntax, due to their 3-party setting and the key-homomorphism feature: evaluation itself only uses a single, symmetric key, which is derived from a master secret, but there is also an additional key pair  $(bsk, bpk)$  involved in blinding. A blinding public key  $bpk$  is used by the first and second party for blinding and during the execution, respectively, while the blinding secret key  $bsk$  is used by the third, receiving party, to reveal the evaluation without having performed the blinding themselves. Coming back to obliviousness, the work envisions passing  $bpk$  to the adversary. This is somewhat hard to translate into our syntax, but we chose in Fig. 17 to give the adversary our syntax’s public key, since  $pk$ , just as  $bpk$ , is involved in the **Request** algorithm.

*Proof.* As the adversary in [Leh19]’s definition only receives one challenge request, generated exactly like the challenges in our definition, it is easy to see our model encompasses the adversarial abilities of [Leh19]. Note how [Leh19]’s adversary has a state  $st_{\mathcal{A}}$  for continuity between the two phases of the game, which is irrelevant to our single-phase game.

Let  $\Pi$  be a request-oblivious OPRF scheme and  $\mathcal{A}_{\text{leh}}$  be an adversary playing  $\text{Game}_{\Pi, \mathcal{A}}^{\text{obliv-leh}}$ . We define a reduction  $\mathcal{A}_{\text{obliv-req}}$  that uses  $\mathcal{A}_{\text{leh}}$  to win  $\text{obliv-req}$ .

We show  $\mathcal{A}_{\text{obliv-req}}$  in Fig. 17: we simply send the two inputs of  $\mathcal{A}_{\text{leh}}$  to our **Request** oracle and receive them back blinded and in random order. We return  $\bar{x}_b$  to  $\mathcal{A}_{\text{leh}}$  s.t. we can simply forward the bit  $b'$  guessed by  $\mathcal{A}_{\text{leh}}$  to  $\text{obliv-req}$ . The reduction is clearly efficient and we win if  $\mathcal{A}_{\text{leh}}$  guesses  $b'$  correctly, so:

$$\text{Adv}_{\Pi, \mathcal{A}_{\text{leh}}}^{\text{obliv-leh}}(\lambda) \leq \text{Adv}_{\Pi, \mathcal{A}_{\text{obliv-req}}}^{\text{obliv-req}}(\lambda)$$

□

Lehmann’s definition gives the adversary far less power and knowledge concerning the evaluation keys – they only learn the public key and have no influence over its distribution. We could show equivalence between  $\text{obliv-req}$  and  $\text{obliv-leh}$  only with some adaptations: we would have to strengthen  $\text{obliv-leh}$  to output the  $sk$  as well, and weaken key generation in  $\text{obliv-req}$  via a **KGen** oracle in order to be able to program specific keys for the adversary. For the interested reader, we show such a version of  $\text{obliv-req}$  in Fig. 18.

After these adaptations, we could reduce  $\text{obliv-req}$  to  $\text{obliv-leh}$ , roughly, by doing 2 game hops per challenge the  $\text{obliv-req}$  adversary makes, where we inverse the order of each challenge  $(\bar{x}_b, \overline{x_{1-b}})$ . A reduction  $\mathcal{A}_{\text{leh}}$  chooses their own

bit  $b$  and simulates all challenges up to the challenge treated in the current hybrid using this  $b$ , then all challenges afterwards with  $1 - b$ . For the challenge in question,  $\mathcal{A}_{\text{leh}}$  would program  $\mathcal{O}_{\text{KGen}}$  at the right moment to return their own key pair. Then,  $\mathcal{A}_{\text{leh}}$  sets the first  $\bar{x}$  to be their own challenge from  $\text{obliv-leh}$  and computes the second one as  $\text{Request}(pk, x_{1-b})$ . Thus, the challenge might be  $(\bar{x}_b, \bar{x}_{1-b})$  or  $(\bar{x}_{1-b}, \bar{x}_{1-b})$ . These hybrids must be indistinguishable as otherwise  $\mathcal{A}_{\text{leh}}$  wins  $\text{obliv-leh}$ . A second reduction repeats the same strategy, but now obtains the second  $\bar{x}$  from  $\text{obliv-leh}$ , such that either  $(\bar{x}_{1-b}, \bar{x}_{1-b})$  or  $(\bar{x}_{1-b}, \bar{x}_b)$  are created, completing the flip of the challenge. At the end of the hybrid series, all challenges by  $\mathcal{O}_{\text{Request}}$  are in reverse, and we were able to bound the total probability of winning  $\text{obliv-req}$  via  $\text{obliv-leh}$ .

| $\text{Game}_{\Pi, \mathcal{A}}^{\text{obliv-req}}(\lambda)$   | $\mathcal{O}_{\text{KGen}}$              | $\mathcal{O}_{\text{Request}}(pk, x_0, x_1)$                          |
|----------------------------------------------------------------|------------------------------------------|-----------------------------------------------------------------------|
| $pp \leftarrow \$ \text{Setup}(\lambda)$                       | $(pk, sk) \leftarrow \$ \text{KGen}(pp)$ | <b>if</b> $pk \notin K$ : <b>return</b> $\perp$                       |
| $K := \emptyset$                                               | $K := K \cup \{pk\}$                     | $(\bar{x}_b, st_b) \leftarrow \$ \text{Request}(pk, x_b)$             |
| $b \leftarrow \$ \{0, 1\}$                                     | <b>return</b> $(pk, sk)$                 | $(\bar{x}_{1-b}, st_{1-b}) \leftarrow \$ \text{Request}(pk, x_{1-b})$ |
| $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Request}}}(pp)$ |                                          | <b>return</b> $(\bar{x}_b, \bar{x}_{1-b})$                            |
| <b>return</b> $b \stackrel{?}{=} b'$                           |                                          |                                                                       |

**Fig. 18.** Alternative request obliviousness game with the honest-but-curious attacker model made more explicit: The adversary can only use honestly generated keys for oracle queries.

**Lemma 17:  $\text{obliv-req}$  implies  $\text{obliv-tcr}$  (for correct schemes)**

*Proof.* It might seem that [TCR<sup>+</sup>22] and [BRRA24], due to providing full transcripts instead of just blind requests, developed security definitions stronger than  $\text{obliv-req}$ . However, we will see that all additional information contained in the transcripts can be computed by  $\mathcal{A}$  themselves. Starting with  $\text{obliv-tcr}$ , we show that if no adversary with a  $\text{Request}$  oracle can link the blind values to the requester’s input or output, then it is equally hard for any adversary with a [TCR<sup>+</sup>22]’s  $\text{Transcript}$  oracle.

Let  $\Pi$  be a request-oblivious and correct OPRF scheme and  $\mathcal{A}_{\text{tcr}}$  be an adversary playing  $\text{Game}_{\Pi, \mathcal{A}}^{\text{obliv-tcr}}$ . Then we can define an adversary  $\mathcal{A}_{\text{obliv-req}}$  that uses  $\mathcal{A}_{\text{tcr}}$  (Fig. 17) to win  $\text{obliv-req}$ .

We first generate an honest key pair, send it to  $\mathcal{A}_{\text{tcr}}$  and use it subsequently to answer any query to  $\mathcal{O}_{\text{Trans}}$ . In detail, queries  $(x_0, x_1)$  to the transcript oracle are answered as follows: we obtain the requests from  $\mathcal{O}_{\text{Request}}$  and compute  $\text{BlindEval}$  ourselves with the known secret key. We cannot finalize the responses since we lack the state  $st$ , but we can run  $\text{Eval}$  with the cleartext input and the secret key, obtaining the unique correct output of an honestly executed OPRF protocol. Once  $\mathcal{A}_{\text{tcr}}$  terminates the game by outputting a bit  $b'$ , we forward  $b'$  to  $\text{obliv-req}$ . This reduction  $\mathcal{A}_{\text{obliv-req}}$  is efficient and due to correctness,  $\mathcal{O}_{\text{Trans}}$  is perfectly

simulated. Thus:

$$\text{Adv}_{\Pi, \mathcal{A}_{\text{tcr}}}^{\text{obliv-tcr}}(\lambda) \leq \text{Adv}_{\Pi, \mathcal{A}_{\text{obliv-req}}}^{\text{obliv-req}}(\lambda)$$

□

Once more, we could achieve equivalence between the two notions by managing key generation in **obliv-req** via a **KGen** oracle, as shown in Fig. 18. This time, the argument is simpler; we only need one hybrid per key the **obliv-req** adversary uses – in each hybrid we swap the  $b$  uses for all challenges belonging to one key, and bound the difference with a reduction to **obliv-tcr**.

**Lemma 18: **obliv-req** implies **obliv-brra****

*Proof.* We only sketch out the reduction, it follows closely the proof of Lemma 17, since the definitions by [BRRA24] and [TCR<sup>+</sup>22] are similar.

Assuming  $\mathcal{A}_{\text{brra}}$  is an adversary in **obliv-brra**, we construct a reduction  $\mathcal{A}_{\text{obliv-req}}$  to **obliv-req** as follows (see Fig. 17).  $\mathcal{A}_{\text{obliv-req}}$  simulates **obliv-brra**. Upon a query  $(x_0, x_1, (pk, sk))$  by  $\mathcal{A}_{\text{brra}}$  to  $\mathcal{O}_{\text{Run}}$ , the reduction forwards the inputs and public key to their **Request** oracle.  $\mathcal{A}_{\text{obliv-req}}$  samples the randomness  $\omega$  for executing **BlindEval** and blindly evaluates each request using the secret key and applying  $\omega$ . The reduction forwards  $\mathcal{A}_{\text{brra}}$ 's output  $b'$  to **obliv-req**. It is easy to see the reduction is efficient, the oracle is simulated perfectly, and  $\mathcal{A}_{\text{obliv-req}}$  succeeds exactly if  $\mathcal{A}_{\text{brra}}$  succeeds. Thus, any OPRF that is request-oblivious must also be oblivious according to **obliv-brra**. □

Using the same modification of adding a **KGen** oracle to **obliv-req** as before, we could show that **obliv-brra** implies **obliv-req**, this time with a very straightforward reduction, as the two definitions are more similar.

**F.7 Lemma 8: Multi-key obliviousness is strictly stronger than single-key obliviousness, which in turn is strictly stronger than request obliviousness.**

**obliv-k+ is strictly stronger than obliv-1k**

*Proof.* We omit a formal proof of **obliv-k+  $\Rightarrow$  obliv-1k**, since the implication is straightforward: the only difference is the additional input and stricter assertion in  $\mathcal{O}_{\text{Finalize}}$  for **obliv-1k**, in a reduction to **obliv-k+** this assertion is easily simulated.

**obliv-k+  $\not\Leftarrow$  obliv-1k.** Next, we show there is an OPRF scheme that is single-key oblivious, but not multi-key oblivious. The additional check in  $\mathcal{O}_{\text{Finalize}}$  of **obliv-1k** that ensures only correct evaluations under a single key are returned was introduced exactly to exclude multi-key attacks. If one of the blind responses  $\bar{y}', \bar{y}''$  was evaluated under a different key, an adversary in **obliv-1k** simply receives  $\perp$  as an output, while an adversary in **obliv-k+** can learn which output was evaluated under what key.

Therefore, any single-key oblivious scheme that allows for multi-key attacks, e.g., HashDH or 2HashDH, are a suitable example to show a single-key oblivious scheme is not necessarily also multi-key oblivious. □

**obliv-1k is strictly stronger than obliv-req**

*Proof.* We omit the implication  $\text{obliv-1k} \Rightarrow \text{obliv-req}$  since it is trivial to show –  $\text{obliv-1k}$  is an exact copy of  $\text{obliv-req}$ , but with an additional oracle.

$\text{obliv-1k} \not\Leftarrow \text{obliv-req}$ . We already hinted at the fact that despite its restrictions, the single-key obliviousness  $\text{Finalize}$  oracle does add abilities for the adversary. Now we show a concrete instance of an OPRF that is request-oblivious, but not single-key oblivious.

Let  $\Pi = (\text{Setup}, \text{KGen}, \text{Eval}, \text{Request}, \text{BlindEval}, \text{Finalize})$  be a request-oblivious OPRF that has an evaluation length of  $n$ . Let  $\Pi'$  have identical algorithms except the following two modifications:

**BlindEval'**( $sk, \bar{x}$ ): **return** ( $\text{BlindEval}(sk, \bar{x}), 0$ )  
**Finalize'**( $\bar{y}, st$ ): **parse** ( $\tilde{y}, \text{flag}$ ) =  $\bar{y}$ ;  $y \leftarrow \text{Finalize}(\tilde{y}, st)$ ;  
     **parse** the bits of  $y$ :  $y_1 y_2 \dots y_n = y$ ;  
     **if**  $y_1 = 1$  : **modify** the first bit, **returning**  $(y_1 \oplus \text{flag}) y_2 \dots y_n$

In  $\Pi'$ , a flag is appended to the blind evaluation. During finalization, if the first bit  $y_1$  of the output is 1, we add the flag to  $y_1$  with the exclusive or. This way, the construction is still correct, as for an honest protocol execution  $\text{flag}$  is always 0 and does not make a change.  $\Pi'$  is still weakly pseudorandom, because in the corresponding game the adversary has no access to a  $\text{Finalize}$  oracle.

However, the bug can be used to win  $\text{obliv-1k}$ : If the flag in the response is set, then a call to  $\mathcal{O}_{\text{Finalize}}$  will fail only if the corresponding evaluation starts with 1. The full attack works as follows. The adversary chooses two inputs  $x_0, x_1$  such that under key  $sk$  the first bit of  $y_0$  is 1, the first bit of  $y_1$  is 0. They call  $\mathcal{O}_{\text{Request}}$  to receive  $\bar{x}_b, \bar{x}_{1-b}$ . Then they compute  $\text{BlindEval}$  honestly to obtain  $\bar{y}_b, \bar{y}_{1-b}$ . For the first response  $\bar{y}_b$  they set the flag to 1, for the second to 0. Lastly, they send the modified responses and  $sk$  to  $\mathcal{O}_{\text{Finalize}}$ .  $\text{Finalize}'$  will flip a bit for response  $(\bar{x}_b, 1)$  only if the corresponding OPRF output starts with 1. In this case, the oracle outputs an error and the adversary learns that  $b = 0$ . If there is no error, then the evaluation behind  $\bar{x}_b$  started with 0, so  $b = 1$ . This attack is evidently efficient and always succeeds, so  $\text{obliv-req}$  cannot imply  $\text{obliv-1k}$ .  $\square$

The above example construction hints at what points of failure request-oblivious OPRFs might exhibit, even if we exclude multi-key attacks. Here, malicious blind evaluations caused incorrect finalizations, but dependent on the private output. Here, the non-acceptance of an incorrect evaluation itself leaked the input. The example could rightfully be critiqued as contrived. But with the requirement of single-key obliviousness and verifiability for multi-key obliviousness in mind, we stress that this type of weakness must be explicitly taken care of to achieve multi-key obliviousness.

**F.8 Lemma 9: Correctness and multi-key obliviousness imply verifiability**

*Proof.* The idea of the proof is that a successful  $\mathcal{A}_{\text{vf}}$  outputs a blind response  $\bar{y}$  that finalizes into some incorrect evaluation. Since  $\Pi$  is also correct, executing

the blind evaluation honestly should result into a correct evaluation, so we can send  $\mathcal{A}_{\text{vf}}$ 's response  $\bar{y}$  together an honest one  $\bar{y}_{\text{honest}}$  to the **Finalize** oracle of the multi-key obliviousness game, and link the results to the responses by checking which one is correct.

However, we encounter a technicality: in order to use  $\mathcal{A}_{\text{vf}}$ 's output for the multi-key obliviousness game, we also need to provide the requests from the obliviousness game, such that  $\mathcal{A}_{\text{vf}}$ 's verifiability-breaking response can then be sent to  $\mathcal{O}_{\text{Finalize}}$  as described above. But the corresponding **obliv-k+** oracle takes two inputs and produces two requests, and similarly  $\mathcal{O}_{\text{Finalize}}$  always expects two responses.  $\mathcal{A}_{\text{vf}}$  will only provide one response – so how do we get a response for the second request, matching the key used by  $\mathcal{A}_{\text{vf}}$ ? To overcome this problem (which we consider an artifact of our definitions), we make use the Knowledge of the Secret Key Model [Bol03]. This essentially allows us to learn an adversary's key by imposing them to prove knowledge of the secret key during registration of the public key (and we reject use of non-registered key throughout the protocol). The model can be realized by asking the adversary for proofs of knowledge from which the secret key can be extracted, but following [Bol03], we make the notationally simpler assumption that the adversary outputs the key pair during generation.

The verifiability game is modified by this model as follows: adversaries must send any keys to an additional oracle  $\mathcal{O}_{\text{RegisterKey}}$  prior to usage in the game. The oracle rejects keys that are not well-formed, and throughout the game the adversary is limited to use registered public keys, here, in the  $\mathcal{O}_{\text{Request}}$  oracle.

Let us now provide a proof by contradiction. Let  $\Pi$  be a *correct, multi-key oblivious* OPRF scheme that is *not verifiable*. i.e. there exists a PPT adversary  $\mathcal{A}_{\text{vf}}$  that wins the verifiability game with non-negligible probability. Using  $\mathcal{A}_{\text{vf}}$ , we construct an adversary  $\mathcal{A}_{\text{obliv-k+}}$  for the multi-key obliviousness game, given in Fig. 19.

Concretely, in the reduction we answer  $\mathcal{A}_{\text{vf}}$ 's  $\mathcal{O}_{\text{Request}}$  and  $\mathcal{O}_{\text{Finalize}}$  calls by forwarding the queries to our own oracles as follows: we send any  $x$  requested by  $\mathcal{A}_{\text{vf}}$  twice to our own  $\mathcal{O}_{\text{Request}}$ , receive  $(\bar{x}', \bar{x}'')$  and return  $\bar{x}'$ . For every  $\mathcal{O}_{\text{Request}}^{\text{vf}}$  query, we record the public key, cleartext input and generated blind request for later use. In  $\mathcal{O}_{\text{Finalize}}$ , we make use of KOSK to fetch the adversary's secret key and generate an honest response  $\bar{y}''$  for the request  $\bar{x}''$  that was disregarded at first, and send both responses to  $\mathcal{O}_{\text{Finalize}}$ . If we notice that one for them is not correct, we may stop the game here already: we know the result of  $\bar{y}''$  must be the correct evaluation, so only  $\mathcal{A}_{\text{vf}}$ 's response can be incorrect. This allows us to identify  $b$  and win. Otherwise, if both responses are identical or **Finalize** returned  $\perp$ , we are returning exactly the output  $\mathcal{A}_{\text{vf}}$  expects from the oracle. Thus, we either get a perfect simulation or win multi-key obliviousness already during a  $\mathcal{O}_{\text{Finalize}}$  call.

Let us in the following only consider the case that we never abort  $\mathcal{A}_{\text{vf}}$  during  $\mathcal{O}_{\text{Finalize}}$ . Upon output  $(sk, j, \bar{y})$  by  $\mathcal{A}_{\text{vf}}$ , we compute a second blind evaluation of the same request, this time honestly. We send  $\bar{y}$  and  $\bar{y}_{\text{honest}}$  to  $\mathcal{O}_{\text{Finalize}}$ , check which of the returned evaluations is incorrect, and return the corresponding bit.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <hr/> <b>Adversary <math>\mathcal{A}_{\text{obliv-k}+}^{\mathcal{O}_{\text{Request}}, \mathcal{O}_{\text{Finalize}}}(pp)</math></b><br><hr/> $c_{\text{req}} := 0; \quad K := []$<br>$(sk, j, \bar{y}) \leftarrow \mathcal{A}_{\text{vf}}^{\mathcal{O}_{\text{Request}}^{\text{vf}}, \mathcal{O}_{\text{Finalize}}^{\text{vf}}}(pp)$<br><b>if</b> $\text{public}(sk) \neq pk_j$ : <b>return</b> $b' \leftarrow_{\$} \{0, 1\}$<br>$\bar{y}_{\text{honest}} \leftarrow_{\$} \text{BlindEval}(sk, \bar{x}_j)$<br>$y_{\text{honest}} \leftarrow \text{Eval}(sk, x_j)$<br>$(y_0, y_1)/\perp \leftarrow \mathcal{O}_{\text{Finalize}}(\bar{y}, \bar{y}_{\text{honest}}, j)$<br><b>if</b> output was $\perp \vee y_0 = y_1 = y_{\text{honest}}$ :<br><b>return</b> $b' \leftarrow_{\$} \{0, 1\}$<br><b>if</b> $y_0 \neq y_{\text{honest}}$ : <b>return</b> 0<br><b>return</b> 1<br><hr/> $\mathcal{O}_{\text{RegisterKey}}(pk, sk)$<br><hr/> <b>if</b> $\text{public}(sk) \neq pk$ : <b>return</b> $\perp$<br>$K[pk] := sk$ | <hr/> $\mathcal{O}_{\text{Request}}^{\text{vf}}(pk, x)$<br><hr/> <b>if</b> $pk \notin K$ : <b>return</b> $\perp$<br>$c_{\text{req}} := c_{\text{req}} + 1$<br>$pk_{c_{\text{req}}} := pk; \quad x_{c_{\text{req}}} := x$<br>$(\bar{x}'_{c_{\text{req}}}, \bar{x}''_{c_{\text{req}}}) \leftarrow_{\$} \mathcal{O}_{\text{Request}}(pk, x, x)$<br><b>return</b> $\bar{x}'_{c_{\text{req}}}$<br><hr/> $\mathcal{O}_{\text{Finalize}}^{\text{vf}}(\bar{y}, i)$<br><hr/> $sk_i := K[pk_i]$<br>$\bar{y}'' \leftarrow_{\$} \text{BlindEval}(sk_i, \bar{x}''_{c_{\text{req}}})$<br>$(y_0, y_1)/\perp \leftarrow \mathcal{O}_{\text{Finalize}}(\bar{y}, \bar{y}'', i)$<br><b>if</b> output was $\perp$ : <b>return</b> $\perp$<br><b>elseif</b> $y_0 = y_1$ : <b>return</b> $y_0$<br>// One of the responses is incorrect!<br>Abort $\mathcal{A}_{\text{vf}}$<br>$y_{\text{honest}} \leftarrow \text{Eval}(sk_i, x_i)$<br><b>if</b> $y_0 \neq y_{\text{honest}}$ : <b>return</b> 0<br><b>return</b> 1 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Fig. 19. Adversary  $\mathcal{A}_{\text{obliv-k}+}$  for the proof of Lemma 9.

During the reduction, we catch all cases in which the  $\mathcal{A}_{\text{vf}}$  would fail to win vf: they might have used ill-formed keys, their response  $\bar{y}$  could be rejected in the Finalize algorithm, or it could result in a correct evaluation. Any of these possibilities leave us with a situation where we cannot determine  $b$ , so we default to guessing. In this case we win with probability 1/2:

$$\Pr \left[ \text{Game}_{\Pi, \mathcal{A}_{\text{obliv-k}+}^{\text{obliv-k}+}}(\lambda) \mid \neg \mathcal{A}_{\text{vf}} \text{ wins} \right] = 1/2$$

Consequently, we may assume that if we reach the end of the algorithm, then  $\bar{y}$  is indeed finalized into an incorrect evaluation, and we can determine which response belongs to which output and win obliv-k+ with probability 1.

$$\Pr \left[ \text{Game}_{\Pi, \mathcal{A}_{\text{obliv-k}+}^{\text{obliv-k}+}}(\lambda) \mid \mathcal{A}_{\text{vf}} \text{ wins} \right] = 1$$

Overall, if  $\mathcal{A}_{\text{vf}}$  wins with non-negligible probability,  $\mathcal{A}_{\text{obliv-k}+}$  has a non-negligible advantage:

$$\begin{aligned}
 \text{Adv}_{\Pi, \mathcal{A}_{\text{obliv-k}+}^{\text{obliv-k}+}}(\lambda) &= 2 \cdot (\Pr \left[ \text{Game}_{\Pi, \mathcal{A}_{\text{obliv-k}+}^{\text{obliv-k}+}}(\lambda) \mid \mathcal{A}_{\text{vf}} \text{ wins} \right] \cdot \Pr[\mathcal{A}_{\text{vf}} \text{ wins}] + \\
 &\quad \Pr \left[ \text{Game}_{\Pi, \mathcal{A}_{\text{obliv-k}+}^{\text{obliv-k}+}}(\lambda) \mid \neg \mathcal{A}_{\text{vf}} \text{ wins} \right] \cdot \Pr[\neg \mathcal{A}_{\text{vf}} \text{ wins}]) - 1 \\
 &= 2 \cdot (\Pr \left[ \text{Game}_{\Pi, \mathcal{A}_{\text{vf}}}^{\text{vf}}(\lambda) \right] + 1/2 \cdot \Pr \left[ \text{Game}_{\Pi, \mathcal{A}_{\text{vf}}}^{\text{vf}}(\lambda) = 0 \right]) - 1 \\
 &= \Pr \left[ \text{Game}_{\Pi, \mathcal{A}_{\text{vf}}}^{\text{vf}}(\lambda) \right] / 2 = \text{Adv}_{\Pi, \mathcal{A}_{\text{vf}}}^{\text{vf}}(\lambda) / 2
 \end{aligned}$$

In conclusion, we showed that if an OPRF scheme is not verifiable, then it follows that it is either not correct or not multi-key oblivious. This is a contradiction to our initial assumption, which concludes the proof.  $\square$

### F.9 Verifiability does not imply multi-key obliviousness

Verifiability alone only provides a guarantee about the blind evaluation being correct, but multi-key obliviousness requires much more than that – input and request as well as response and output must be unlinkable, respectively. It is therefore easy to provide a construction that provides no requester privacy although being implicitly verifiable.

*Proof.* We provide an OPRF construction that is implicitly verifiable, but not multi-key oblivious: Let  $\Pi$  be some verifiable scheme. In  $\Pi'$ , we intentionally make the blinding leaky:

**Request'**( $pk, x$ ):  $(\bar{x}', st) \leftarrow \$ \text{Request}(pk, x)$ ; **return**  $((\bar{x}', x), st)$   
**BlindEval'**( $sk, \bar{x}$ ): **parse**  $(\bar{x}', x) = \bar{x}$ ; **return** **BlindEval**( $sk, \bar{x}'$ )

Implicit verifiability is not affected by this change, but in  $\Pi'$  request and input are trivially linkable, breaking obliviousness entirely.

### F.10 Lemma 10: Request obliviousness and verifiability do not imply multi-key obliviousness

Multi-key obliviousness requires blindness of requests, implicit verifiability in order to prevent multi-key attacks, but we also need that the **Finalize** algorithm does not leak anything about in- or output. The former two conditions are covered by request obliviousness and implicit verifiability, but the latter is not. We demonstrate this particularly subtle attack vector in the following with a scheme that is not multi-key oblivious solely due to a bug in the **Finalize** algorithm. We employ a very similar trick as for the separation between **obliv-req** and **obliv-1k**.

*Proof.* Let  $\Pi = (\text{Setup}, \text{KGen}, \text{Eval}, \text{Request}, \text{BlindEval}, \text{Finalize})$  be an OPRF that is request-oblivious and implicitly verifiable. Let  $\Pi'$  have identical algorithms except the following two modifications:

**BlindEval'**( $sk, \bar{x}$ ): **return** **BlindEval**( $sk, \bar{x}$ ), 0)  
**Finalize'**( $\bar{y}, st$ ): **parse**  $(\tilde{y}, \text{flag}) = \bar{y}$ ;  $y \leftarrow \text{Finalize}(\tilde{y}, st)$ ;  
     Define  $y_1$  as the first bit of  $y$ ;  
     **if**  $\text{flag} = 1 \wedge y_1 = 1$  : **return**  $\perp$ ; **else return**  $y$

Blind evaluation now appends a flag that the evaluator can set. If **flag** is set to 1, then during **Finalize'** the first bit of the output  $y$  is checked, and if it is 1 as well, the algorithm returns  $\perp$ .

Let us first reason why  $\Pi'$  is correct, request-oblivious, and implicitly verifiable, before showing an attack against **obliv-k+**. Firstly, correctness is not

affected by the modifications to  $\Pi$  since with an honest flag 0,  $\text{Finalize}'$  returns the output of  $\text{Finalize}$ . Secondly,  $\text{obliv-req}$  evidently still holds since the  $\text{Request}$  algorithm is unchanged. Thirdly, the scheme achieves  $\text{vf}$ : if  $\text{flag} = 0 \vee y_1 = 0$  then  $y$  is returned by the scheme, which due to verifiability of  $\Pi$  must be the correct evaluation of the initial request under a given key. Otherwise, if  $\text{flag} = 1 \wedge y_1 = 1$ , then an error  $\perp$  is thrown. In neither case, the adversary is able to create an incorrect evaluation which would win them the verifiability game.

The strange  $\text{Finalize}'$  behavior can indeed be exploited to break  $\text{obliv-k+}$ . An adversary chooses two inputs  $x_0, x_1$  such that under key  $sk$  the first bit of  $\text{Eval}(sk, x_0)$  is 1 and the first bit of  $\text{Eval}(sk, x_1)$  is 0. They obtain from  $\mathcal{O}_{\text{Request}}$  the requests  $\overline{x_b}, \overline{x_{1-b}}$ . Then they run  $\text{BlindEval}$  with  $sk$  on the two requests which yields  $\overline{y_b} \leftarrow \text{BlindEval}(sk, \overline{x_b})$  and  $\overline{y_{1-b}} \leftarrow \text{BlindEval}(sk, \overline{x_{1-b}})$ . They set the flag for only the first response:  $\overline{y}' := (\overline{y_b}, 1)$ ;  $\overline{y}'' := (\overline{y_{1-b}}, 0)$ . Lastly, they send the responses  $\overline{y}'$  and  $\overline{y}''$  to the  $\text{Finalize}'$  oracle. If the oracle returns  $\perp$ , then the response with the activated flag had an initial output bit 1, so  $b = 0$ . If the oracle returns regular outputs on the other hand, the adversary learns  $b = 1$ .

This attack is efficient and always succeeds, which completes the proof.  $\square$

#### F.11 Lemma 11: Single-key obliviousness and verifiability imply multi-key obliviousness

Intuitively, from  $\text{obliv-1k}$  we get that an evaluator can link two blind evaluations neither from the blinded requests, nor from a failing  $\text{Finalize}$  function, and from  $\text{vf}$  we get that they cannot link them by a non-failing, but incorrect response.

*Proof.* Since in both  $\text{obliv-1k}$  and  $\text{vf}$  the adversary outputs their  $sk$  at some point, but we never see the key in  $\text{obliv-k+}$ , we once more use the Knowledge of the Secret Key Model (KOSK), introduced briefly in Appendix F.8. This adds a  $\mathcal{O}_{\text{RegisterKey}}$  oracle to the  $\text{obliv-k+}$  game and a check in  $\mathcal{O}_{\text{Request}}$  to reject non-registered public keys.

Our proof will essentially decompose  $\text{obliv-k+}$  into the guarantees provided by  $\text{vf}$  and  $\text{obliv-1k}$ . First, we make use of  $\text{vf}$  to exclude that  $\text{Finalize}$  ever returns an incorrect evaluation. This will leave us with a game identical to  $\text{obliv-1k}$ .

Let  $G_0$  be  $\text{obliv-k+}$ . In  $G_1$  (Fig. 20), we modify  $\mathcal{O}_{\text{Finalize}}$ : before returning  $y_0, y_1$ , we check whether both are identical to the correct evaluations, and if not, we set a flag **bad**.  $G_0$  and  $G_1$  are identical-until-**bad**, so by the difference lemma [Sho04], we have that

$$|\Pr[G_0, \Pi, \mathcal{A}(\lambda)] - \Pr[G_1, \Pi, \mathcal{A}(\lambda)]| \leq \Pr[\text{bad} = 1]$$

Observe that **bad** occurs exactly if  $\mathcal{A}$  sends a response that does not fail in  $\text{Finalize}$ , but unblinds into an incorrect value. If an adversary is able to construct such a response, then this would also allow them to break  $\text{vf}$ . Hence we bound the probability of the event **bad** via the verifiability of  $\Pi$ .

A bounding adversary  $\mathcal{A}_{\text{vf}}$  is shown in Fig. 20.  $\mathcal{A}_{\text{vf}}$  has to simulate  $\mathcal{O}_{\text{Request}}$  and  $\mathcal{O}_{\text{Finalize}}$  of  $G_1$ . For  $\mathcal{O}_{\text{Request}}$ , they call their own  $\text{Request}$  oracle and put the



|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\boxed{G_0, \Pi, \mathcal{A}(\lambda)} \quad (= \text{Game}_{\Pi, \mathcal{A}}^{\text{obliv-k}+}, \boxed{G_1, \Pi, \mathcal{A}(\lambda)})$ <pre> pp ← Setup(λ) creq := 0; K := [] <b>bad := 0</b> b ← {0, 1} b' ← A<sup>O<sub>Request</sub>, O<sub>Finalize</sub></sup>(pp) <b>return</b> b <math>\stackrel{?}{=}</math> b'  O<sub>RegisterKey</sub>(pk, sk) <b>if</b> public(sk) ≠ pk : <b>return</b> ⊥ K[pk] := sk  O<sub>Request</sub>(pk, x<sub>0</sub>, x<sub>1</sub>) <b>if</b> pk ∉ K : <b>return</b> ⊥ creq := creq + 1; x<sub>creq,0</sub> := x<sub>0</sub>; x<sub>creq,1</sub> := x<sub>1</sub> (ṡb, st<sub>creq,b</sub>) ← Request(pk, x<sub>b</sub>) (ṡ<sub>1-b</sub>, st<sub>creq,1-b</sub>) ← Request(pk, x<sub>1-b</sub>) <b>return</b> (ṡb, ṡ<sub>1-b</sub>) </pre> | $\mathcal{O}_{\text{Finalize}}(\bar{y}', \bar{y}'', i)$ <pre> <b>if</b> i &gt; c<sub>req</sub> : <b>return</b> ⊥ y<sub>b</sub> ← Finalize(ṡ', st<sub>i,b</sub>) y<sub>1-b</sub> ← Finalize(ṡ'', st<sub>i,1-b</sub>) <b>if</b> y<sub>b</sub> = ⊥ ∨ y<sub>1-b</sub> = ⊥ : <b>return</b> ⊥ <b>return</b> (y<sub>0</sub>, y<sub>1</sub>)  sk<sub>i</sub> := K[pk<sub>i</sub>] y<sub>0</sub><sup>corr</sup> ← Eval(sk<sub>i</sub>, x<sub>i,0</sub>) y<sub>1</sub><sup>corr</sup> ← Eval(sk<sub>i</sub>, x<sub>i,1</sub>) <b>if</b> y<sub>0</sub> ≠ y<sub>0</sub><sup>corr</sup> ∨ y<sub>1</sub> ≠ y<sub>1</sub><sup>corr</sup> :     <b>bad := 1</b> <b>return</b> ⊥ <b>return</b> (y<sub>0</sub><sup>corr</sup>, y<sub>1</sub><sup>corr</sup>) </pre>                                                                                                                                                                                                                        |
| $\text{Adversary } \mathcal{A}_{\text{vf}}^{\mathcal{O}_{\text{Request}}, \mathcal{O}_{\text{Finalize}}}(pp)$ <pre> creq := 0; K := [] b ← {0, 1} Start A<sup>O<sub>Request</sub>, O<sub>Finalize</sub></sup>(pp)  O<sub>RegisterKey</sub>(pk, sk) <b>if</b> public(sk) ≠ pk : <b>return</b> ⊥ K[pk] := sk  O<sub>Request</sub>(pk, x<sub>0</sub>, x<sub>1</sub>) <b>if</b> pk ∉ K : <b>return</b> ⊥ creq := creq + 1 pk<sub>creq</sub> := pk; x<sub>creq,0</sub> := x<sub>0</sub>; x<sub>creq,1</sub> := x<sub>1</sub> <b>return</b> (O<sub>Request</sub>(pk, x<sub>b</sub>), O<sub>Request</sub>(pk, x<sub>1-b</sub>)) </pre>                                                                                                                                                      | $\mathcal{O}_{\text{Finalize}}(\bar{y}', \bar{y}'', i)$ <pre> <b>if</b> i &gt; c<sub>req</sub> : <b>return</b> ⊥ // Adjust i since we query twice as often as A<sub>vf</sub>: y<sub>b</sub> ← O<sub>Finalize</sub>(ṡ', 2i - 1); y<sub>1-b</sub> ← O<sub>Finalize</sub>(ṡ'', 2i) <b>if</b> y<sub>0</sub> = ⊥ ∨ y<sub>1</sub> = ⊥ : <b>return</b> ⊥ sk<sub>i</sub> := K[pk<sub>i</sub>] y<sub>0</sub><sup>corr</sup> ← Eval(sk<sub>i</sub>, x<sub>i,0</sub>); y<sub>1</sub><sup>corr</sup> ← Eval(sk<sub>i</sub>, x<sub>i,1</sub>) <b>if</b> y<sub>0</sub><sup>corr</sup> = y<sub>0</sub> ∧ y<sub>1</sub><sup>corr</sup> = y<sub>1</sub> : <b>return</b> (y<sub>0</sub><sup>corr</sup>, y<sub>1</sub><sup>corr</sup>) // Found an incorrect response! <b>abort</b> A <b>if</b> y<sub>b</sub><sup>corr</sup> ≠ y<sub>b</sub> : Output (sk<sub>i</sub>, 2i - 1, ṡ') <b>elseif</b> y<sub>1-b</sub><sup>corr</sup> ≠ y<sub>1-b</sub> : Output (sk<sub>i</sub>, 2i, ṡ'') </pre> |

**Fig. 20.** Algorithms for proving Lemma 11. **Top:** Games  $G_0, G_1$ . **Boxed** code appears only in  $G_0$ , **Colored** code only in  $G_1$ . **Bottom:** Adversary  $\mathcal{A}_{\text{vf}}$ .

outputs into an order determined by a randomly chosen  $b$ . For  $\mathcal{O}_{\text{Finalize}}$ , they first call their own **Finalize** oracle and return  $\perp$  if one of the finalizations fails. Otherwise, they check if either of  $y_b, y_{1-b}$  is incorrect, leveraging the KOSK model to retrieve the adversary's secret key. If they find an incorrect value – this is where flag **bad** would have been set – we found a blind response to win the verifiability game!

Through the game hop we got rid of the differences between **obliv-k+** and **obliv-1k**, in particular, the **Finalize** oracle of  $G_1$  behaves identical to the corresponding oracle of **obliv-1k**.

To conclude, we can bound the advantage of any efficient **obliv-k+** adversary by the game hop and the winning probability of an adversary in  $G_1$ , which yields exactly the desired relation:

$$\begin{aligned} \text{Adv}_{\Pi, \mathcal{A}}^{\text{obliv-k}+}(\lambda) &= 2 \cdot (|\Pr[G_{0, \mathcal{A}, \Pi}(\lambda)] - \Pr[G_{1, \mathcal{A}, \Pi}(\lambda)]| + \Pr[G_{1, \mathcal{A}, \Pi}(\lambda)]) - 1 \\ &\leq 2 \cdot (\Pr[\text{Game}_{\Pi, \mathcal{A}_{\text{vf}}}^{\text{vf}}(\lambda)] + \Pr[\text{Game}_{\Pi, \mathcal{A}_{\text{obliv-1k}}}^{\text{obliv-1k}}(\lambda)]) - 1 \\ &= 2 \cdot \text{Adv}_{\Pi, \mathcal{A}_{\text{vf}}}^{\text{vf}}(\lambda) + \text{Adv}_{\Pi, \mathcal{A}_{\text{obliv-1k}}}^{\text{obliv-1k}}(\lambda) \end{aligned}$$

□

## G Full proofs for HashDH

### G.1 Correctness

It is easy to see that HashDH and vf-HashDH compute the expected function, we include the proofs for completeness.

**Theorem 12.** *HashDH is a correct OPRF.*

*Proof.* For any security parameter  $\lambda$  let  $pp \leftarrow \$ \text{Setup}(\lambda)$ ,  $(\perp, sk) \leftarrow \$ \text{KGen}(pp)$ , let  $x$  be some input  $\in \mathcal{X}$ , and  $(\bar{x}, st) \leftarrow \$ \text{Request}(\perp, x)$ . We parse  $r = st$  for consistency in notation. Then we have

$$\begin{aligned} \text{Finalize}(\text{BlindEval}(sk, \bar{x}), st) &= \text{Finalize}(\text{BlindEval}(sk, H(x)^r), r) \\ &= \text{Finalize}(H(x)^{r \cdot sk}, r) \\ &= H(x)^{r \cdot sk \cdot 1/r} = H(x)^{sk} \\ &= \text{Eval}(sk, x) \end{aligned}$$

Intuitively, for a correct NIZK scheme, an honestly generated proof is always accepted by the Vf algorithm. This property is needed for vf-HashDH, as we want to guarantee **Finalize** proceeds smoothly given an honestly generated proof of correct blind evaluation.

**Theorem 13.** *Vf-HashDH is a correct OPRF if the NIZK scheme is correct.*

*Proof.* For any security parameter  $\lambda$  let  $pp \leftarrow \$ \text{Setup}(\lambda)$ ,  $(pk, sk) \leftarrow \$ \text{KGen}(pp)$ , let  $x$  be some input  $\in \mathcal{X}$ , and  $(\bar{x}, st) \leftarrow \$ \text{Request}(pk, x)$ . We parse  $(r, \bar{x}) = st$ . Then we have

$$\begin{aligned} \text{Finalize}(\text{BlindEval}(sk, \bar{x}), st) &= \text{Finalize}(\text{BlindEval}(sk, H(x)^r), st) \\ &= \text{Finalize}((H(x)^{r \cdot sk}, \pi), st) \\ &= H(x)^{sk} \text{ if } \pi \text{ verifies w.r.t. } (pk, \bar{x}, H(x)^{r \cdot sk}) \\ &= \text{Eval}(sk, x) \text{ if } \pi \text{ verifies w.r.t. } (pk, \bar{x}, H(x)^{r \cdot sk}) \end{aligned}$$

Since NIZK is correct,  $\pi \leftarrow \$ \text{NIZK}\{(sk) : pk = g^{sk} \wedge H(x)^{r \cdot sk} = \bar{x}^{sk}\}(pk, \bar{x}, \bar{y})$  was honestly generated, and the relation  $pk = g^{sk} \wedge H(x)^{r \cdot sk} = \bar{x}^{sk}$  holds for  $sk$ , the proof is accepted and the honest interactive OPRF execution yields exactly  $\text{Eval}(sk, x)$ .

## G.2 Theorem 1: Unpredictability

The computation in HashDH that we want to be unpredictable for the requester is exponentiation by the secret key  $sk$ . Weak unpredictability therefore naturally reduces to a CDH-type problem: If the requester is able to give a fresh HashDH evaluation, they raised a fresh group element to the secret exponent. In contrast to standard CDH, **unpred** does not give a single challenge to evaluate, but the adversary wins with any fresh evaluation. We get the cleanest proof with the one-more CDH assumption (see Appendix D), which follows this structure. Proving security through standard CDH would still be possible via challenge re-randomization, albeit less readable.

The difference in strong one-more unpredictability is the additional verification oracle that the reduction must simulate. This requires solving the DDH problem efficiently, which is not feasible with regular OM-CDH. Thus, we rely on a stronger assumption for **str-unpred**, **gap-OM-CDH** (see Appendix D). Here, an extra oracle  $\mathcal{O}_{\text{DDH}}$  allows to efficiently simulate  $\mathcal{O}_{\text{Vf}}$ .

| Adversary $\mathcal{A}_{\text{gap-OM-CDH}}^{\mathcal{O}_{\text{Chal}}, \mathcal{O}_{\text{CDH}}, \mathcal{O}_{\text{DDH}}}((G, q, g), g^{\tilde{x}})$                       | $\mathcal{O}_{\text{BlindEval}}(\bar{x})$                     | $\mathcal{O}_H(x)$                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|------------------------------------------------------------|
| $\text{idx}, R := []; \quad c_{\text{Chal}} := 0; \quad \text{dh} := ()$                                                                                                    | <b>return</b> $\mathcal{O}_{\text{CDH}}(\bar{x})$             | <b>if</b> $x \notin R$ :                                   |
| $(x_1, y_1), \dots, (x_\ell, y_\ell) \leftarrow \mathcal{A}_{\text{str-unpred}}^{\mathcal{O}_{\text{BlindEval}}, \mathcal{O}_{\text{Vf}}, \mathcal{O}_H}((G, q, g), \perp)$ | $\mathcal{O}_{\text{Vf}}(x, y)$                               | $c_{\text{Chal}} := c_{\text{Chal}} + 1$                   |
| <b>foreach</b> $(x_\alpha, y_\alpha)$ <b>do</b> :                                                                                                                           | <b>return</b> $\mathcal{O}_{\text{DDH}}(g^{\tilde{x}}, x, y)$ | $Y_{c_{\text{Chal}}} \leftarrow \mathcal{O}_{\text{Chal}}$ |
| <b>if</b> $x_\alpha \notin \text{idx} : H(x_\alpha)$ // assign challenge to $x_\alpha$                                                                                      |                                                               | $R[x] := Y_{c_{\text{Chal}}}$                              |
| Append $(\text{idx}[x_\alpha], y_\alpha)$ to $\text{dh}$                                                                                                                    |                                                               | $\text{idx}[x] := c_{\text{Chal}}$                         |
| <b>return</b> $\text{dh}$                                                                                                                                                   |                                                               | <b>return</b> $R[x]$                                       |

**Fig. 21.** Adversaries  $\mathcal{A}_{\text{OM-CDH}}$  (plain) and  $\mathcal{A}_{\text{gap-OM-CDH}}$  (plain plus colored code) for proving weak and strong unpredictability of HashDH, respectively. We denote the secret exponent of the (gap-)OM-CDH game as  $\tilde{x}$  to avoid confusion with OPRF inputs  $x$ .

*Proof.* Let  $\Pi = (\text{Setup}, \text{KGen}, \text{Eval}, \text{Request}, \text{BlindEval}, \text{Finalize})$  be defined according to HashDH and  $\text{GGen}$  be an appropriate group generator. We provide a proof by contradiction and assume there exists a PPT adversary  $\mathcal{A}_{\text{unpred}}$  that wins the **unpred** game with non-negligible probability.

From  $\mathcal{A}_{\text{unpred}}$  we construct a reduction  $\mathcal{A}_{\text{OM-CDH}}$  (see Fig. 21) that breaks the OM-CDH assumption. Initially, we set up  $\mathcal{A}_{\text{unpred}}$  with the group parameters and public key. The secret exponent of the OM-CDH game will thus be the secret key of the unpredictability game. To simulate  $\mathcal{O}_{\text{BlindEval}}$ , queries are forwarded to  $\mathcal{O}_{\text{CDH}}$ , since we simply need an exponentiation by the secret key.

For every HashDH evaluation  $\mathcal{A}_{\text{unpred}}$  constructs, they need to first hash the input before exponentiating by  $sk$ . We can therefore program the random oracle to pass the CDH challenges to  $\mathcal{A}_{\text{unpred}}$ . Thus, if  $\mathcal{A}_{\text{unpred}}$  provides some fresh HashDH pair at the end, we receive a CDH challenge raised to the secret exponent. At the same time the hash oracle is simulated perfectly since the challenge  $Y_{c_{\text{Chal}}}$  is a randomly chosen element of  $\mathbb{G}$ .

Finally,  $\mathcal{A}_{\text{unpred}}$  will output  $\ell$  pairs  $(x_\alpha, y_\alpha)$ . We iterate through the pairs and check if each  $x_\alpha$  was already queried at  $\mathcal{O}_H$ . If not, then we still need to get the corresponding CDH challenge, so we call  $H$  ourselves. Then we are able to construct our own output list of challenges indices and responses.

If  $\mathcal{A}_{\text{unpred}}$  wins the one-more unpredictability game, we now also fulfill all three conditions to win the OM-CDH game. We know that  $\mathcal{A}_{\text{unpred}}$  made fewer than  $\ell$  queries to  $\mathcal{O}_{\text{BlindEval}}$ , so we made less than  $\ell$  queries to  $\mathcal{O}_{\text{CDH}}$ .  $\mathcal{A}_{\text{unpred}}$ 's output  $x_\alpha$  are distinct, so our challenge indices for the OM-CDH game are distinct, too. Finally, all pairs  $(x_\alpha, y_\alpha)$  fulfill  $H(x_\alpha)^{\tilde{x}} = y_\alpha$ . Since we programmed  $H(x_\alpha)$  to be CDH challenge  $Y_{c_{\text{Chal}}}$  for some index  $c_{\text{Chal}}$ , we also have  $(Y_{c_{\text{Chal}}})^{\tilde{x}} = y_\alpha$  and  $(c_{\text{Chal}}, y_\alpha)$  is a valid DH pair.

We conclude that  $\mathcal{A}_{\text{OM-CDH}}$  has the same winning probability as  $\mathcal{A}_{\text{unpred}}$ . This contradicts the OM-CDH assumption, so HashDH must be one-more unpredictable. Formally, we bound the advantage of  $\mathcal{A}_{\text{unpred}}$  by  $\mathcal{A}_{\text{OM-CDH}}$ :

$$\text{Adv}_{\text{HashDH}, \mathcal{A}_{\text{unpred}}}^{\text{unpred}}(\lambda) \leq \text{Adv}_{\mathcal{A}_{\text{OM-CDH}}, \text{GGen}}^{\text{om-cdh}}(\lambda)$$

The proof of **str-unpred** follows the same reasoning as the weak variant. The only change is that  $\mathcal{A}_{\text{gap-OM-CDH}}$  (also shown in Fig. 21) relies on the DDH oracle of **gap-OM-CDH** to simulate the verification oracle of the strong one-more unpredictability game. Thus, we can bound the advantage of any  $\mathcal{A}_{\text{str-unpred}}$  in the **str-unpred** game via **gap-OM-CDH**:

$$\text{Adv}_{\text{HashDH}, \mathcal{A}_{\text{str-unpred}}}^{\text{str-unpred}}(\lambda) \leq \text{Adv}_{\mathcal{A}_{\text{gap-OM-CDH}}, \text{GGen}}^{\text{gap-om-cdh}}(\lambda)$$

□

### G.3 Theorem 3: One-more pseudorandomness

Everspaugh et al. [ECS<sup>+</sup>15] prove a pairing-based version of HashDH one-more pseudorandom. A translation to standard HashDH is sketched out below.

Again, we base security on a DDH-type assumption, since winning the one-more pseudorandomness game is distinguishing HashDH evaluations from random group elements. As mentioned before, the way one-more pseudorandomness generates challenges makes it incompatible with many standard assumptions, so Everspaugh et al. [ECS<sup>+</sup>15] use a tailored one-more bilinear DDH assumption, which we convert into an analogous one-more DDH assumption shown in Appendix D.

*Proof (sketch).* We show the theorem by reduction. Assume there is a PPT adversary  $\mathcal{A}_{\text{om-prnd}}$  that wins the one-more pseudorandomness game with non-negligible probability. From  $\mathcal{A}_{\text{om-prnd}}$ , we are able to construct an adversary  $\mathcal{A}_{\text{OM-DDH}}$  for the one-more DDH game. Wlog we assume the adversary never makes redundant queries. If  $\mathcal{A}_{\text{om-prnd}}$  makes a call to the hash oracle, we use this to deliver the challenges, to this end we obtain a target base from  $\mathcal{O}_{\text{Target}}$  and return it, whilst also saving it for later use. When  $\mathcal{A}_{\text{om-prnd}}$  queries  $\mathcal{O}_{\text{Chal}}$ , we retrieve the assigned hash value, then obtain a challenge from the one-more DDH challenge oracle. This challenge is exactly a random group element or a blind evaluation with the secret key, so we achieve a perfect simulation. Queries to  $\mathcal{O}_{\text{BlindEval}}$  are answered using the helper CDH oracle.

The output of  $\mathcal{A}_{\text{om-prnd}}$ , a list of indices and a bit, is forwarded to the one-more DDH game. If  $\mathcal{A}_{\text{om-prnd}}$  wins, then they distinguished a fresh pseudorandomness challenge without querying **BlindEval**, so we also solved a fresh DDH challenge without querying the CDH oracle.

#### G.4 Theorem 2: Weak pseudorandomness

The intuition of HashDH achieving **obliv-req** is simple: A random-oracle-chosen group element is exponentiated with a random exponent, which in appropriate groups is computationally indistinguishable from choosing a random group element according to the DDH assumption. Since the weak pseudorandomness game hands out multiple challenges to the adversary via the **Eval** oracle, we employ a variation of the classic DDH assumption with multiple challenges, **DDH-mult** (see Appendix D).

The main challenge of the proof is the **BlindEval** oracle. In contrast to standard DDH assumptions,  $\mathcal{O}_{\text{BlindEval}}$  essentially leaks some CDH pairs of adversary-chosen bases, but prohibiting the adversary to query the challenge bases. We will solve this problem by introducing a game hop in which both the challenges from  $\mathcal{O}_{\text{Eval}}$  and the hints from  $\mathcal{O}_{\text{BlindEval}}$  are truly random. Using the **DDH-mult** assumption twice, we are able to show both the real and the random world of HashDH are indistinguishable from this intermediate game.

*Proof.* Let  $G_0$  be the random world of **obliv-req** and  $G_2$  be the real world of **obliv-req**. In between,  $G_1$  produces truly random values in both blind and regular evaluations, as shown in Fig. 22. For  $\mathcal{O}_{\text{Eval}}$  this is straightforward, for  $\mathcal{O}_{\text{BlindEval}}$  we make use of the fact that HashDH's blinding is easy to reverse; we first sample the random output  $y$  and then use the blinding  $st$  created by the **Request** algorithm to generate  $\bar{y}$  as  $y^{st}$ .

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $G_1(\lambda)$<br><hr/> $pp \leftarrow \text{Setup}(\lambda)$<br>$(pk, sk) \leftarrow \text{KGen}(pp)$<br>$E := []$<br>$b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Eval}}, \mathcal{O}_{\text{BlindEval}}}(pp, pk)$<br><b>return</b> $b'$<br><hr/> $\mathcal{O}_{\text{Eval}}(x)$<br><hr/> <b>if</b> $x \notin E$ : $E[x] \leftarrow \mathbb{G}$<br><b>return</b> $E[x]$<br><hr/> $\mathcal{O}_{\text{BlindEval}}(x)$<br><hr/> <b>if</b> $x \in E$ : <b>return</b> $\perp$<br>$E[x] \leftarrow \mathbb{G}; \quad \omega \leftarrow \Omega$<br>$(\bar{x}, st) \leftarrow \text{Request}(pk, x; \omega)$<br>$\bar{y} := E[x]^{st}$<br><b>return</b> $(\bar{x}, \bar{y}, E[x], \omega)$ | <div>Adversaries <math>\mathcal{A}_{\text{DDH-mult},1}^{\mathcal{O}_{\text{Chal}}}, \mathcal{A}_{\text{DDH-mult},2}^{\mathcal{O}_{\text{Chal}}}((\mathbb{G}, q, g), g^{\tilde{x}})</math></div> <hr/> $R, E, \text{Chals} := []$<br>$\text{Start } \mathcal{A}^{\mathcal{O}_{\text{Eval}}, \mathcal{O}_{\text{BlindEval}}, \mathcal{O}_{\text{H}}}((\mathbb{G}, q, g), g^{\tilde{x}})$<br><div> <div> <math>\mathcal{O}_{\text{Eval}}(x)</math><br/> <hr/> <b>if</b> <math>x \notin E</math> :<br/> <math>Y \leftarrow \mathcal{O}_{\text{H}}(x)</math><br/> <math>E[x] := \text{Chals}[Y]</math><br/> <math>E[x] \leftarrow \mathbb{G}</math><br/> <b>return</b> <math>E[x]</math> </div> <div> <math>\mathcal{O}_{\text{BlindEval}}(x)</math><br/> <hr/> <b>if</b> <math>x \in E</math> : <b>return</b> <math>\perp</math><br/> <math>Y \leftarrow \mathcal{O}_{\text{H}}(x)</math><br/> <math>E[x] := \text{Chals}[Y]</math><br/> <math>\omega \leftarrow \Omega</math><br/> <math>(\bar{x}, st) \leftarrow \text{Request}(pk, x; \omega)</math><br/> <math>\bar{y} := E[x]^{st}</math><br/> <b>return</b> <math>(\bar{x}, \bar{y}, E[x], \omega)</math> </div> </div> <hr/> $\mathcal{O}_{\text{H}}(x)$<br><hr/> <b>if</b> $x \notin R$ :<br>$(Y, Z) \leftarrow \mathcal{O}_{\text{Chal}}$<br>$\text{Chals}[Y] := Z$<br>$R[x] := Y$<br><b>return</b> $R[x]$ |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 22.** For weak pseudorandomness proof of HashDH. **Left:** Intermediate game  $G_1$ . **Right:** Reductions  $\mathcal{A}_{\text{DDH-mult},1}$  and  $\mathcal{A}_{\text{DDH-mult},2}$ . We denote the “secret exponent” of the DDH game with  $\tilde{x}$  here.

$G_0 \rightarrow G_1$ . In  $G_0$ , the oracle  $\mathcal{O}_{\text{Eval}}$  outputs randomly sampled values, whereas  $\mathcal{O}_{\text{BlindEval}}$  returns OPRF evaluations. Recall also that in *obliv-req*, the two oracles can never be queried on the same input. Distinguishing  $G_0$  and  $G_1$  therefore comes down to distinguishing the responses  $\mathcal{O}_{\text{BlindEval}}$  returns in each case – either a DH tuple  $\bar{x}, \bar{y}$ , or two unrelated values. Naturally, distinguishing the two games reduces to DDH-mult.

Our reduction  $\mathcal{A}_{\text{DDH-mult},1}$  is shown in Fig. 22.  $\mathcal{A}_{\text{DDH-mult}}$  simply returns random values in  $\mathcal{O}_{\text{Eval}}$ . Whenever they need to hash some  $x$ , they instead obtain a DDH challenge  $(Y, Z)$ . They use  $Y$  as the hash of  $x$  and  $Z$  as the corresponding evaluation. Thus, if DDH-mult provides DH tuples,  $G_0$  is simulated, if DDH-mult provides random tuples,  $G_1$  is simulated. We bound the difference between the two games by  $\mathcal{A}_{\text{DDH-mult},1}$ :

$$| \Pr[G_{0, \text{HashDH}, \mathcal{A}}(\lambda)] - \Pr[G_{1, \text{HashDH}, \mathcal{A}}(\lambda)] | \leq \text{Adv}_{\mathcal{A}_{\text{DDH-mult},1}, \text{GGen}}^{\text{ddh-mult}}(\lambda)$$

$G_1 \rightarrow G_2$ . Next, from the all-random game  $G_1$  we transition to  $G_2$ , where both  $\mathcal{O}_{\text{Eval}}$  and  $\mathcal{O}_{\text{BlindEval}}$  supply OPRF evaluations. The reduction is very similar to the first step. A DDH-mult adversary  $\mathcal{A}_{\text{DDH-mult},2}$  (see Fig. 22) uses their DDH challenges to simulate either  $G_1$  or  $G_2$ , this time also returning challenges in  $\mathcal{O}_{\text{Eval}}$ . If DDH-mult supplies DH tuples, we get  $G_2$ , else  $G_1$ . We conclude that if any distinguisher can tell between the two cases, DDH-mult is solved, and therefore bound the game hop by the reduction:

$$| \Pr[G_{1, \text{HashDH}, \mathcal{A}}(\lambda)] - \Pr[G_{2, \text{HashDH}, \mathcal{A}}(\lambda)] | \leq \text{Adv}_{\mathcal{A}_{\text{DDH-mult},2}, \text{GGen}}^{\text{ddh-mult}}(\lambda)$$

Combining the two game hops, we see that if the DDH-mult assumption holds for GGen, the two worlds of obliv-req are each indistinguishable from  $G_1$  and ultimately indistinguishable from each other:

$$\text{Adv}_{\mathcal{A}, \text{HashDH}}^{\text{weak-prnd}}(\lambda) \leq \text{Adv}_{\mathcal{A}_{\text{DDH-mult}, 1}, \text{GGen}}^{\text{ddh-mult}}(\lambda) + \text{Adv}_{\mathcal{A}_{\text{DDH-mult}, 2}, \text{GGen}}^{\text{ddh-mult}}(\lambda)$$

□

### G.5 Theorem 4: Strong pseudorandomness

In the following we show that HashDH is not strongly pseudorandom.

| $\mathcal{A}_{\text{prnd}}^{\mathcal{O}_{\text{Eval}}, \mathcal{O}_{\text{BlindEval}}, \mathcal{O}_{\text{Prim}}}(pp, pk_b)$ | $\mathcal{A}_{\text{hint-CDH}}((\mathbb{G}, q, g))$                                                  | $\mathcal{O}_{\text{Eval}}(x)$                |
|------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| $x \leftarrow \$\mathcal{X}; \quad r \leftarrow \$\mathbb{Z}_q$                                                              | $x \leftarrow \$\mathcal{X}; \quad Y^* := \perp; \quad c_{\text{BE}}, c_{\text{S}} := 0$             | <b>return</b> $Y^*$                           |
| $x' \leftarrow \mathcal{O}_{\text{Prim}}(x)$                                                                                 | $(st_{\text{S}}, pk_0) \leftarrow \text{S.Setup}^{\mathcal{O}_{\text{LimEval}}}((\mathbb{G}, q, g))$ | $\mathcal{O}_{\text{LimEval}}(x)$             |
| $\bar{x} := (x')^r$                                                                                                          | $(x', st_{\text{S}}) \leftarrow \text{S.Prim}(x, st_{\text{S}})$                                     | $c_{\text{S}} := c_{\text{S}} + 1$            |
| $\bar{y} \leftarrow \mathcal{O}_{\text{BlindEval}}(\bar{x})$                                                                 | $Y', Y^* \leftarrow \text{Game}_{\mathcal{A}, \text{GGen}}^{\text{hint-CDH}}(x')$                    | <b>if</b> $c_{\text{S}} \leq c_{\text{BE}}$ : |
| $y \leftarrow \mathcal{O}_{\text{Eval}}(x)$                                                                                  | $c_{\text{BE}} := 1$                                                                                 | <b>return</b> $\mathcal{O}_{\text{Eval}}(x)$  |
| <b>return</b> $y^r \stackrel{?}{=} \bar{y}$                                                                                  | $(\bar{y}, st_{\text{S}}) \leftarrow \text{S.BlindEval}(Y', st_{\text{S}})$                          | <b>return</b> $\perp$                         |
|                                                                                                                              | <b>return</b> $\bar{y}$                                                                              |                                               |

**Fig. 23.** For the proof of Theorem 4. **Left:** Adversary  $\mathcal{A}_{\text{prnd}}$  against strong pseudorandomness of HashDH. **Right:** Adversary  $\mathcal{A}_{\text{hint-CDH}}$  extracting a hint-CDH solution from a successful simulator S.

*Proof.* We start by giving a simple attack against HashDH’s strong pseudorandomness, where an adversary sends a single request to  $\mathcal{O}_{\text{BlindEval}}$  and checks whether the result is consistent with  $\mathcal{O}_{\text{Eval}}$ . It turns out that no simulator is able to restore this consistency unless it is capable of breaking the CDH-type assumption hint-CDH.

Fig. 23 describes our attack against str-prnd in detail: the attacker  $\mathcal{A}_{\text{prnd}}$  samples some input  $x$ , creates a blind request, queries  $\mathcal{O}_{\text{BlindEval}}$ , finalizes the response and compares the result to  $\mathcal{O}_{\text{Eval}}(x)$ . If the values are consistent,  $\mathcal{A}_{\text{prnd}}$  concludes they are in the real world.

To see why this attack succeeds, let us take the perspective of the simulator: it first receives input  $x$  at  $\mathcal{O}_{\text{Prim}}$ , to which it may respond with an arbitrary output. Note how the simulator cannot yet check  $\mathcal{O}_{\text{Eval}}(x)$ , because it is forced by  $\mathcal{O}_{\text{LimEval}}$  to wait until the requester made at least one  $\mathcal{O}_{\text{BlindEval}}$  query. After answering the requester’s random oracle query, the simulator next sees a request  $\bar{x}$  at  $\mathcal{O}_{\text{BlindEval}}$ . At this point, it is allowed to retrieve  $y \leftarrow \mathcal{O}_{\text{Eval}}(x)$ . It faces the problem of returning some response  $\bar{y}$  consistent with  $y$ , in particular, such that  $\text{Finalize}(\bar{y}, st) = \bar{y}^{1/r} = y$ . But  $r$  and  $\mathcal{O}_{\text{Eval}}(x)$  were randomly chosen, and S knows nothing about  $r$  except that whatever it produced at  $\mathcal{O}_{\text{Prim}}$  earlier raised to  $r$  is  $\bar{y}$ .

The problem of raising a random base to an exponent about which one has only partial information is captured in CDH assumptions. A peculiarity here is that the simulator may obtain a self-chosen base raised to  $r$  as a “hint”, which exceeds the scope of existing CDH assumptions. We therefore introduce a new assumption we call **hint-CDH** that features this modification. The **hint-CDH** assumption is introduced and analyzed in the AGM in Section D.1. We estimate the security degradation of the “hint” to be low, since one has to commit to a base before seeing the CDH challenge, which limits the attack surface.

Coming back to the simulator, in order to restore consistency and convince  $\mathcal{A}_{\text{prnd}}$  they are in the real world, the simulator actually solves a **hint-CDH** instance. We formalize this observation in a reduction in the following.

The reduction  $\mathcal{A}_{\text{hint-CDH}}$  (see Fig. 23) plays the **hint-CDH** game and replicates the exact strategy of  $\mathcal{A}_{\text{prnd}}$ : hashing one input, sending one request. But instead of blinding the hash themselves,  $\mathcal{A}_{\text{hint-CDH}}$  sends the hash to the **hint-CDH** game, requesting a hint. **hint-CDH** raises the hash to the random secret exponent, so this process is identical to the blinding of HashDH. Next,  $\mathcal{A}_{\text{hint-CDH}}$  uses this hint as a request to the simulator’s **BlindEval** oracle. As in the strong pseudorandomness game, we now allow the simulator to make one  $\mathcal{O}_{\text{LimEval}}$  query, but program the oracle to return the **hint-CDH** challenge  $Y^*$ . This does not impact the distribution, since  $Y^*$  is a random group element, as expected. Finally, the simulator returns some  $\bar{y}$ , which  $\mathcal{A}_{\text{hint-CDH}}$  outputs to the **hint-CDH** game.

The simulator’s  $\bar{y}$  must be equal to  $\mathcal{O}_{\text{Eval}}(x)^{\tilde{x}}$  for a successful simulation (where  $\tilde{x}$  is the secret exponent of **hint-CDH**) – but then  $\bar{y}$  is exactly  $(Y^*)^{\tilde{x}}$ , and  $\mathcal{A}_{\text{hint-CDH}}$  wins. In summary, we bound the probability of any efficient simulator  $S$  to convince  $\mathcal{A}_{\text{prnd}}$  of being in the real world as follows:

$$\Pr\left[\text{Game}_{\text{HashDH}, \mathcal{A}_{\text{prnd}}, S}^{\text{str-prnd}, b=0}(\lambda) = 1\right] \leq \Pr\left[\text{Game}_{\mathcal{A}_{\text{hint-CDH}}, \text{GGen}}^{\text{hint-CDH}}(\lambda)\right]$$

On the other hand, if  $\mathcal{A}_{\text{prnd}}$  is in the real world of strong pseudorandomness, then they will always output 1, since consistency between blind and standard evaluation is ensured. For any group in which **hint-CDH** holds, the probability of  $\mathcal{A}_{\text{prnd}}$  outputting the right bit is overwhelmingly high, making HashDH *not* strongly pseudorandom:

$$\text{Adv}_{\text{HashDH}, \mathcal{A}_{\text{prnd}}, S}^{\text{str-prnd}}(\lambda) \geq 1 - \Pr\left[\text{Game}_{\mathcal{A}_{\text{hint-CDH}}, \text{GGen}}^{\text{hint-CDH}}(\lambda)\right]$$

□

## G.6 Theorem 5: Request obliviousness

In the following we repeat the argument Bellare et al. [BRRA24] give for **obliv-req** of 2HashDH. The proof is immediately valid also for HashDH since request obliviousness only concerns the **Request** algorithm, which generates requests identically for 2HashDH and HashDH.



*Proof (sketch).* For any  $x$ , if  $H(x) \neq 1$ , then  $H(x)$  is a generator and for a uniformly random  $r \in \mathbb{Z}_q \setminus \{0\}$ , the request  $H(x)^r$  is a uniformly random element of  $\mathbb{G}$ , which makes it perfectly blind.

Only if the hash function maps an input to 1 the blinding does not work and `obliv-req` is trivial to win. We denote the event that any input  $x$  is mapped to 1 with  $H(x) = 1$  and denote the number of queries to  $H$ , both by  $\mathcal{A}$  and internally by the `Request` oracle with  $c_H$ . Assuming that  $H$  maps to 1 with a probability of  $1/q$ , we bound the advantage of any adversary in `obliv-req` as follows:

$$\begin{aligned} \text{Adv}_{\text{HashDH}, \mathcal{A}}^{\text{obliv-req}}(\lambda) &= 2 \cdot (\Pr[\text{Game}_{\text{HashDH}, \mathcal{A}}^{\text{obliv-req}}(\lambda) \mid H(x) \neq 1] \cdot \Pr[H(x) \neq 1] \\ &\quad + \Pr[\text{Game}_{\text{HashDH}, \mathcal{A}}^{\text{obliv-req}}(\lambda) \mid H(x) = 1] \cdot \Pr[H(x) = 1]) - 1 \\ &\leq 2 \cdot (\Pr[\text{Game}_{\text{HashDH}, \mathcal{A}}^{\text{obliv-req}}(\lambda) \mid H(x) \neq 1] + \Pr[H(x) = 1]) - 1 \\ &= 2 \cdot \left(\frac{1}{2} + \frac{c_H}{q}\right) - 1 = \frac{c_H}{q} \end{aligned}$$

The term  $c_H/q$  is evidently negligible with a polynomially-bound adversary and a sufficiently large group order  $q$ , so request obliviousness is achieved.  $\square$

## G.7 Theorem 6: Single-key obliviousness

Let us now detail how `obliv-1k` and `obliv-req` are equivalent in the case of `HashDH`.

*Proof.* We will argue an adversary gains no advantage from the additional abilities in `obliv-1k`, starting with the ability of generating  $\mathcal{O}_{\text{Request}}$  inputs maliciously. The first argument,  $pk$ , is not used by `HashDH`'s blinding algorithm, so there is no change. The other two arguments,  $x_0$  and  $x_1$ , could already be chosen freely from the input space  $\{0, 1\}^*$  for `obliv-req`.

The second additional ability is the `Finalize` oracle. We show that the adversary learns nothing they could not compute themselves. Recall the possible outputs of  $\mathcal{O}_{\text{Finalize}}$  are  $\perp$  or the evaluations  $\text{Eval}(sk, x_{i,0})$ ,  $\text{Eval}(sk, x_{i,1})$  of the inputs  $x_{i,0}, x_{i,1}$  and the given secret key  $sk$ . We discussed during the design of `obliv-1k` how the “true power” of  $\mathcal{O}_{\text{Finalize}}$  is not obtaining the cleartext evaluations – these can be computed by  $\mathcal{A}$  through running the `Eval` algorithm – but learning whether the finalization succeeds or fails.

But for `HashDH`, the adversary can compute even this information without help. To see this, let us first argue that `HashDH`'s `Finalize` algorithm is a bijection if we fix some blinding  $r$ . We show this by invertibility. Each blinding exponent  $r \in \mathbb{Z}_q \setminus \{0\}$  is co-prime to the prime  $q$  and therefore has a multiplicative inverse in  $\mathbb{Z}_q$ . Therefore,  $\text{Finalize}_r(\cdot) = (\cdot)^{1/r}$  is invertible; the inverse function is exactly  $\text{Finalize}_r^{-1}(\cdot) = (\cdot)^r$ . For any  $\bar{y} \in \mathbb{G}$ , we have  $\text{Finalize}_r^{-1}(\text{Finalize}_r(\bar{y})) = \bar{y}^{r \cdot 1/r} = \bar{y}$  and for any  $y \in \mathbb{G}$  we have  $\text{Finalize}_r(\text{Finalize}_r^{-1}(y)) = y^{1/r \cdot r} = y$ .

By this bijection, we can now transform the check that the `Finalize` oracle makes in order to determine whether to output  $\perp$  or not. We show the transfor-

mation for  $\bar{y}'$ , but it works analogously for  $\bar{y}''$ :

$$\begin{aligned}
\text{Finalize}(\bar{y}', st_{i,b}) &\stackrel{?}{=} \text{Eval}(sk, x_{i,b}) \\
&\Leftrightarrow \bar{y}'^{1/r_{i,b}} \stackrel{?}{=} H(x_{i,b})^{sk} \\
&\Leftrightarrow \bar{y}'^{1/r_{i,b} \cdot r_{i,b}} \stackrel{?}{=} H(x_{i,b})^{sk \cdot r_{i,b}} \\
&\Leftrightarrow \bar{y}' \stackrel{?}{=} \text{BlindEval}(\bar{x}_{i,b}, sk)
\end{aligned}$$

Clearly, the last check can be computed without help of the oracle, since all values are known to the attacker. We conclude that for HashDH, the abilities of an adversary in the **obliv-1k** game are identical to the abilities of an adversary in **obliv-req**, therefore HashDH must also be single-key oblivious.  $\square$

### G.8 Theorem 7: Multi-key obliviousness and implicit verifiability

We make use of Lemma 9, showing that HashDH is not implicitly verifiable, it follows that multi-key obliviousness is not achieved, either.

*Proof.* The following multi-key attack succeeds with probability 1 against **vf**: an adversary  $\mathcal{A}_{\text{vf}}$  generates two keys  $(\perp, sk_1), (\perp, sk_2) \leftarrow \text{KGen}(pp)$  with  $sk_1 \neq sk_2$ . They choose an arbitrary  $x$  and have  $\mathcal{O}_{\text{Request}}$  blind it into  $\bar{x}$ . Using  $sk_2$ , they blindly evaluate and obtain  $\bar{y} := \bar{x}^{sk_2}$ . Finally,  $\mathcal{A}_{\text{vf}}$  outputs  $(sk_1, 1, \bar{y})$  to the game.

The adversary fulfills all winning conditions: Since there is no public key,  $sk_1$  is always accepted as the “secret counterpart” to any  $\mathcal{O}_{\text{Request}}$  call with  $pk = \perp$ . But the evaluation key and the announced secret key are distinct, so the result of **Finalize** fails  $sk$ -verification:

$$\text{Finalize}(\bar{x}, st_1) = H(x)^{r \cdot sk_2 \cdot 1/r} = H(x)^{sk_2} \neq H(x)^{sk_1} = \text{Eval}(sk_1, x)$$

$\square$

### G.9 Theorem 8: Single-key obliviousness of **vf-HashDH**

The justification that **Vf-HashDH** achieves **obliv-req** follows directly the corresponding proof for HashDH (see Appendix G.6), since blinding is identical.

For **obliv-1k**, recall that we showed **obliv-1k** of the basic HashDH construction by arguing that any information obtainable from the new  $\mathcal{O}_{\text{Finalize}}$  is already contained in the weaker **obliv-req**, making the two notions equivalent for this specific construction (see Appendix G.7). We build on top of this proof and argue that even with the added  $sk$ -verification mechanism, adversaries do not learn more in  $\text{Game}_{\Pi, \mathcal{A}}^{\text{obliv-1k}}$  than in  $\text{Game}_{\Pi, \mathcal{A}}^{\text{obliv-req}}$ .

*Proof (sketch).* The **Request** algorithm of **Vf-HashDH** stores the public key and request in the state, but this change is irrelevant to  $\mathcal{A}_{\text{obliv-1k}}$ ’s view. **BlindEval** now additionally appends a **NIZK** to the blind response, which is subsequently verified by **Finalize**. Thus, in theory, the adversary learns from the **Finalize** oracle whether one or both proofs were invalid. However,  $\mathcal{A}_{\text{obliv-1k}}$  holds all necessary values to verify the proof themselves.

### G.10 Theorem 9: Implicit verifiability

We show in the following implicit verifiability of the vf-HashDH variant, for which we use the soundness property of the NIZK scheme.

*Proof.* Assume the NIZK used in vf-HashDH is sound. For the sake of a contradiction, let us say there is a PPT adversary  $\mathcal{A}_{\text{vf}}$  that wins the verifiability game against vf-HashDH with non-negligible probability.

According to the vf game, if  $\mathcal{A}_{\text{vf}}$  wins, they output a response that passes the proof verification during **Finalize**, but still results in an incorrect evaluation. Let  $(sk, j, \bar{y})$  with  $\bar{y} = (\bar{y}', \pi)$  be the output of  $\mathcal{A}_{\text{vf}}$ . Let  $st_j = (r_j, pk_j, \bar{x})$  be the blinding state,  $pk_j$  the public key, and  $x_j$  the input used in the  $j^{\text{th}}$  request, respectively. Let  $y' \leftarrow \text{Finalize}(y, st_j)$  be the result of finalizing the adversary's response. We know vf outputs 1 if and only if

$$pk_j = g^{sk} \wedge \pi \text{ verifies w.r.t. } (pk_j, \bar{x}, \bar{y}') \wedge y' \neq \text{HashDH.Eval}(sk, x_j)$$

This allows us to break soundness of the NIZK for the equality-of-DL relation. The NIZK proof  $\pi$  is supposed to show exactly that there exists an exponent  $\alpha$  such that  $g^\alpha = pk$  and  $\bar{x}^\alpha = \bar{y}'$ . But since  $y' \neq \text{Eval}(sk, x_j) = H(x_j)^{sk}$ , we also know that  $\bar{y}' \neq H(x_j)^{r \cdot sk} = \bar{x}^{sk}$ . So for  $g$  and  $pk_j$  the exponent  $\alpha$  is  $sk$ , but for  $\bar{x}$  and  $\bar{y}'$  it is not. The adversary was able to generate a proof for the equality of two discrete logarithms that are, in fact, not equal. This contradicts soundness of the NIZK, so we conclude that there cannot exist an attacker  $\mathcal{A}_{\text{vf}}$ .  $\square$

## H 2HashDH

2HashDH achieves all security properties of our framework except **obliv-k+** and **vf**; the latter are fulfilled by adding a verification mechanism into **Finalize** as we did for vf-HashDH. Section H.1 describes the OPRF construction; the remaining parts prove correctness, and the security and privacy properties stated in Theorem 10.

### H.1 Construction (vf-)2HashDH

2HashDH was proposed by Jarecki et al. [JKK14] as a UC-secure variant of HashDH that essentially adds an outer hash<sup>3</sup>:

$$\text{2HashDH.Eval}(sk, x) := H_2(x, H_1(x)^{sk})$$

The algorithms are largely identical to HashDH. During **Finalize**, we take what would have been the HashDH output and hash it together with the input  $x$ . **Request** therefore stores  $x$  for it to be available during finalization. We obtain a  $sk$ -verifiable variant vf-2HashDH by adding a proof of honest behavior to the blind evaluation. We specify the new **Request** and **Finalize** algorithms in the following, where the additional steps with  $sk$ -verification are shown in gray.

<sup>3</sup> There also is a version of 2HashDH that includes the public key in the outer hash.

**Request**( $pk, x$ ) :  $r \leftarrow \mathbb{Z}_q \setminus \{0\}$ ;  $\bar{x} := H_1(x)^r$ ;  $st := (r, x, pk, \bar{x})$ ; **return**  $(\bar{x}, st)$   
**Finalize**( $\bar{y}, st$ ) : **parse**  $(\bar{y}', \pi) = \bar{y}$ ; **parse**  $(r, x, pk, \bar{x}) = st$ ;  
 verify  $\pi$  w.r.t.  $(g, pk, \bar{x}, \bar{y}')$ ;  $y' := \bar{y}'^{(1/r)}$ ;  $y := H_2(x, y')$ ; **return**  $y$

## H.2 Correctness

Correctness is easy to see for 2HashDH and vf-2HashDH, we include proofs for completeness.

**Theorem 14.** *2HashDH is a correct OPRF.*

*Proof.* For any security parameter  $\lambda$  let  $pp \leftarrow \text{Setup}(\lambda)$ ,  $(\perp, sk) \leftarrow \text{KGen}(pp)$ , let  $x$  be some cleartext input  $\in \mathcal{X}$ , and  $(\bar{x}, st) \leftarrow \text{Request}(\perp, x)$ . Then we have:

$$\begin{aligned} \text{Finalize}(\text{BlindEval}(sk, \bar{x}), st) &= \text{Finalize}(\text{BlindEval}(sk, H_1(x)^r), (r, x)) \\ &= \text{Finalize}(H_1(x)^{r \cdot sk}, (r, x)) \\ &= H_2(x, H_1(x)^{r \cdot sk \cdot 1/r}) = H_2(x, H_1(x)^{sk}) \\ &= \text{Eval}(sk, x) \end{aligned}$$

□

**Theorem 15.** *vf-2HashDH is a correct OPRF if the NIZK is correct.*

*Proof.* For any security parameter  $\lambda$  let  $pp \leftarrow \text{Setup}(\lambda)$ ,  $(pk, sk) \leftarrow \text{KGen}(pp)$ , let  $x$  be some cleartext input  $\in \mathcal{X}$ , and  $(\bar{x}, st) \leftarrow \text{Request}(pk, x)$ . Then we have:

$$\begin{aligned} \text{Finalize}(\text{BlindEval}(sk, \bar{x}), st) &= \text{Finalize}(\text{BlindEval}(sk, H_1(x)^r), st) \\ &= \text{Finalize}((H_1(x)^{r \cdot sk}, \pi), st) \\ &= H_2(x, H_1(x)^{sk}) \\ &\quad \text{if } \pi \text{ verifies w.r.t } (g, pk, \bar{x}, H_1(x)^{r \cdot sk}) \\ &= \text{Eval}(sk, x) \text{ if } \pi \text{ verifies w.r.t } (g, pk, \bar{x}, H_1(x)^{r \cdot sk}) \end{aligned}$$

Since  $\pi \leftarrow \text{NIZK}\{(sk) : pk = g^{sk} \wedge H_1(x)^{r \cdot sk} = \bar{x}^{sk}\}(g, pk, \bar{x}, \bar{y})$  was honestly generated and the relation  $pk = g^{sk} \wedge H_1(x)^{r \cdot sk} = \bar{x}^{sk}$  holds for  $sk$ , the proof verifies if NIZK is correct and we get exactly the correct evaluation  $\text{Eval}(sk, x)$  out of the exchange. □

## H.3 Security of 2HashDH

**Pseudorandomness.** 2HashDH achieves strong pseudorandomness, which makes all other security properties follow.

Tyagi et al. [TCR<sup>+</sup>22] sketch a proof of strong pseudorandomness for a verifiable 2HashDH. In the following, we extend their sketch and adapt it to standard 2HashDH.

*Proof.* We perform a series of game hops. Our starting game is strong pseudo-randomness in the real world ( $b = 1$ ), which we denote here as  $G_0$ . The final game  $G_2$  will be the strong pseudorandomness game in the ideal world ( $b = 0$ ) with simulator  $S^4$ . By bounding each game hop we will show that the probability of  $\mathcal{A}_{\text{prnd}}$  distinguishing between the real and the ideal world is negligible. Fig. 24 shows all three games.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $G_0(\lambda) (= \text{Game}_{\Pi, \mathcal{A}, S}^{\text{str-prnd}, b=1})$<br><hr/> $pp \leftarrow \$ \text{Setup}(\lambda)$<br>$(\perp, sk) \leftarrow \$ \text{KGen}(pp)$<br>$st_S \leftarrow \$ S.\text{Setup}(pp)$<br>$R_1, R_2 := []$<br>$b' \leftarrow \mathcal{A}_{\text{prnd}}^{\mathcal{O}_{\text{Eval}}, \mathcal{O}_{\text{BlindEval}}, \mathcal{O}_{H_1}, \mathcal{O}_{H_2}}(pp, \perp)$<br><b>return</b> $b'$<br><hr/> $\mathcal{O}_{\text{Eval}}(x)$<br><b>return</b> $H_2(x, H_1(x)^{sk})$<br><hr/> $\mathcal{O}_{\text{BlindEval}}(\bar{x})$<br><b>return</b> $\bar{x}^{sk}$<br><hr/> $\mathcal{O}_{H_1}(x)$<br><b>if</b> $x \notin R_1 : R_1[x] \leftarrow \$ \mathbb{G}$<br><b>return</b> $R_1[x]$<br><hr/> $\mathcal{O}_{H_2}(x, y')$<br><b>if</b> $(x, y') \notin R_2 :$<br>$R_2[x, y'] \leftarrow \$ \{0, 1\}^n$<br><b>return</b> $R_2[x, y']$ | $G_1(\lambda)$<br><hr/> $E := []$<br>// otherwise unchanged<br><hr/> $\mathcal{O}_{\text{Eval}}(x)$<br><b>if</b> $x \notin E : E[x] \leftarrow \$ \{0, 1\}^n$<br><b>return</b> $E[x]$<br><hr/> $\mathcal{O}_{H_2}(x, y')$<br><b>if</b> $(x, y') \notin R_2 :$<br><b>if</b> $y' = H_1(x)^{sk} :$<br>$R_2[x, y'] \leftarrow \mathcal{O}_{\text{Eval}}(x)$<br><b>else :</b><br>$R_2[x, y'] \leftarrow \{0, 1\}^n$<br><b>return</b> $R_2[x, y']$ | $G_2(\lambda) (= \text{Game}_{\Pi, \mathcal{A}, S}^{\text{str-prnd}, b=0})$<br><hr/> $c_{BE}, c_{Fin} := 0$<br>$bad := 0$<br>// otherwise unchanged<br><hr/> $\mathcal{O}_{\text{BlindEval}}(\bar{x})$<br>$c_{BE} := c_{BE} + 1$<br>// otherwise unchanged<br><hr/> $\mathcal{O}_{H_2}(x, y')$<br><b>if</b> $(x, y') \notin R_2 :$<br><b>if</b> $y' = H_1(x)^{sk} :$<br>$c_{Fin} := c_{Fin} + 1$<br><b>if</b> $c_{Fin} > c_{BE} :$<br>$bad := 1$<br><b>return</b> $\perp$<br>$R_2[x, y'] \leftarrow \mathcal{O}_{\text{Eval}}(x)$<br><b>else :</b><br>$R_2[x, y'] \leftarrow \{0, 1\}^n$<br><b>return</b> $R_2[x, y']$ |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 24.** Games  $G_0, G_1, G_2$  for proving pseudorandomness of 2HashDH. Highlighted lines are changed in the current game hop.

$G_0 \rightarrow G_1$ .  $G_1$  changes the cleartext evaluation into sampling truly random values. To maintain consistency, we also program  $H_2$  to return the same value in case the inputs match a previous evaluation. The distribution of the PRF outputs does not change, since in the ROM the values returned by the outer hash functions are already truly random in  $G_0$ . We also take care of consistency, so the two games are indistinguishable for an attacker.

$$|\Pr[G_{0, 2\text{HashDH}, \mathcal{A}}(\lambda)] - \Pr[G_{1, 2\text{HashDH}, \mathcal{A}}(\lambda)]| = 0$$

<sup>4</sup> Formally, we do lazy sampling in  $G_2$ , so we would need another game hop that samples with a random function  $f$  in order to be identical to the strong pseudorandomness game. But this transition is straightforward, so we omit it here.

| Adversary $\mathcal{A}_{\text{gap-OM-CDH}}^{\mathcal{O}_{\text{Chal}}, \mathcal{O}_{\text{CDH}}, \mathcal{O}_{\text{DDH}}}((\mathbb{G}, q, g), g^{\tilde{x}})$ |                                               | $\mathcal{O}_{H_2}(x, y')$                                                                         |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------------|
| $c_{\text{BE}}, c_{\text{Fin}}, i := 0; \quad \text{dh} := (); \quad \text{idx}, R_1, R_2, E := []$                                                            |                                               | <b>if</b> $(x, y') \notin R_2$ :                                                                   |
| Start $\mathcal{A}_{\text{prnd}}^{\mathcal{O}_{\text{Eval}}, \mathcal{O}_{\text{BlindEval}}, \mathcal{O}_{H_1}, \mathcal{O}_{H_2}}((\mathbb{G}, q, g), \perp)$ |                                               | <b>if</b> $\mathcal{O}_{\text{DDH}}(g, g^{\tilde{x}}, \mathcal{O}_{H_1}(x), y') = 1$ :             |
|                                                                                                                                                                |                                               | $c_{\text{Fin}} := c_{\text{Fin}} + 1$                                                             |
|                                                                                                                                                                |                                               | Add $(\text{idx}[x], y')$ to $\text{dh}$                                                           |
|                                                                                                                                                                |                                               | <b>if</b> $c_{\text{BE}} \geq c_{\text{Fin}} : R_2[x, y'] \leftarrow \mathcal{O}_{\text{Eval}}(x)$ |
|                                                                                                                                                                |                                               | <b>else</b> : // Found a fresh CDH value                                                           |
|                                                                                                                                                                |                                               | <b>abort</b> $\mathcal{A}_{\text{prnd}}$                                                           |
|                                                                                                                                                                |                                               | Output $\text{dh}$                                                                                 |
|                                                                                                                                                                |                                               | <b>else</b> : $R_2[x, y'] \leftarrow \{0, 1\}^n$                                                   |
|                                                                                                                                                                |                                               | <b>return</b> $R_2[x, y']$                                                                         |
| $\mathcal{O}_{\text{Eval}}(x)$                                                                                                                                 | $\mathcal{O}_{H_1}(x)$                        |                                                                                                    |
| <b>if</b> $x \notin E$ :                                                                                                                                       | <b>if</b> $x \notin R_1$ :                    |                                                                                                    |
| $E[x] \leftarrow \{0, 1\}^n$                                                                                                                                   | // Assign a challenge                         |                                                                                                    |
| <b>return</b> $E[x]$                                                                                                                                           | $i := i + 1$                                  |                                                                                                    |
|                                                                                                                                                                | $\text{idx}[x] := i$                          |                                                                                                    |
|                                                                                                                                                                | $R_1[x] \leftarrow \mathcal{O}_{\text{Chal}}$ |                                                                                                    |
| $\mathcal{O}_{\text{BlindEval}}(\bar{x})$                                                                                                                      | <b>return</b> $R_1[x]$                        |                                                                                                    |
| $c_{\text{BE}} := c_{\text{BE}} + 1$                                                                                                                           |                                               |                                                                                                    |
| <b>return</b> $\mathcal{O}_{\text{CDH}}(\bar{x})$                                                                                                              |                                               |                                                                                                    |

**Fig. 25.** Adversary  $\mathcal{A}_{\text{gap-OM-CDH}}$  to bound game hop  $G_1$  to  $G_2$ . We denote the “secret exponent” of the **gap-OM-CDH** assumption as  $\tilde{x}$  here to avoid variable collision.

$G_1 \rightarrow G_2$   $G_1$  already was very similar to the pseudorandomness game in the ideal world, where blind evaluation and the hash functions are handled by  $\mathbf{S}$ . The remaining difference we need to overcome is that a simulator cannot query the evaluation oracle directly but has the restricted oracle  $\mathcal{O}_{\text{LimEval}}$ , which fails if there are no calls left. This limit is reached if  $\mathcal{O}_{\text{LimEval}}$  is queried more times than there have been calls to  $\mathcal{O}_{\text{BlindEval}}$ .

Therefore, in  $G_2$  we track the  $\mathcal{O}_{\text{BlindEval}}$  calls with counter  $c_{\text{BE}}$  and increment a second counter  $c_{\text{Fin}}$  whenever we “finalize” an evaluation in the  $H_2$  oracle – so whenever  $\mathbf{S}$  would have to query  $\mathcal{O}_{\text{LimEval}}$ . We encounter a problem in  $\mathcal{O}_{H_2}$  if the adversary’s queries included  $c_{\text{Fin}}$  finalizations of  $2\text{HashDH}$  evaluations but less than  $c_{\text{Fin}}$  queries were made to  $\mathcal{O}_{\text{BlindEval}}$ . Since a simulator could not handle this situation, we set a flag **bad** and return  $\perp$ . In this case, however, the queries sent to  $H_2$  must include a fresh DH pair: at some point, the adversary sent a  $H_1(x)^{sk}$  without obtaining the exponentiation by the secret key from the **BlindEval** oracle. This allows us to break the **gap-OM-CDH** assumption.

Via the Difference Lemma [Sho04], we have that

$$|\Pr[G_{1,2\text{HashDH},\mathcal{A}}(\lambda)] - \Pr[G_{2,2\text{HashDH},\mathcal{A}}(\lambda)]| \leq \Pr[\text{bad} = 1]$$

We can bound the probability of the flag **bad** being set to the advantage of an adversary  $\mathcal{A}_{\text{gap-OM-CDH}}$  that wins the **gap-OM-CDH** game when the flag **bad** is set.  $\mathcal{A}_{\text{gap-OM-CDH}}$  is given in Fig. 25.

$\mathcal{A}_{\text{gap-OM-CDH}}$  checks for each pair  $x, y'$  in  $\mathcal{O}_{H_2}$  if  $y' = H_1(x)^{\tilde{x}}$ , by using their  $\mathcal{O}_{\text{DDH}}$  oracle. In this case an OPRF evaluation is being finalized. If a **bad** event occurs,  $\mathcal{A}_{\text{prnd}}$  computed at least one  $y' = H_1(x)^{\tilde{x}}$  without using  $\mathcal{O}_{\text{BlindEval}}$ . In particular, they were able to exponentiate some  $H_1(x)$  with the secret exponent  $\tilde{x}$ . Once we encounter this surplus exponentiation in  $\mathcal{O}_{H_2}$ , we output all of the DH pairs we saw so far, and there will be one more than we handed out.

We need to ensure two conditions in order to win  $\text{Game}_{\mathcal{A}, \text{GGen}}^{\text{gap-OM-CDH}}$  with the collected DH pairs: they must be exponentiations of challenges obtained from

$\mathcal{O}_{\text{Chal}}$ , and the exponent must be  $\tilde{x}$ . For the former condition, we feed the challenges to the adversary in  $\mathcal{O}_{H_1}$ . For the latter, we forward  $\mathcal{O}_{\text{BlindEval}}$  queries to our own oracle  $\mathcal{O}_{\text{CDH}}$ .

Thus, we bound the difference between  $G_1$  and  $G_2$  by **gap-OM-CDH**:

$$|\Pr[G_{1,2\text{HashDH},\mathcal{A}}(\lambda)] - \Pr[G_{2,2\text{HashDH},\mathcal{A}}(\lambda)]| \leq \Pr\left[\text{Game}_{\mathcal{A}_{\text{gap-OM-CDH}}, \text{GGen}}^{\text{gap-OM-CDH}}(\lambda)\right]$$

$G_2$  is now identical to the strong pseudorandomness game in the ideal world with a simulator  $\mathbf{S} = (\mathbf{S}.\text{Setup}, \mathbf{S}.\text{BlindEval}, \mathbf{S}.\mathbf{H}_1, \mathbf{S}.\mathbf{H}_2)$  that we specify in Fig. 26. It is easy to see  $\mathbf{S}$  runs exactly  $G_2$  but calls  $\mathcal{O}_{\text{LimEval}}$  instead of  $\mathcal{O}_{\text{Eval}}$  (highlighted in the code).

| $\mathbf{S}.\text{Setup}(pp)$               | $\mathbf{S}.\text{BlindEval}^{\mathcal{O}_{\text{LimEval}}}(\bar{x}, st_S)$ | $\mathbf{S}.\mathbf{H}_2^{\mathcal{O}_{\text{LimEval}}}(x, y', st_S)$ |
|---------------------------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Parse $(G, q, g) = pp$                      | Parse $(pp, sk, R_1, R_2) = st_S$                                           | Parse $(pp, sk, R_1, R_2) = st_S$                                     |
| $R_1, R_2 := []$                            | <b>return</b> $(\bar{x}^{sk}, st_S)$                                        | <b>if</b> $(x, y') \notin R_2$ :                                      |
| $sk \leftarrow \mathbb{Z}_q$                | $\mathbf{S}.\mathbf{H}_1^{\mathcal{O}_{\text{LimEval}}}(x, st_S)$           | <b>if</b> $y' = \mathbf{S}.\mathbf{H}_1(x)^{sk}$ :                    |
| <b>return</b> $((pp, sk, R_1, R_2), \perp)$ | Parse $(pp, sk, R_1, R_2) = st_S$                                           | $R_2[x, y'] \leftarrow \mathcal{O}_{\text{LimEval}}(x)$               |
|                                             | <b>if</b> $x \notin R_1$ :                                                  | <b>else</b> :                                                         |
|                                             | $R_1[x] \leftarrow \mathbb{G}$                                              | $R_2[x, y'] \leftarrow \mathbb{Z}_q$                                  |
|                                             | <b>return</b> $(R_1[x], st_S)$                                              | <b>return</b> $(R_2[x, y'], st_S)$                                    |

**Fig. 26.** Simulator  $\mathbf{S}$  for strong pseudorandomness of 2HashDH.

In conclusion, we bound the advantage of any adversary  $\mathcal{A}_{\text{prnd}}$  in the **str-prnd** game against 2HashDH, with simulator  $\mathbf{S}$ , as follows:

$$\text{Adv}_{2\text{HashDH}, \mathcal{A}_{\text{prnd}}, \mathbf{S}}^{\text{str-prnd}}(\lambda) \leq \Pr\left[\text{Game}_{\mathcal{A}_{\text{gap-OM-CDH}}, \text{GGen}}^{\text{gap-OM-CDH}}(\lambda)\right]$$

Since we assumed the **gap-OM-CDH** assumption to hold for  $\text{GGen}$ , the above advantage must be negligible and 2HashDH is strongly pseudorandom.  $\square$

**Unpredictability.** Both unpredictability properties of 2HashDH also come as a corollary of **str-prnd**. Additionally, we provide in the following a direct proof for **unpred**. This allows to use the simpler **OM-CDH** assumption instead of the **gap-OM-CDH** assumption. The proof proceeds similarly to that of HashDH, so we only sketch the differences below and refer to Section 6.1 for details.

Note that while Theorem 10 formally asks for both hash functions to be random oracles, the requirements for the outer function are actually somewhat weaker. The inner hash function  $H_1$  has to be programmable, as we will program it in the reduction to return CDH challenges. Whereas  $H_2$  is not programmed to specific values, we only rely on *observability* to extract the responses to our CDH challenges. Specifically, we need the guarantee that we can observe any input to  $H_2$ , and that the adversary can not simply *guess* correct hash values without calling the oracle.

*Proof (sketch).* For HashDH, the adversary's outputs in the unpredictability game were of the form  $(x, H(x)^{sk})$ , so we were able to extract DH pairs directly. In contrast, for 2HashDH we have outputs of the form  $(x, H_2(x, H_1(x)^{sk}))$ .

| Adversary $\mathcal{A}_{\text{OM-CDH}}^{\mathcal{O}_{\text{Chal}}, \mathcal{O}_{\text{CDH}}}((\mathbb{G}, q, g), g^{\tilde{x}})$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | $\mathcal{O}_{H_1}(x)$                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{chalidx}, R_1, R_2, R_2^{-1} := []; \quad c_{\text{Chal}} := 0; \quad \text{dh} := ()$<br>$(x_1, y_1), \dots, (x_n, y_n) \leftarrow \mathcal{A}_{\text{unpred}}^{\mathcal{O}_{\text{BlindEval}}, \mathcal{O}_{H_1}, \mathcal{O}_{H_2}}((\mathbb{G}, q, g), \perp)$<br><b>foreach</b> $y_\alpha$ <b>do</b> :<br><b>if</b> $y_\alpha \notin R_2^{-1}$ : <b>abort</b> // $H_2$ value was guessed<br>Parse $(x_\alpha, y'_\alpha) = R_2^{-1}(y_\alpha)$<br><b>if</b> $x_\alpha \notin \text{chalidx} : H(x_\alpha)$ // assign challenge to $x_\alpha$<br>Append $(\text{chalidx}[x_\alpha], y'_\alpha)$ to $\text{dh}$<br><b>return</b> $\text{dh}$<br><hr/> $\mathcal{O}_{\text{BlindEval}}(\bar{x})$<br><b>return</b> $\mathcal{O}_{\text{CDH}}(\bar{x})$ | <b>if</b> $x \notin R_1$ :<br>$c_{\text{Chal}} := c_{\text{Chal}} + 1$<br>$Y_{c_{\text{Chal}}} \leftarrow \mathcal{O}_{\text{Chal}}$<br>$R_1[x] := Y_{c_{\text{Chal}}}$<br>$\text{chalidx}[x] := c_{\text{Chal}}$<br><b>return</b> $R_1[x]$<br><hr/> $\mathcal{O}_{H_2}(x, y')$<br><b>if</b> $(x, y') \notin R_2$ :<br>$R_2[x, y'] \leftarrow \$ \{0, 1\}^n$<br>$R_2^{-1}[R_2[x, y']] := (x, y')$<br><b>return</b> $R_2[x, y']$ |

**Fig. 27.** Adversary  $\mathcal{A}_{\text{OM-CDH}}$  for proving unpredictability of 2HashDH. We denote the secret exponent of the OM-CDH game as  $\tilde{x}$  to avoid confusion with OPRF inputs  $x$ .

The reduction to the OM-CDH problem is shown in Fig. 27, with the important changes to the corresponding HashDH proof highlighted. To deal with the outer hash, we simply let the random oracle  $H_2$  store all inputs. From  $\mathcal{A}_{\text{unpred}}$ 's outputs  $(x_1, y_1), \dots, (x_\ell, y_\ell)$  we retrieve the preimage of each  $y_\alpha$ . If  $y_\alpha = H_2(x_\alpha, H_1(x_\alpha)^{sk})$  and we programmed  $H_1$  to return some DH challenges as before, then we saw in  $\mathcal{O}_{H_2}$  exactly the response we need.

We only encounter a problem if  $y_\alpha$  was never seen at  $\mathcal{O}_{H_2}$ , but then the  $(x_\alpha, y_\alpha)$  is either not a valid 2HashDH pair, or the adversary was able to guess the right hash value. The probability of guessing a hash is negligible for a sufficiently large output space  $\{0, 1\}^n$ , so we can safely ignore this case.

Informally, via programming the random oracles and observing the inputs the adversary sends to  $\mathcal{O}_{H_2}$ , we succeed in building our list of one-more CDH pairs at the end of the game and win (with rare exceptions) if  $\mathcal{A}_{\text{unpred}}$  wins.

#### H.4 Privacy of 2HashDH

Privacy is achieved similarly to HashDH, but the outer hash causes a few technical intricacies. As before, we make use of Lemma 11 to avoid a complex direct proof of *obliv-k+* and show instead the weaker obliviousness properties and *vf*.

**Request and single-key obliviousness.** Since HashDH and 2HashDH share the same request generation algorithm, the proof for *obliv-req* follows directly the proof of Theorem 5 for both regular and verifiable 2HashDH.



While obliv-1k of 2HashDH is similar to the case of HashDH, here the outer hash function requires some care. In the proof we will build on collision resistance of  $H_2$ , so we first define this notion formally. Fig. 28 shows the game for collision resistance.

---


$$\begin{array}{l}
 \text{Game}_{H,A}^{\text{CR}}(\lambda) \\
 \hline
 (m_0, m_1) \leftarrow \mathcal{A}(1^\lambda) \\
 \text{return } m_0 \stackrel{?}{\neq} m_1 \wedge H(m_0) \stackrel{?}{=} H(m_1)
 \end{array}$$


---

**Fig. 28.** Game for collision resistance of a hash function.

**Definition 14 (Collision resistance of hash functions).** We call  $H$  collision resistant if  $\forall$  PPT adversaries  $\mathcal{A}$  the following advantage is negligible:

$$\text{Adv}_{H,A}^{\text{cr}}(\lambda) := \Pr \left[ \text{Game}_{H,A}^{\text{CR}}(\lambda) \right]$$

Let us build some intuition for why we need collision resistance. Recall that during the **Finalize** procedure, the cleartext input  $x$  is hashed together with the unblinded response,  $H_1(x)^{sk}$ . If an adversary sends a dishonest response  $\bar{y}_{\text{mal}}$ , hopefully the result of  $H_2(x, y_{\text{mal}})$  differs from the correct  $H_2(x, H_1(x)^{sk})$ . Unfortunately, it will not differ if  $H_2$  happens to collide for the two inputs.

A badly chosen  $H_2$  could directly be exploited in the obliv-1k game. The adversary  $\mathcal{A}$  selects two inputs  $x_0, x_1$ , s.t.  $(x_0, H_1(x_0)^{sk})$  has many collisions in  $H_2$ , but  $(x_1, H_1(x_1)^{sk})$  does not. They obtain requests  $\bar{x}_b, \bar{x}_{1-b}$  from  $\mathcal{O}_{\text{BlindEval}}$  and evaluate them blindly to obtain  $\bar{y}_b, \bar{y}_{1-b}$ . They replace one of these responses with a different random group element and send them to  $\mathcal{O}_{\text{Finalize}}$ . Whether the oracle fails or not indicates if the random response caused a hash collision, which should happen much more often if  $\mathcal{A}$  replaced the response belonging to  $x_0$ .

Next, we give a formal proof.

*Proof.* We analyze whether the adversary gains additional powers with respect to the **obliv-req** game. Recall that **obliv-1k** makes the attacker active and provides an additional **Finalize** oracle.

Firstly, moving to the active model does not change what the adversary can learn from  $\mathcal{O}_{\text{Request}}$ : The inputs  $x_0, x_1$  could already be chosen without restrictions in the passive version, and the public key is unused.

Secondly, we want to argue that any information given by  $\mathcal{O}_{\text{Finalize}}$  could be computed by the adversary themselves. The outputs  $y_0, y_1$  can evidently be determined by running the **Eval** algorithm on the inputs and any chosen secret key. But  $\mathcal{O}_{\text{Finalize}}$  also implicitly tells whether finalizing the responses resulted in the correct evaluations. This check cannot be computed by the adversary directly, since they cannot unblind the responses.

We therefore do a game hop where we replace the check to something  $\mathcal{A}$  can compute autonomously. The modified game is shown in Fig. 29.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $G_0(\lambda) \quad (= \text{Game}_{\text{H}, \mathcal{A}}^{\text{obliv-1k}}, \overline{G_1(\lambda)})$<br>$pp \leftarrow \text{Setup}(\lambda)$<br>$c_{\text{req}} := 0$<br>$b \leftarrow \{0, 1\}$<br>$b' \leftarrow \mathcal{O}_{\text{Request}, \mathcal{O}_{\text{Finalize}}}(pp)$<br><b>return</b> $b \stackrel{?}{=} b'$<br><hr/> $\mathcal{O}_{\text{Request}}(pk, x_0, x_1)$<br>$c_{\text{req}} := c_{\text{req}} + 1; \quad x_{c_{\text{req}}, 0} := x_0; \quad x_{c_{\text{req}}, 1} := x_1$<br>$(\overline{x_{c_{\text{req}}, b}}, st_{c_{\text{req}}, b}) \leftarrow \text{Request}(pk, x_b)$<br>$(\overline{x_{c_{\text{req}}, 1-b}}, st_{c_{\text{req}}, 1-b}) \leftarrow \text{Request}(pk, x_{1-b})$<br><b>return</b> $(\overline{x_b}, \overline{x_{1-b}})$ | $\mathcal{O}_{\text{Finalize}}(\overline{y}', \overline{y}'', i, sk)$<br><hr/> <b>if</b> $i > c_{\text{req}}$ : <b>return</b> $\perp$<br>$y_b \leftarrow \text{Finalize}(\overline{y}', st_{i, b})$<br>$y_{1-b} \leftarrow \text{Finalize}(\overline{y}'', st_{i, 1-b})$<br><b>if</b> $y_b = \perp \vee y_{1-b} = \perp$ : <b>return</b> $\perp$<br><b>if</b> $y_0 \neq \text{Eval}(sk, x_{i, 0}) \vee y_1 \neq \text{Eval}(sk, x_{i, 1})$ :<br><b>return</b> $\perp$<br><b>if</b> $\overline{y}' \neq (\overline{x_{i, b}})^{sk} \vee \overline{y}'' \neq (\overline{x_{i, 1-b}})^{sk}$ :<br><b>if</b> $y_0 = \text{Eval}(sk, x_{i, 0}) \wedge y_1 = \text{Eval}(sk, x_{i, 1})$ :<br>$\text{bad} := 1$<br><b>return</b> $\perp$<br><b>return</b> $(y_0, y_1)$ |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 29.** Games for proving single-key obliviousness of 2HashDH. **Colored** code is only contained in  $G_0$ , **boxed** code only in  $G_1$ .

$G_0$  is the regular obliv-1k game. Recall that here  $\mathcal{O}_{\text{Finalize}}$  tests whether each finalization is correct, i.e., for  $y_b$  whether  $y_b \neq \text{Eval}(sk, x_{i, b})$ . Observe first that  $y_b = H_2(x_{i, b}, \overline{y}^{1/r_{i, b}})$  and  $\text{Eval}(sk, x_{i, b}) = H_2(x_{i, b}, H_1(x_{i, b})^{sk})$  can only be unequal if  $\overline{y}^{1/r_{i, b}} \neq H_1(x_{i, b})^{sk}$ .

Using this observation, we replace the check in  $G_1$  (to be precise, we compare the blind version of each value, but this is equivalent). However, by doing this we introduce a subtle change in the oracle's behavior: it might happen that even though we find an inequality,  $H_2$  has a hash collision and outputs the same result, and we now return  $\perp$  where the original oracle would not. To account for this behavioral change we set a flag **bad**. Otherwise, the behavior is unchanged, so the  $G_0$  and  $G_1$  are identical-until-bad.

Then, since we know **bad** is only set in case of a hash collision of  $H_2$ , we can bound the probability of this event using the collision resistance property of  $H_2$ .

Fig. 30 shows the reduction.  $\mathcal{A}_{\text{CR}}$  simulates the game  $G_1$  and extracts a  $H_2$  collision if the **bad** event occurs: if we find a dishonest response  $\overline{y}'$  or  $\overline{y}''$  but the resulting evaluation is correct, then this response must have a hash collision with the correct response, which we use to win the collision resistance game.

In  $G_1$  the check in  $\mathcal{O}_{\text{Finalize}}$  can evidently be computed by the adversary, so the oracle becomes obsolete. Consequently, adversaries in  $G_1$  do not have any stronger abilities than in the obliv-req game, which we know to be hard for 2HashDH. In conclusion, we bound the advantage of any obliv-1k adversary by collision resistance of  $H_2$  and obliv-req of 2HashDH:

$$\begin{aligned}
 \text{Adv}_{2\text{HashDH}, \mathcal{A}}^{\text{obliv-1k}}(\lambda) &\leq \text{Adv}_{2\text{HashDH}, \mathcal{A}}^{G_1}(\lambda) + \Pr[\text{bad} = 1] \\
 &\leq \text{Adv}_{2\text{HashDH}, \mathcal{A}}^{\text{obliv-req}}(\lambda) + \text{Adv}_{H_2, \mathcal{A}_{\text{CR}}}^{\text{cr}}(\lambda)
 \end{aligned}$$

□

Adding a proof of honest behavior to the blind evaluation in fact simplifies the treatment of obliv-1k, since the attack exploiting  $H_2$  collisions described above

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <hr/> <b>Adversary</b> $\mathcal{A}_{\text{CR}}(1^\lambda)$<br><hr/> $pp \leftarrow \$ \text{Setup}(\lambda)$<br>$c_{\text{req}} := 0$<br>$b \leftarrow \$ \{0, 1\}$<br>Start $\mathcal{A}_{\text{obliv-1k}}^{\mathcal{O}_{\text{Request}}, \mathcal{O}_{\text{Finalize}}}(pp)$<br><hr/> $\mathcal{O}_{\text{Request}}(pk, x_0, x_1)$<br><hr/> $c_{\text{req}} := c_{\text{req}} + 1$<br>$(\overline{x_{c_{\text{req}}, b}}, st_{c_{\text{req}}, b}) \leftarrow \$ \text{Request}(pk, x_b)$<br>$(\overline{x_{c_{\text{req}}, 1-b}}, st_{c_{\text{req}}, 1-b}) \leftarrow \$ \text{Request}(pk, x_{1-b})$<br><b>return</b> $(\overline{x_{c_{\text{req}}, b}}, \overline{x_{c_{\text{req}}, 1-b}})$ | <hr/> $\mathcal{O}_{\text{Finalize}}(\overline{y'}, \overline{y''}, i, sk)$<br><hr/> <b>if</b> $i > c_{\text{req}}$ : <b>return</b> $\perp$<br>$y_b \leftarrow \text{Finalize}(\overline{y'}, st_{i, b})$<br>$y_{1-b} \leftarrow \text{Finalize}(\overline{y''}, st_{i, 1-b})$<br><b>if</b> $y_b = \perp \vee y_{1-b} = \perp$ : <b>return</b> $\perp$<br><b>if</b> $\overline{y'} \neq (\overline{x_{i, b}})^{sk} \vee \overline{y''} \neq (\overline{x_{i, 1-b}})^{sk}$ :<br><b>if</b> $y_0 = \text{Eval}(sk, x_{i, 0}) \wedge y_1 = \text{Eval}(sk, x_{i, 1})$ :<br>// Found a $H_2$ collision<br><b>abort</b> $\mathcal{A}_{\text{obliv-1k}}$<br><b>if</b> $\overline{y'} \neq (\overline{x_{i, b}})^{sk} : m_{\text{col}} := \overline{y'}^{1/r_{i, b}}$<br><b>else</b> : $m_{\text{col}} := \overline{y''}^{1/r_{i, 1-b}}$<br><b>Output</b> $(x_{i, b}, m_{\text{CR}}), (x_{i, b}, H_1(x_{i, b})^{sk})$<br><b>return</b> $\perp$<br><b>return</b> $y_0, y_1$ |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 30.** Adversary for the single-key obliviousness proof of 2HashDH:  $\mathcal{A}_{\text{CR}}$  extracts a  $H_2$  collision if the **bad** event happens in  $G_1$ .

is no longer possible – in order to find some dishonest response that leads to a hash collision with the honest one, an attacker would first have to forge a NIZK for this response.

**Theorem 16.** *vf-2HashDH is oblivious-1k if  $H_1$  is a RO and the NIZK is sound.*

*Proof (sketch).* We argue once more about the following check made in  $\mathcal{O}_{\text{Finalize}}$  of **obliv-1k**:

$$y_0 \neq \text{Eval}(sk, x_{i, 0}) \vee y_1 \neq \text{Eval}(sk, x_{i, 1})$$

For vf-2HashDH, this check can in fact never fail, except if the adversary forged a NIZK. Recall from the proof of **obliv-1k** for 2HashDH above that whenever the check returns false, it must also hold that the response is different from the honest blind evaluation of the blind request:

$$\overline{y'} \neq (\overline{x_{i, b}})^{sk} \vee \overline{y''} \neq (\overline{x_{i, 1-b}})^{sk},$$

But this time, the adversary actually had to prove honest evaluation, and the prove was verified earlier in  $\mathcal{O}_{\text{Finalize}}$  when executing the **Finalize** algorithm on each input. Therefore, if later in the oracle, the check still fails, then we must have received a forged NIZK, which contradicts soundness. Formally, we can extract a forged proof whenever the check fails.

It follows that for a sound NIZK system, the check for correctness of  $y_0$  and  $y_1$  should fail only with negligible probability. On the other hand, if the check is skipped every time, then the oracle becomes pointless: It either outputs  $\perp$  because one of the proofs did not verify, or the correct evaluations  $y_0, y_1$ , both of which a **obliv-req** adversary without the **Finalize** could compute, too. In conclusion, with a sound NIZK, **obliv-1k** is equivalent to **obliv-req**.

**Verifiability.** Since 2HashDH and HashDH have an identical blind evaluation algorithm, verifiability is analogous between the two constructions. We therefore only provide brief sketches of the arguments here and refer to the proofs of Theorem 7 and Theorem 9 for further details.

**Theorem 17.** *2HashDH is not vf.*

*Proof (sketch).* We can win the verifiability game against 2HashDH as follows: if we blindly evaluate a request with a secret key  $sk_1$  and announce a distinct secret key  $sk_2$ , then the resulting cleartext evaluation will differ from the expected correct evaluation.  $sk_2$  is not rejected by the implicit verifiability game, since there is no public key to check against.

As for HashDH, **obliv-k+** and **vf** can be achieved by adding a proof of honest behavior on the evaluator side, here, via a sound NIZK.

*Proof (sketch).* Just as **vf-HashDH**, **vf-2HashDH** adds a NIZK for the equality of the discrete logarithms between generator and public key, and blind request and response. If an adversary  $\mathcal{A}_{\text{vf}}$  is able to win the verifiability game, they were able to forge a proof for a dishonest evaluation. Consequently, we can reduce security to the soundness of the NIZK: an adversary simulates **Request** and **Finalize** oracle for  $\mathcal{A}_{\text{vf}}$ , and, once  $\mathcal{A}_{\text{vf}}$  terminates by outputting a blind response, we extract the forged proof contained in the response, breaking soundness exactly if  $\mathcal{A}_{\text{vf}}$  wins.

**Multi-key obliviousness.** We established that the regular 2HashDH is correct, but not implicitly verifiable, so from Lemma 9 it follows that multi-key obliviousness is not achieved.

**Corollary 3.** *2HashDH is not obliv-k+.*

On the other hand, after adding a proof of honest behavior in **vf-HashDH**, implicit verifiability is fulfilled, so taken together with **obliv-1k** we get **obliv-k+** by Lemma 11.

## I Generic attack on verifiability definition by [ADDG24]

Fig. 31 repeats the definition by Albrecht et al. [ADDG24] that we based **vf** on. As mentioned in Section 5.2, the lack of checks on the adversary-chosen keys  $sk$  and  $pk$  leaves room for a non-matching keys attack, shown in the same figure.

In the beginning of the **vf** game,  $\mathcal{A}$  commits to a secret and public key as well as to an input  $x$ . A challenge request is then generated from  $pk$  and  $x$ . The goal of the attacker is to provide a response  $\bar{y}$ , that, once finalized, differs from the correct evaluation of  $x$  under the pre-chosen  $sk$ .

This version of verifiability can trivially be won by  $\mathcal{A}$  outputting a secret key  $sk_1$  and public key  $pk_0$  that do not match each other. As a response,  $\mathcal{A}$  then simply computes the blind evaluation of  $\bar{y}$  using the  $sk_0$  belonging to the

| $\text{Game}_{\Pi, \mathcal{A}}^{\text{vf}}(\lambda)$                                  | $\mathcal{A}(pp)$                                    |
|----------------------------------------------------------------------------------------|------------------------------------------------------|
| $pp \leftarrow \text{Setup}(\lambda)$                                                  | $(pk_0, sk_0) \leftarrow \text{KGen}(pp)$            |
| $(sk, pk, x) \leftarrow \mathcal{A}(pp)$                                               | $(pk_1, sk_1) \leftarrow \text{KGen}(pp)$            |
| $(st, \bar{x}) \leftarrow \text{Request}(pk, x)$                                       | $x \leftarrow \mathcal{X}$                           |
| $\bar{y} \leftarrow \mathcal{A}^{\text{Request}, \text{Finalize}, \text{RO}}(\bar{x})$ | Send $(sk_1, pk_0, x)$ , receive $\bar{x}$           |
| $y' \leftarrow \text{Finalize}(\bar{y}, st)$                                           | $\bar{y} \leftarrow \text{BlindEval}(sk_0, \bar{x})$ |
| $y \leftarrow \text{Eval}(sk, x)$                                                      | <b>return</b> $\bar{y}$                              |
| <b>return</b> $y' \neq y$                                                              |                                                      |

**Fig. 31. Left:** Security game by Albrecht et al. [ADDG24] for verifiability, syntax adapted. **Right:** Non-matching-keys attack on the same game.

announced public key. With an overwhelming probability, this evaluation will be distinct from the “correct” evaluation with  $sk_1$ , and  $\mathcal{A}$  wins.

Note how even binding via a ZKP the request to  $pk_0$ , as in vf-HashDH and vf-2HashDH, does not inhibit the attack, because the winning blind evaluation indeed involved  $sk_0$ .

The remedy is fortunately simple – as shown in Section 5.2, we just need to make sure that the announced keys match each other.

## J Generic construction of a strongly pseudorandom OPRF

Recall Theorem 11 constructed a strongly pseudorandom OPRF  $\Pi_{\text{prnd}}$  as follows: let  $\Pi_{\text{unpred}} = (\text{Setup}, \text{KGen}, \text{Eval}, \text{Request}, \text{BlindEval}, \text{Finalize})$  be an OPRF that achieves unpred and  $H$  be a random oracle that maps from the output space of  $\Pi_{\text{unpred}}$  to strings of length  $n$ . Let  $\Pi_{\text{prnd}}$  be identical to  $\Pi_{\text{unpred}}$  except the following modified algorithms:

$\text{Eval}'(sk, x) : \text{return } H(x, \text{Eval}(sk, x))$   
 $\text{Request}'(pk, x) : (\bar{x}, st) \leftarrow \text{Request}(pk, x); \text{return } (\bar{x}, (st, x))$   
 $\text{Finalize}'(\bar{y}, st) : \text{parse } (st', x) = st; y' \leftarrow \text{Finalize}(\bar{y}, st'); \text{ if } y' = \perp : \text{return } \perp;$   
                   **else return**  $H(x, y')$

**Security.** We show any such  $\Pi_{\text{prnd}}$  is strongly pseudorandom, which follows closely the str-prnd proof of 2HashDH given in Appendix H.3.

*Proof.* As before, we do two game hops, shown in Fig. 32.  $G_0$  is str-prnd in the real world: the Eval oracle returns evaluations of  $\Pi_{\text{prnd}}$  and the BlindEval oracle returns blind evaluations of the inner OPRF  $\Pi_{\text{unpred}}$  (which are by construction also the blind evaluations of  $\Pi_{\text{prnd}}$ ). The random oracle returns random values by lazy sampling.

In  $G_1$ , we change  $\mathcal{O}_{\text{Eval}}$  to return truly random values from the output space of  $H$ . There is no statistical change between  $G_0$  and  $G_1$ : the random values in  $G_1$

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $G_0, \Pi, \mathcal{A}(\lambda)$<br>$pp \leftarrow \$ \Pi_{\text{prnd}}.\text{Setup}'(\lambda)$<br>$(pk, sk) \leftarrow \$ \Pi_{\text{prnd}}.\text{KGen}'(pp)$<br>$R := []$<br>$b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Eval}}, \mathcal{O}_{\text{BlindEval}}, \mathcal{O}_{\text{H}}}(pp, pk)$<br><b>return</b> $b'$<br><hr/> $\mathcal{O}_{\text{Eval}}(x)$<br><b>return</b> $\Pi_{\text{prnd}}.\text{Eval}'(sk, x)$<br><hr/> $\mathcal{O}_{\text{BlindEval}}(\bar{x})$<br><b>return</b> $\Pi_{\text{unpred}}.\text{BlindEval}(sk, \bar{x})$<br><hr/> $\mathcal{O}_{\text{H}}(x, y')$<br><b>if</b> $(x, y') \notin R$ :<br>$R[x, y'] \leftarrow \$ \{0, 1\}^n$<br><b>return</b> $R[x, y']$ | $G_1, \Pi, \mathcal{A}(\lambda)$<br>$E := []$<br>// otherwise unchanged<br><hr/> $\mathcal{O}_{\text{Eval}}(x)$<br><b>if</b> $x \notin E : E[x] \leftarrow \$ \{0, 1\}^n$<br><b>return</b> $E[x]$<br><hr/> $\mathcal{O}_{\text{H}}(x, y')$<br><b>if</b> $(x, y') \notin R$ :<br><b>if</b> $y' = \Pi_{\text{unpred}}.\text{Eval}(sk, x)$ :<br>$R[x, y'] \leftarrow \mathcal{O}_{\text{Eval}}(x)$<br><b>else</b> :<br>$R[x, y'] \leftarrow \$ \{0, 1\}^n$<br><b>return</b> $R[x, y']$ | $G_2, \Pi, \mathcal{A}(\lambda)$<br>$C_{\text{BE}}, C_{\text{Fin}} := 0$<br>$\text{bad} := 0$<br>// otherwise unchanged<br><hr/> $\mathcal{O}_{\text{BlindEval}}(\bar{x})$<br>$C_{\text{BE}} := C_{\text{BE}} + 1$<br>// otherwise unchanged<br><hr/> $\mathcal{O}_{\text{H}}(x, y')$<br><b>if</b> $(x, y') \notin R$ :<br><b>if</b> $y' = \Pi_{\text{unpred}}.\text{Eval}(sk, x)$ :<br>$C_{\text{Fin}} := C_{\text{Fin}} + 1$<br><b>if</b> $C_{\text{Fin}} > C_{\text{BE}}$ :<br>$\text{bad} := 1$<br><b>return</b> $\perp$<br>$R[x, y'] \leftarrow \mathcal{O}_{\text{Eval}}(x)$<br><b>else</b> : $R[x, y'] \leftarrow \$ \{0, 1\}^n$<br><b>return</b> $R[x, y']$ |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Fig. 32.** Games  $G_0, G_1, G_2$  for proving the generic construction from Theorem 11 is strongly pseudorandom. For readability, we write explicitly if algorithms of the inner OPRF  $\Pi_{\text{unpred}}$  or outer OPRF  $\Pi_{\text{prnd}}$  are used.

are identically distributed to the evaluations in  $G_0$ , because the final operation of the regular **Eval** algorithm is **H**, too. We only need to fix consistency, because the adversary has the option to obtain evaluations by using the **BlindEval** oracle and finalizing the returned response  $\bar{y}$ . According to the **Finalize'** algorithm, the adversary needs to hash  $x$  together with the evaluation of  $x$  by the inner OPRF. We program the random oracle to check for such inputs and return a value consistent with other evaluations in this case. Again, this does not change the distribution of the game.

$G_1$  is already almost the random world of **str-prnd** game, but currently the random oracle hands out unlimited evaluations of  $\Pi_{\text{prnd}}$ . This is not something we would intuitively expect from a strongly pseudorandom OPRF. In the strong pseudorandomness game, this is fixed by  $\mathcal{O}_{\text{LimEval}}$ : the simulator takes care of  $\mathcal{O}_{\text{BlindEval}}$  and  $\mathcal{O}_{\text{H}}$  with the help of  $\mathcal{O}_{\text{LimEval}}$ , but the limited oracle allows the simulator to only retrieve as many evaluations as there have been **BlindEval** queries. To arrive at a game identical to the random world of **str-prnd**, we therefore transition to  $G_2$ . Here, a **bad** flag is set and  $\perp$  is output from  $\mathcal{O}_{\text{H}}$  if we are about to leak an additional evaluation.

The **bad** event only happens if the adversary sent us more evaluations of  $\Pi_{\text{unpred}}$  than they obtained from  $\mathcal{O}_{\text{BlindEval}}$  and luckily, since  $\Pi_{\text{unpred}}$  is unpredictable, this cannot happen except with negligible probability. Formally, we construct an adversary  $\mathcal{A}_{\text{str-unpred}}$  that wins the **str-unpred** game if the event **bad** happens, which is straightforward. We show  $\mathcal{A}_{\text{str-unpred}}$  in Fig. 33.

| Adversary $\mathcal{A}_{\text{str-unpred}}^{\mathcal{O}_{\text{BlindEval}}^{\text{str-unpred}}, \mathcal{O}_{\text{Vf}}^{\text{str-unpred}}}(pp, pk)$                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | $\mathcal{O}_{\text{H}}(x, y')$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $c_{\text{BE}}, c_{\text{Fin}} := 0; \quad \mathbf{E}, \mathbf{R} := []; \quad \text{out} := ()$<br>Start $\mathcal{A}_{\text{Eval}}^{\mathcal{O}_{\text{Eval}}^{\text{str-prnd}}, \mathcal{O}_{\text{BlindEval}}^{\text{str-prnd}}, \mathcal{O}_{\text{H}}}(pp, pk)$<br>$\mathcal{O}_{\text{Eval}}^{\text{str-prnd}}(x)$<br><b>if</b> $x \notin \mathbf{E} : \mathbf{E}[x] \leftarrow \$ \{0, 1\}^n$<br><b>return</b> $\mathbf{E}[x]$<br>$\mathcal{O}_{\text{BlindEval}}^{\text{str-prnd}}(\bar{x})$<br>$c_{\text{BE}} := c_{\text{BE}} + 1$<br><b>return</b> $\mathcal{O}_{\text{BlindEval}}^{\text{str-unpred}}(\bar{x})$ | $\mathbf{if} (x, y') \notin \mathbf{R} :$<br>$\mathbf{if} \mathcal{O}_{\text{Vf}}^{\text{str-prnd}}(x, y') = 1 :$<br>$c_{\text{Fin}} := c_{\text{Fin}} + 1$<br>Add $(x, y')$ to out<br><b>if</b> $c_{\text{BE}} \geq c_{\text{Fin}} : \mathbf{R}[x, y'] \leftarrow \mathcal{O}_{\text{Eval}}^{\text{str-prnd}}(x)$<br><b>else</b> : // Found a one-more forgery<br>$\mathbf{abort} \mathcal{A}_{\text{prnd}}$<br>Output out<br><b>else</b> : $\mathbf{R}[x, y'] \leftarrow \$ \{0, 1\}^n$<br><b>return</b> $\mathbf{R}[x, y']$ |

**Fig. 33.** Adversary  $\mathcal{A}_{\text{str-unpred}}$  to bound game hop  $G_1$  to  $G_2$ .

$\mathcal{A}_{\text{str-unpred}}$  simulates  $G_2$  using their own  $\text{BlindEval}$  oracle for blind evaluations, and generating random values of  $\mathcal{O}_{\text{Eval}}^{\text{prnd}}$  and  $\mathcal{O}_{\text{H}}^{\text{prnd}}$  themselves. Whenever they see an evaluation of  $\Pi_{\text{prnd}}$  being finalized in the hash oracle, they add it to an internal list of known evaluations. Here, it becomes apparent why we need the unpredictability variant that includes a verification oracle: the adversary needs a way to recognize  $\Pi_{\text{unpred}}$  evaluations. As soon as the bad event happens,  $\mathcal{A}_{\text{str-unpred}}$  aborts  $\mathcal{A}_{\text{prnd}}$  and outputs the list of evaluations they collected.  $\mathcal{A}_{\text{str-unpred}}$  wins because each pair is a unique, valid evaluation, and the number of pairs surpasses the number of  $\mathcal{O}_{\text{BlindEval}}$  calls.

| S.Setup( $pp$ )                                                                                                                | S.BlindEval $^{\mathcal{O}_{\text{LimEval}}(\bar{x}, st_S)}$                                                                                     | S.H $^{\mathcal{O}_{\text{LimEval}}(x, y', st_S)}$                                                                                                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $(pk, sk) \leftarrow \$ \Pi_{\text{prnd}}.\text{KGen}(pp)$<br>$\mathbf{R} := []$<br><b>return</b> $((pp, sk, \mathbf{R}), pk)$ | Parse $(pp, sk, \mathbf{R}) = st_S$<br>$\bar{y} \leftarrow \Pi_{\text{unpred}}.\text{BlindEval}(sk, \bar{x})$<br><b>return</b> $(\bar{y}, st_S)$ | Parse $(pp, sk, \mathbf{R}) = st_S$<br><b>if</b> $(x, y') \notin \mathbf{R} :$<br>$\mathbf{if} y' = \Pi_{\text{unpred}}.\text{Eval}(sk, x) :$<br>$\mathbf{R}[x, y'] \leftarrow \mathcal{O}_{\text{LimEval}}(x)$<br><b>else</b> :<br>$\mathbf{R}[x, y'] \leftarrow \$ \{0, 1\}^n$<br><b>return</b> $(\mathbf{R}[x, y'], st_S)$ |

**Fig. 34.** Simulator  $S_{\text{gen}}$  for str-prnd of the generic construction  $\Pi_{\text{prnd}}$ .

Finally  $G_2$  is identical to the random world of str-prnd with the simulator  $S_{\text{gen}} = (\text{S.Setup}, \text{S.BlindEval}, \text{S.Prim})$  given in Fig. 34. The overall difference between  $G_0$  and  $G_2$  corresponds to the advantage of an adversary  $\mathcal{A}_{\text{prnd}}$  in str-prnd against  $\Pi_{\text{prnd}}$ , with simulator  $S_{\text{gen}}$ . Hence, we get the following bound:

$$\text{Adv}_{\Pi_{\text{prnd}}, \mathcal{A}_{\text{prnd}}, S_{\text{gen}}}^{\text{str-prnd}}(\lambda) \leq \text{Adv}_{\Pi_{\text{unpred}}, \mathcal{A}_{\text{str-unpred}}}^{\text{str-unpred}}(\lambda)$$

□

**Privacy.** Next we analyze the privacy properties of  $\Pi_{\text{str-prnd}}$ . The **obliv-k+** of  $\Pi_{\text{prnd}}$  easily follows from **obliv-k+** of  $\Pi_{\text{unpred}}$ .

*Proof.* In particular, given a **obliv-k+** adversary  $\mathcal{A}$  against  $\Pi_{\text{prnd}}$ , we can build a **obliv-k+** adversary  $\mathcal{B}$  against  $\Pi_{\text{unpred}}$  as follows.  $\mathcal{B}$  simply echoes almost all messages between the **obliv-k+** challenger and  $\mathcal{A}$ : it forwards the  $pp$  and  $\mathcal{O}_{\text{Request}}$  queries. While forwarding  $\mathcal{O}_{\text{Request}}$  queries,  $\mathcal{B}$  also stores the inputs  $x_{i,0}$  and  $x_{i,1}$  to use later in  $\mathcal{O}_{\text{Finalize}}$  queries. For a  $\mathcal{O}_{\text{Finalize}}$  query,  $\mathcal{B}$  receives the response of the challenger. If the challenger returns  $\perp$ , then  $\mathcal{B}$  returns  $\perp$  to  $\mathcal{A}$  as well. Otherwise, if the challenger returns  $(y_{i,0}, y_{i,1})$ , then  $\mathcal{B}$  returns  $(H(x_{i,0}, y_{i,0}), H(x_{i,1}, y_{i,1}))$  to  $\mathcal{A}$ . Finally, when  $\mathcal{A}$  returns its guess  $b'$ ,  $\mathcal{B}$  forwards it directly to the challenger.  $\mathcal{B}$  simulates  $\mathcal{A}$ 's view perfectly by relying on the same challenge bit as the challenger, so:

$$\text{Adv}_{\Pi_{\text{prnd}}, \mathcal{A}}^{\text{obliv-k+}}(\lambda) \leq \text{Adv}_{\Pi_{\text{unpred}}, \mathcal{B}}^{\text{obliv-k+}}(\lambda)$$

□